

C语言从入门到精通

刚好遇见你

- 目标读者：**编程初学者、大学生、想巩固基础的开发者
- 核心价值：**系统化学习 C 语言，理解底层原理，为后续学习打下坚实基础

这份学习计划专为你设计，每天的内容都是精心安排的，包含**重点知识点讲解**、**知识点实战例题（含解题思路）**、**进阶易错点**、**作业练习**（含挑战题）和**自主探索**；每天文末提供**参考答案**，建议先独立完成再对照。开发环境统一推荐使用 **Dev-C++**，降低入门门槛。

学习建议：先看知识点 → 跟做「实战例题」→ 独立完成作业 → 最后翻到当天「参考答案」核对。

整体学习计划

第一周：基础入门与核心语法

天数	主题	主要内容	参考教程
Day 1	环境搭建与 Hello World	Dev-C++安装与操作、第一个C程序、编译链接执行	01-C语言入门 、 02-开发环境搭建 、 03-程序执行原理
Day 2	数据类型与变量	基本数据类型、变量声明与初始化、常量	04-数据类型与变量
Day 3	运算符与表达式	算术运算符、关系运算符、逻辑运算符、优先级	05-运算符与表达式
Day 4	流程控制（上）	if-else语句、条件运算符、简单程序设计	06-流程控制
Day 5	流程控制（下）	switch语句、综合练习、菜单程序	06-流程控制
Day 6	循环结构（上）	while循环、do-while循环、循环应用	07-循环结构
Day 7	循环结构（下）	for循环、循环嵌套、break/continue	07-循环结构

第二周：核心概念与实战应用

天数	主题	主要内容	参考教程
Day 8	数组	数组声明与初始化、数组操作、数组作为函数参数	08-数组与字符串
Day 9	字符串	字符串概念、字符串处理函数、字符串操作	08-数组与字符串
Day 10	函数	函数定义与调用、参数传递、函数原型、递归	09-函数
Day 11	指针 (上)	指针概念、取地址与解引用、指针与数组	10-指针
Day 12	指针 (下)	指针与函数、动态内存分配、指针高级应用	10-指针 、 11-内存管理
Day 13	结构体	结构体定义与使用、结构体数组、结构体指针	12-结构体与共用体
Day 14	综合实战	三个实战项目、知识回顾与总结	13-文件操作 、 14-综合实战 、 15-学习路线

📖 详细学习内容

Day 1：环境搭建与Hello World

今日目标：安装并熟练使用 Dev-C++，编写、编译、运行并简单调试第一个 C 程序

重点知识点讲解

1.1 为什么推荐 Dev-C++

新生学习 C 语言，最怕一上来就被「编译器 + 编辑器 + 环境变量」劝退。**Dev-C++** 把编辑器、GCC 编译器、调试器打包在一起，安装完成后即可写代码、按快捷键运行，非常适合两周入门。

方案	特点	本计划建议
Dev-C++ (推荐)	一站式 IDE，自带 MinGW/GCC	默认使用
VS Code + MinGW	灵活强大，但配置步骤多	入门后再学也可
Visual Studio	功能全、体积大	学校要求再用

本计划主推 **Embarcadero Dev-C++** (Windows)。若你已安装 **Orwell Dev-C++ 5.11**，操作几乎相同，可直接从「1.3 操作步骤」开始。

说明：Dev-C++ 主要面向 Windows。若你使用 macOS/Linux，可临时使用系统自带的 `gcc` / `clang` + 任意编辑器，但课程演示与截图思路仍以 Dev-C++ 为准。

1.2 Dev-C++ 详细安装步骤 (Windows)

步骤 1: 下载安装包

1. 打开浏览器，搜索 `Embarcadero Dev-C++`，或访问 SourceForge / Embarcadero 官方免费页面下载。
2. 选择较新的 Windows 安装包（文件名通常类似 `Dev-Cpp x.x.x Setup.exe`）。
3. 下载到容易找到的位置（如桌面或「下载」文件夹）。

步骤 2: 运行安装程序

1. 双击安装包。若弹出「用户账户控制 (UAC)」提示，点击是。
2. 选择安装语言（可先选 English，装好后可在软件内改中文界面；若安装包提供中文也可直接选中文）。
3. 阅读许可协议，勾选同意后继续。

步骤 3: 选择安装路径 (重要)

1. 建议安装到简单路径，例如：`C:\Dev-Cpp`
2. 尽量避免：
 - 路径中含中文（如 `C:\编程工具\Dev-Cpp`）
 - 路径中含空格过多的复杂目录（个别旧环境可能出问题）
3. 其余选项保持默认即可（创建桌面快捷方式建议勾选）。
4. 点击安装，等待进度条完成，最后点击完成/Finish。

步骤 4: 首次启动检查

1. 从桌面快捷方式或开始菜单打开 **Dev-C++**。
2. 若首次运行弹出编译器检测 / 配置向导：
 - 选择自带的 **MinGW / TDM-GCC**（以安装包实际名称为准）
 - 点击检查或确定，确保能检测到 `gcc.exe`
3. 菜单大致位置（不同版本用词可能略有差异）：
 - `Tools` → `Compiler Options`：编译器相关
 - `Tools` → `Editor Options`：编辑器字体、Tab 宽度（建议 Tab=4）

步骤 5: 界面语言与编码建议 (可选)

1. 若菜单是英文，可在 `Tools` → `Environment Options` 中查找 Language，改为中文（若有语言包）。
2. 源文件建议统一使用 **UTF-8** 保存，减少中文注释/输出乱码概率。
3. 字体可调大一些（如 Consolas / 微软雅黑，字号 12~14），长时间写代码更轻松。

安装成功的标志：能打开 Dev-C++，且之后能编译运行 Hello World（见下一节）。

1.3 Dev-C++ 操作步骤：从零到 Hello World

方式 A: 直接新建源文件 (最快，推荐第一天使用)

1. 菜单：`File` → `New` → `Source File`（或按 `Ctrl+N`）。
2. 在空白编辑区输入下面的程序。
3. 菜单：`File` → `Save As...`，建议先建文件夹，例如：`D:\C_Learning\day01\`
4. 文件名设为 `hello.c`（扩展名必须是 `.c`，不要存成 `.cpp` 除非你明确在写 C++）。
5. 编译：按 `F9`，或菜单 `Execute` → `Compile`。

- 下方编译日志出现 `Compilation successful` / 成功，即表示通过。

6. 运行：按 `F10`，或 `Execute` → `Run`。

7. 更常用：按 `F11`（Compile & Run，先编译再运行）。

方式 B：新建控制台项目（更接近正式工程）

1. `File` → `New` → `Project...`
2. 选择 **Console Application**，语言选 **C**
3. 输入项目名（如 `HelloWorld`），选择保存目录
4. 在自动生成的 `main.c` 中改写代码，同样用 `F9` / `F10` / `F11`

Hello World 程序：

```
#include <stdio.h>

int main(void) {
    printf("Hello, World!\n");
    return 0;
}
```

代码解析：

- `#include <stdio.h>`：引入标准输入输出库，才能使用 `printf` / `scanf`
- `int main(void)`：程序入口；`void` 表示暂时不接收命令行参数
- `printf(...)`：向屏幕输出；`\n` 表示换行
- `return 0`：返回给操作系统，`0` 通常表示正常结束

运行后你会看到：黑色控制台窗口打印 `Hello, World!`。部分版本窗口一闪而过——可在程序末尾加一句：

```
system("pause"); /* 需要 #include <stdlib.h>, 仅用于本地暂停，正式作业可不写 */
```

或养成习惯：看完输出后看编译日志区，不要立刻关窗口。

1.4 读懂编译错误（必会）

故意把代码改错，例如删掉分号：

```
printf("Hello, World!\n")
```

按 `F9` 后，下方会提示类似 `error: expected ';' before ...`，并给出**行号**。双击错误信息通常能跳到对应行。

习惯：先看第一条错误 → 改 → 再编译。不要一次改十处却不重新编译。

1.5 简单调试（断点与单步）

1. 在某一左侧空白处单击，出现红点 = **断点**（程序运行到这里会暂停）。
2. 使用调试运行（常见为菜单 `Debug` → `Debug` / 快捷键，视版本而定；有的版本用 `F5` 或工具栏虫子图标）。
3. 常用操作：
 - **Step Into / 逐过程进入**：进入函数内部（常见 `F7`）
 - **Step Over / 逐过程跳过**：执行当前行但不进入函数（常见 `F8`）

- **Continue / 继续**：跑到下一个断点

4. 在调试面板中查看变量当前值（CPU 窗口 / Debug 窗口，名称因版本而异）。

第一天目标：会设断点、能单步、能看到变量变化即可，不必精通全部调试菜单。

1.6 程序如何变成可执行文件（加深）

写出 `.c` 源文件后，Dev-C++ 背后大致做了三件事：

1. **编译 (Compile)**：把 C 代码翻译成目标文件（`.o` / `.obj`）
2. **链接 (Link)**：把目标文件和标准库拼成可执行文件（`.exe`）
3. **执行 (Run)**：操作系统加载 `.exe` 运行

因此：改了代码却觉得「没变化」，多半是忘了**重新编译**。养成 `F11` 的习惯。

1.7 补充：命令行方式（了解即可）

Dev-C++ 自带的 `gcc` 也可在命令行使用（需把其 `bin` 目录加入 PATH，或在「Dev-C++ 终端」中操作）。了解下列命令有助于理解 IDE 在做什么：

```
gcc hello.c -o hello.exe
hello.exe
```

本计划日常仍以 Dev-C++ 图形界面为主。

知识点实战例题

例题1：格式化输出与转义字符

题目：编写程序，输出如下三行（注意对齐与换行）：

```
Name:    Alice
ID:      2026001
Score:   95
```

解题思路：

1. 需要多次 `printf`，或一次 `printf` 中写多个 `\n`
2. 列对齐可用多个空格，或使用 `\t` 制表符（制表位取决于控制台设置）
3. 先写第一行跑通，再复制改成三行，最后微调空格

完整代码：

```
#include <stdio.h>

int main(void) {
    printf("Name:\tAlice\n");
    printf("ID:\t2026001\n");
    printf("Score:\t95\n");
    return 0;
}
```

要点/易错：

- `\n` 只换行，不会自动对齐；对齐靠空格或 `\t`
- 字符串里要输出 `\` 本身需写成 `\\`
- 改完务必重新 `F11` 编译运行

例题2: scanf 读入并计算

题目：从键盘输入长和宽（整数），输出矩形周长与面积。

解题思路：

1. 定义两个 `int` 变量存放长、宽
2. 用 `scanf` 读入，变量前加 `&`
3. 周长 = (长 + 宽) * 2，面积 = 长 * 宽
4. 用 `printf` 输出结果

完整代码：

```
#include <stdio.h>

int main(void) {
    int length, width;
    printf("请输入长和宽（用空格分隔）：");
    scanf("%d %d", &length, &width);
    printf("周长 = %d\n", (length + width) * 2);
    printf("面积 = %d\n", length * width);
    return 0;
}
```

要点/易错：

- 忘写 `&` 是新手最常见错误之一
- `scanf` 格式个数要与变量个数一致
- 输入时用空格或回车分隔均可

作业练习

先独立完成，做完再到本节文末「参考答案」对照。

练习1：完成环境验证

- 按上文安装 Dev-C++
- 新建 `hello.c`，编译运行成功
- 故意制造一个语法错误，学会根据行号定位并修复

练习2：输出个人信息

- 创建 `info.c`
- 用多次 `printf` 输出姓名、拟报专业、兴趣
- 在 Dev-C++ 中 `F11` 运行

练习3：计算两个数的和

- 创建 `sum.c`
- 使用 `scanf` 输入两个整数，输出它们的和
- 注意：`scanf("%d", &a);` 中的 `&` 不能丢

挑战题：个人信息卡片

- 使用 `\n`、`\t` 和若干 `=` / `-` 字符，打印一个整齐的「名片」边框
- 要求至少 5 行输出，左右对齐看起来像卡片

常见错误与调试提示

- **错误：**找不到编译器 / `gcc` 相关报错
 - **解决：**重装时勾选自带编译器；`Tools` → `Compiler Options` 中确认编译器路径指向安装目录下的 MinGW
- **错误：**中文输出乱码
 - **解决：**源文件改为 UTF-8 保存；或临时少用中文，先用英文验证环境；控制台也可试 `chcp 65001`
- **错误：**程序一闪而过看不清输出
 - **解决：**用 `F11` 后查看；或调试运行；或临时 `system("pause")`
- **错误：**保存成了 `.cpp` 或 `.txt`
 - **解决：**另存为时文件类型选 All files，文件名写 `xxx.c`
- **错误：**`scanf` 输入后结果不对
 - **解决：**检查格式串是否与类型匹配，变量前是否写了 `&`

自主探索

1. 在 Dev-C++ 中尝试修改字体、主题，找到自己舒服的编辑环境
2. 查一查 `\n`、`\t`、`\\`、`\"` 转义字符的含义并写个小 demo
3. 搜索了解 C 语言简史 (K&R、ANSI C、C99)，写 200 ~ 300 字笔记

参考答案

练习1 参考答案 (hello.c)

思路：最小可运行程序；故意删分号观察报错行号，再改回。

```
#include <stdio.h>

int main(void) {
    printf("Hello, World!\n");
    return 0;
}
```

练习2 参考答案 (info.c)

```
#include <stdio.h>

int main(void) {
    printf("姓名: 张三\n");
    printf("拟报专业: 计算机科学与技术\n");
    printf("兴趣: 编程、篮球\n");
    return 0;
}
```

练习3 参考答案 (sum.c)

思路：定义两个变量 → `scanf` 读入（带 `&`）→ 求和输出。

```
#include <stdio.h>

int main(void) {
    int a, b;
    printf("请输入两个整数: ");
    scanf("%d %d", &a, &b);
    printf("%d + %d = %d\n", a, b, a + b);
    return 0;
}
```

挑战题参考答案

思路：先用 `=` 打出上下边框，中间用 `printf` 输出字段，注意每行宽度大致一致。

```
#include <stdio.h>

int main(void) {
    printf("=====\n");
    printf("| Name:\tXiaoming |\n");
    printf("| Major:\tCS      |\n");
    printf("| Hobby:\tCoding  |\n");
    printf("=====\n");
    return 0;
}
```

Day 2: 数据类型与变量

今日目标：理解C语言的数据类型系统，学会定义和使用变量

重点知识点讲解

2.1 基本数据类型

类型	关键字	大小（字节）	范围
字符型	char	1	-128~127
短整型	short	2	-32768~32767
整型	int	4	-21亿~21亿
长整型	long	4或8	取决于系统
单精度浮点	float	4	约 $\pm 3.4 \times 10^{38}$
双精度浮点	double	8	约 $\pm 1.7 \times 10^{308}$

2.2 变量的声明与初始化

```
// 声明并初始化
int age = 25;
float height = 1.75f;
char gender = 'M';

// 先声明后赋值
int score;
score = 95;

// 多个变量同时声明
int a = 1, b = 2, c = 3;
```

2.3 变量命名规则

- 1. 必须以字母或下划线开头
- 2. 只能包含字母、数字和下划线
- 3. 不能是C语言关键字
- 4. 区分大小写

2.4 常量

```
// 使用#define定义宏常量
#define PI 3.14159
#define MAX_SIZE 100

// 使用const定义常量
const float GRAVITY = 9.8f;
```

2.5 进阶：sizeof、符号与格式匹配（易错）

```
printf("%zu\n", sizeof(int)); /* 打印类型占用字节数 */
unsigned int u = 1;
int s = -1;

/* 有符号与无符号混用时可能出现“惊人”的比较结果，初学阶段尽量统一用 int */
```

要点:

1. `sizeof`: 得到类型或变量的字节数, 结果类型与平台有关, 打印时常用 `%zu` (若编译器不支持可先用 `%d` 并强转 `(int)sizeof(int)` 做实验)。
2. **初始化**: 局部变量若不初始化, 值是不定的 (未定义行为), 务必养成初始化习惯。
3. `printf` / `scanf` **格式必须匹配类型**: `int` 用 `%d`, `float` 用 `%f`, `double` 用 `%lf` (`scanf`) / `%f` 或 `%lf` (`printf`, C99 起对 `double` 也常用 `%f`)。类型与格式不匹配是隐蔽 bug 的常见来源。
4. **整型提升 (了解)**: `char`、`short` 参与运算时, 往往会先提升为 `int` 再计算。

知识点实战例题

例题1: 常量与浮点计算

题目: 用 `#define` 定义税率 `TAX_RATE` 为 `0.13`, 输入商品不含税价格 (浮点数), 输出税额与含税价 (保留两位小数)。

解题思路:

1. 用宏定义税率, 避免魔法数字散落代码中
2. `scanf("%f", &price)` 读入价格
3. 税额 = 价格 × 税率; 含税价 = 价格 + 税额
4. `%.2f` 控制小数位

完整代码:

```
#include <stdio.h>

#define TAX_RATE 0.13

int main(void) {
    float price, tax, total;
    printf("请输入不含税价格: ");
    scanf("%f", &price);
    tax = price * TAX_RATE;
    total = price + tax;
    printf("税额: %.2f\n", tax);
    printf("含税价: %.2f\n", total);
    return 0;
}
```

要点/易错:

- `float` 用 `%f`; 不要写成 `%d`
- 宏定义一般不加分号: `#define TAX_RATE 0.13` 正确

例题2: 字符与 ASCII

题目: 输入一个大写字母, 输出它的小写形式及其 ASCII 码。

解题思路:

1. 大写转小写: `ch + ('a' - 'A')`, 或直接 `ch + 32` (了解即可)
2. 用 `%c` 读字符、`%d` 打印 ASCII (把 `char` 当整数看)

完整代码：

```
#include <stdio.h>

int main(void) {
    char ch;
    printf("请输入一个大写字母: ");
    scanf(" %c", &ch); /* 前面空格可吃掉残留空白 */
    char lower = ch + ('a' - 'A');
    printf("小写: %c, ASCII: %d\n", lower, lower);
    return 0;
}
```

要点/易错：

- 字符用单引号 `'A'`，字符串用双引号 `"A"`
- `scanf("%c")` 前加空格更稳妥，避免读到上次留下的换行

作业练习

先独立完成，再到文末「参考答案」对照。

练习1：计算圆的周长和面积

- 输入半径
- 计算并输出周长和面积
- 使用 `#define` 定义 π

练习2：温度转换

- 输入华氏温度
- 转换为摄氏温度并输出（公式： $C = (F - 32) * 5 / 9$ ）

练习3：变量作用域实验

- 定义全局变量和局部变量
- 观察变量的可见范围（可在不同函数中打印同名/全局变量）

挑战题：类型大小与溢出观察

1. 用 `sizeof` 打印 `char/short/int/long/float/double` 的字节数
2. 令 `unsigned char x = 255; x = x + 1;`，观察输出（理解回绕）
3. 思考：为什么银行/金额计算要特别小心浮点类型？

常见错误与调试提示

- **错误：**使用未初始化的变量
 - **解决：**养成初始化变量的好习惯，避免未定义行为
- **错误：**浮点精度问题
 - **解决：**浮点数比较时要小心，使用合理的精度范围
- **错误：**类型溢出
 - **解决：**注意数据类型的范围，选择合适的类型

- 错误: scanf 写了 %d 却传入 float 变量
 - 解决: 改格式或改变量类型, 保持一致

自主探索

1. 使用 sizeof 运算符查看不同数据类型的大小
2. 尝试将整数转换为字符, 观察ASCII码的变化
3. 搜索了解 auto、static、extern、register 存储类型的区别

参考答案

练习1 参考答案

思路: 宏定义 PI → 读入半径 → 周长 $2*PI*r$, 面积 $PI*r*r$ → 保留两位小数输出。

```
#include <stdio.h>

#define PI 3.14159

int main(void) {
    float radius;
    printf("请输入圆的半径: ");
    scanf("%f", &radius);
    printf("圆的周长: %.2f\n", 2 * PI * radius);
    printf("圆的面积: %.2f\n", PI * radius * radius);
    return 0;
}
```

练习2 参考答案

```
#include <stdio.h>

int main(void) {
    float f, c;
    printf("请输入华氏温度: ");
    scanf("%f", &f);
    c = (f - 32) * 5 / 9;
    printf("摄氏温度: %.2f\n", c);
    return 0;
}
```

练习3 参考答案

思路: 全局变量在所有函数可见; 局部变量只在定义它的函数/块内可见。

```
#include <stdio.h>

int global_n = 100; /* 全局 */

void show(void) {
    int local_n = 1; /* 局部, 仅 show 内可见 */
    printf("in show: global=%d, local=%d\n", global_n, local_n);
}
```

```

}

int main(void) {
    int local_n = 2;
    printf("in main: global=%d, local=%d\n", global_n, local_n);
    show();
    return 0;
}

```

挑战题参考答案

```

#include <stdio.h>

int main(void) {
    printf("char=%d short=%d int=%d long=%d float=%d double=%d\n",
        (int)sizeof(char), (int)sizeof(short), (int)sizeof(int),
        (int)sizeof(long), (int)sizeof(float), (int)sizeof(double));

    unsigned char x = 255;
    x = x + 1;
    printf("unsigned char 255+1 = %u (回绕到0) \n", (unsigned)x);

    /* 思考: 浮点有精度误差, 金额更适合用整数分/专用小数类型 */
    return 0;
}

```

Day 3: 运算符与表达式

今日目标: 掌握C语言的各种运算符, 理解表达式的计算规则

重点知识点讲解

3.1 算术运算符

```

int a = 10, b = 3;
a + b; // 加法: 13
a - b; // 减法: 7
a * b; // 乘法: 30
a / b; // 除法: 3 (整数除法, 截断小数)
a % b; // 取余: 1

```

3.2 关系运算符

```

a > b; // 大于: 1 (真)
a < b; // 小于: 0 (假)
a >= b; // 大于等于: 1
a <= b; // 小于等于: 0
a == b; // 等于: 0
a != b; // 不等于: 1

```

3.3 逻辑运算符

```
(a > 0) && (b > 0); // 逻辑与：两个都为真才为真
(a > 0) || (b > 0); // 逻辑或：一个为真即为真
!(a > 0);           // 逻辑非：取反
```

3.4 自增自减运算符

```
int x = 5;
int y = ++x; // 前缀自增：先加1再使用, y=6, x=6
int z = x++; // 后缀自增：先使用再加1, z=6, x=7
```

3.5 运算符优先级

- 括号 > 单目运算符 > 算术运算符 > 关系运算符 > 逻辑运算符 > 赋值运算符

拿不准时：**加括号**，可读性远比炫技重要。

3.6 进阶：短路求值与赋值陷阱

短路求值：

- `&&`：左边为假则不再计算右边
- `||`：左边为真则不再计算右边

```
int x = 0;
(x != 0) && (10 / x > 1); /* x==0 时不会执行除法，避免除零 */
```

赋值 vs 比较：

```
if (x = 0) { /* 这是赋值！条件几乎总是假，且改写了 x */ }
if (x == 0) { /* 这才是比较 */ }
```

逗号运算符（了解）：`a = (1, 2, 3);` 结果为 `3`，日常少用，读代码时能认出即可。

知识点实战例题

例题1：整数除法与取余

题目：输入总秒数，换算成「时:分:秒」输出（例如 `3661` → `1:1:1`）。

解题思路：

- 小时 = 总秒 / 3600
- 剩余秒 = 总秒 % 3600
- 分钟 = 剩余 / 60, 秒 = 剩余 % 60

完整代码：

```
#include <stdio.h>

int main(void) {
    int total, h, m, s;
    printf("请输入总秒数: ");
    scanf("%d", &total);
    h = total / 3600;
    m = (total % 3600) / 60;
    s = total % 60;
    printf("%d:%d:%d\n", h, m, s);
    return 0;
}
```

要点/易错:

- `/` 对整数是截断除法；要小数需写成浮点运算
- `%` 只能用于整数

例题2：前缀/后缀自增

题目：设 `x=5`，分别计算并打印 `a=++x` 与 `b=x++` 后 `a,b,x` 的值（分两段实验，每次从 `x=5` 开始）。

解题思路：

1. `++x`：先加再取值
2. `x++`：先取值再加
3. 分开写两段，避免在同一表达式里多次改 `x`

完整代码：

```
#include <stdio.h>

int main(void) {
    int x = 5, a, b;
    a = ++x;
    printf("a=++x → a=%d, x=%d\n", a, x);
    x = 5;
    b = x++;
    printf("b=x++ → b=%d, x=%d\n", b, x);
    return 0;
}
```

要点/易错：不要在一个表达式里对同一变量多次 `++/--`（未定义行为）。

作业练习

先独立完成，再到文末「参考答案」对照。

练习1：判断闰年

- 输入年份
- 判断是否为闰年（能被4整除但不能被100整除，或能被400整除）

练习2：计算BMI指数

- 输入体重(kg)和身高(m)
- 计算BMI并判断等级（可参考：<18.5偏瘦，18.5~24正常，24~28偏胖，≥28肥胖）

练习3：位运算权限控制

- 用一个 `unsigned int flags` 表示权限
- 约定：第 0 位=读，第 1 位=写，第 2 位=执行
- 练习：打开某权限、关闭某权限、判断是否拥有某权限（`|`、`&`、`~`）

挑战题：

1. 不用临时变量，仅用算术或异或交换两个整数，并说明各自局限
2. 写一段代码验证短路：在 `&&` / `||` 右侧放入带 `printf` 的表达式，观察是否打印

常见错误与调试提示

- **错误：**使用 `=` 代替 `==`
 - **解决：**将常量放在前面，如 `if (10 == x)`，误写为 `=` 会产生编译错误
- **错误：**整数除法结果不符合预期
 - **解决：**使用强制类型转换实现浮点除法，如 `(float)a / b`
- **错误：**在同一表达式中多次修改同一变量
 - **解决：**这种行为未定义，应避免

自主探索

1. 搜索了解短路求值的概念，编写代码验证
2. 尝试使用位运算实现交换两个数（不使用临时变量）
3. 搜索了解运算符优先级表，总结常用的优先级规则

参考答案

练习1 参考答案

思路：闰年条件写成一个表达式，结果为 1/0，再用 if 输出。

```
#include <stdio.h>

int main(void) {
    int year;
    printf("请输入年份: ");
    scanf("%d", &year);
    int is_leap = (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
    if (is_leap) {
        printf("%d年是闰年\n", year);
    } else {
        printf("%d年不是闰年\n", year);
    }
    return 0;
}
```

练习2 参考答案


```

#include <stdio.h>

int main(void) {
    float weight, height, bmi;
    printf("请输入体重(kg)和身高(m): ");
    scanf("%f %f", &weight, &height);
    bmi = weight / (height * height);
    printf("BMI=%.2f, ", bmi);
    if (bmi < 18.5f) {
        printf("偏瘦\n");
    } else if (bmi < 24.0f) {
        printf("正常\n");
    } else if (bmi < 28.0f) {
        printf("偏胖\n");
    } else {
        printf("肥胖\n");
    }
    return 0;
}

```

练习3 参考答案

```

#include <stdio.h>

#define READ  (1u << 0)
#define WRITE (1u << 1)
#define EXEC  (1u << 2)

int main(void) {
    unsigned int flags = 0;
    flags |= READ;           /* 打开读 */
    flags |= WRITE;          /* 打开写 */
    flags &= ~WRITE;         /* 关闭写 */
    if (flags & READ) {
        printf("拥有读权限\n");
    }
    if (!(flags & WRITE)) {
        printf("没有写权限\n");
    }
    flags |= EXEC;
    printf("flags=%u\n", flags);
    return 0;
}

```

挑战题参考答案

```

#include <stdio.h>

int main(void) {
    int a = 3, b = 5;
    /* 算术交换: 注意溢出风险 */
    a = a + b;
}

```

```
b = a - b;
a = a - b;
printf("算术交换后 a=%d b=%d\n", a, b);

a = 3;
b = 5;
a ^= b;
b ^= a;
a ^= b;
printf("异或交换后 a=%d b=%d\n", a, b);

int x = 0;
(x != 0) && printf("不会打印 (短路) \n");
(x == 0) || printf("也不会打印 (短路) \n");
(x == 0) && printf("会打印\n");
return 0;
}
```

Day 4: 流程控制（上）

今日目标：学会使用if-else语句实现条件判断

重点知识点讲解

4.1 基本if语句

```
if (条件) {
    // 条件为真时执行
}
```

4.2 if-else语句

```
if (条件) {
    // 条件为真时执行
} else {
    // 条件为假时执行
}
```

4.3 嵌套if-else语句

```
if (score >= 90) {
    printf("优秀");
} else if (score >= 80) {
    printf("良好");
} else if (score >= 60) {
    printf("及格");
} else {
    printf("不及格");
}
```

4.4 条件运算符（三元运算符）

```
int max = (a > b) ? a : b;
```

4.5 进阶：Dangling else 与输入合法性

Dangling else（悬挂 else）： `else` 总是匹配最近的、尚未匹配的 `if`。用花括号消除歧义：

```
if (a)
    if (b)
        printf("b");
else    /* 这个 else 匹配的是内层 if (b), 不是 if (a)! */
    printf("?");
```

输入合法性： 用户可能输入负数、超范围数据。关键程序应对 `scanf` 返回值或范围做检查：

```
if (scanf("%d", &score) != 1) {
    printf("输入无效\n");
    return 1;
}
```

知识点实战例题

例题1：成绩等级

题目：输入百分制成绩，输出等级：≥90 优秀，≥80 良好，≥60 及格，否则不及格。非法成绩（<0 或 >100）提示错误。

解题思路：

1. 先校验范围
2. 用 `if-else if` 从高到低判断（注意顺序）
3. 每个分支只打印一种结果

完整代码：

```
#include <stdio.h>

int main(void) {
    int score;
    printf("请输入成绩(0-100): ");
    if (scanf("%d", &score) != 1) {
        printf("输入无效\n");
        return 1;
    }
    if (score < 0 || score > 100) {
        printf("成绩超出范围\n");
    } else if (score >= 90) {
        printf("优秀\n");
    } else if (score >= 80) {
        printf("良好\n");
    } else if (score >= 60) {
```

```
    printf("及格\n");
} else {
    printf("不及格\n");
}
return 0;
}
```

要点/易错：若先写 `>=60` 再写 `>=90`，高分会被误判为及格。

例题2：三元运算符求较大值

题目：输入两个整数，用三元运算符输出较大者。

解题思路： `max = (a > b) ? a : b;`

完整代码：

```
#include <stdio.h>

int main(void) {
    int a, b;
    printf("输入两个整数: ");
    scanf("%d %d", &a, &b);
    printf("较大值=%d\n", (a > b) ? a : b);
    return 0;
}
```

作业练习

先独立完成，再到文末「参考答案」对照。

练习1：判断奇偶性

- 输入一个整数
- 判断是奇数还是偶数

练习2：判断闰年（使用if-else实现）

练习3：简单计算器（加减乘除）

- 输入两个数和运算符
- 使用if-else判断运算符并计算
- 注意处理除数为 0

挑战题：

1. 实现分段函数： `x < 0 → -1` ; `x == 0 → 0` ; `x > 0 → 1` （也可用三元运算符嵌套试写）
2. 输入三个整数，输出从大到小排序后的结果（只用 if-else，不用数组）

常见错误与调试提示

- **错误：** if后面误加分号
 - **解决：** `if (x > 0); { ... }` 会导致块总是执行
- **错误：** 条件表达式逻辑错误

- **解决**：使用括号明确优先级，逐步调试
- **错误**：嵌套过多导致逻辑混乱
 - **解决**：简化条件，使用函数封装复杂逻辑

自主探索

1. 尝试使用三元运算符实现成绩等级判断
2. 搜索了解Dangling Else问题，编写代码验证
3. 尝试实现一个分段函数计算器

参考答案

练习1 参考答案

```
#include <stdio.h>

int main(void) {
    int n;
    printf("请输入整数: ");
    scanf("%d", &n);
    if (n % 2 == 0) {
        printf("偶数\n");
    } else {
        printf("奇数\n");
    }
    return 0;
}
```

练习2 参考答案

```
#include <stdio.h>

int main(void) {
    int year;
    printf("请输入年份: ");
    scanf("%d", &year);
    if (year % 400 == 0) {
        printf("闰年\n");
    } else if (year % 100 == 0) {
        printf("平年\n");
    } else if (year % 4 == 0) {
        printf("闰年\n");
    } else {
        printf("平年\n");
    }
    return 0;
}
```

练习3 参考答案

```
#include <stdio.h>
```

```

int main(void) {
    float num1, num2;
    char op;
    printf("请输入表达式 (如: 3 + 5) : ");
    scanf("%f %c %f", &num1, &op, &num2);
    if (op == '+') {
        printf("%.2f\n", num1 + num2);
    } else if (op == '-') {
        printf("%.2f\n", num1 - num2);
    } else if (op == '*') {
        printf("%.2f\n", num1 * num2);
    } else if (op == '/') {
        if (num2 != 0) {
            printf("%.2f\n", num1 / num2);
        } else {
            printf("除数不能为0\n");
        }
    } else {
        printf("未知运算符\n");
    }
    return 0;
}

```

挑战题参考答案

```

#include <stdio.h>

int main(void) {
    int x, a, b, c, t;
    printf("输入x: ");
    scanf("%d", &x);
    printf("sign=%d\n", (x > 0) ? 1 : ((x < 0) ? -1 : 0));

    printf("输入三个整数: ");
    scanf("%d %d %d", &a, &b, &c);
    if (a < b) { t = a; a = b; b = t; }
    if (a < c) { t = a; a = c; c = t; }
    if (b < c) { t = b; b = c; c = t; }
    printf("%d %d %d\n", a, b, c);
    return 0;
}

```

Day 5: 流程控制（下）

今日目标: 掌握switch语句, 学会编写菜单程序

重点知识点讲解

5.1 switch语句基本语法

```
switch (表达式) {  
    case 常量1:  
        // 语句1  
        break;  
    case 常量2:  
        // 语句2  
        break;  
    default:  
        // 默认语句  
}
```

5.2 switch语句的执行流程

1. 计算switch后面表达式的值
2. 匹配对应的case
3. 执行case后面的语句
4. 遇到break跳出switch

5.3 case穿透

当case后面没有break时，会继续执行下一个case的语句：

```
switch (grade) {  
    case 'A':  
    case 'B':  
    case 'C':  
        printf("成绩合格");  
        break;  
    case 'D':  
    case 'E':  
        printf("成绩不合格");  
        break;  
}
```

5.4 进阶：switch 的限制与有意穿透

1. `switch` 表达式必须是整型或字符型（不能直接 `switch` 浮点数或字符串）。
2. `case` 标签必须是**常量表达式**，不能是变量。
3. **有意穿透**适合「多种输入同一处理」；无意穿透是 bug。每个 `case` 写完先问自己：要不要 `break`？
4. `default` 不是必须，但强烈建议写上，便于发现非法输入。

知识点实战例题

例题1：用 switch 输出月份天数（不考虑闰年）

题目：输入月份 1~12，输出该月天数；非法月份提示错误。

解题思路：

1. `case 1/3/5/7/8/10/12` 可穿透到同一输出 31
2. `case 4/6/9/11` → 30
3. `case 2` → 28
4. 每个逻辑组末尾 `break`；加 `default`

完整代码：

```
#include <stdio.h>

int main(void) {
    int m;
    printf("输入月份: ");
    scanf("%d", &m);
    switch (m) {
        case 1: case 3: case 5: case 7: case 8: case 10: case 12:
            printf("31天\n");
            break;
        case 4: case 6: case 9: case 11:
            printf("30天\n");
            break;
        case 2:
            printf("28天\n");
            break;
        default:
            printf("月份无效\n");
    }
    return 0;
}
```

要点/易错：有意穿透很好用；忘了 `break` 会掉进下一个 case。

作业练习

先独立完成，再到文末「参考答案」对照。

练习1：星期几查询

- 输入数字1-7
- 使用switch输出对应的星期

练习2：简单计算器（使用switch实现）

练习3：学生成绩管理系统菜单

- 显示菜单选项
- 使用switch处理用户选择（功能可用 printf 占位）

挑战题：增强菜单程序

- 当用户输入非 1 ~ 5 时，提示错误并**继续循环**
- 额外：若 `scanf` 读入失败（例如输入了字母），清空错误状态并提示「请输入数字」

常见错误与调试提示

- **错误：**忘记添加break导致case穿透
 - **解决：**每个case后面都要添加break，除非有意使用case穿透
- **错误：**switch表达式不是整型或字符型
 - **解决：**switch的表达式必须是整型或字符型常量

- **错误**：case后面使用变量
 - **解决**：case必须是常量表达式

自主探索

1. 尝试使用switch语句实现简单的四则运算计算器
2. 搜索了解switch和if-else的性能差异
3. 尝试实现一个简单的命令行菜单系统

参考答案

练习1 参考答案

```
#include <stdio.h>

int main(void) {
    int d;
    printf("输入1-7: ");
    scanf("%d", &d);
    switch (d) {
        case 1: printf("星期一\n"); break;
        case 2: printf("星期二\n"); break;
        case 3: printf("星期三\n"); break;
        case 4: printf("星期四\n"); break;
        case 5: printf("星期五\n"); break;
        case 6: printf("星期六\n"); break;
        case 7: printf("星期日\n"); break;
        default: printf("输入错误\n");
    }
    return 0;
}
```

练习2 参考答案

```
#include <stdio.h>

int main(void) {
    double a, b;
    char op;
    printf("输入表达式 (如 3 + 5) : ");
    scanf("%lf %c %lf", &a, &op, &b);
    switch (op) {
        case '+': printf("%.2f\n", a + b); break;
        case '-': printf("%.2f\n", a - b); break;
        case '*': printf("%.2f\n", a * b); break;
        case '/':
            if (b != 0) printf("%.2f\n", a / b);
            else printf("除数不能为0\n");
            break;
        default: printf("未知运算符\n");
    }
    return 0;
}
```

```
}
```

练习3 参考答案

```
#include <stdio.h>

int main(void) {
    int choice;
    while (1) {
        printf("\n1.添加 2.查询 3.修改 4.删除 5.退出\n选择: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1: printf("执行添加\n"); break;
            case 2: printf("执行查询\n"); break;
            case 3: printf("执行修改\n"); break;
            case 4: printf("执行删除\n"); break;
            case 5: printf("再见\n"); return 0;
            default: printf("输入错误\n");
        }
    }
}
```

挑战题参考答案

思路: `scanf` 返回值不为 1 时, 用循环读掉非法字符到换行, 再提示重输。

```
#include <stdio.h>

int main(void) {
    int choice;
    int ch;
    while (1) {
        printf("\n1-4功能 5退出: ");
        if (scanf("%d", &choice) != 1) {
            while ((ch = getchar()) != '\n' && ch != EOF) { }
            printf("请输入数字\n");
            continue;
        }
        switch (choice) {
            case 1: case 2: case 3: case 4:
                printf("执行功能%d\n", choice);
                break;
            case 5:
                return 0;
            default:
                printf("输入错误\n");
        }
    }
}
```

Day 6: 循环结构 (上)

今日目标：掌握while和do-while循环

重点知识点讲解

6.1 while循环

```
while (条件) {  
    // 循环体  
}
```

执行流程：先判断条件，条件为真执行循环体，否则退出循环

6.2 do-while循环

```
do {  
    // 循环体  
} while (条件);
```

执行流程：先执行循环体，再判断条件，至少执行一次

6.3 while vs do-while

特性	while	do-while
执行顺序	先判断后执行	先执行后判断
执行次数	0次或多次	至少1次

6.4 进阶：哨兵循环与累加器

哨兵值 (sentinel)：用特殊输入表示结束，例如输入 结束成绩录入。

```
int score, sum = 0, count = 0;  
printf("输入成绩, -1结束: ");  
while (scanf("%d", &score) == 1 && score != -1) {  
    sum += score;    /* 累加器 */  
    count++;        /* 计数器 */  
    printf("继续输入, -1结束: ");  
}
```

模式记忆：初始化 → 判断 → 处理 → 更新 → 再判断。缺了「更新」就容易死循环。

知识点实战例题

例题1：累加 1~n

题目：输入正整数 n，用 while 计算 1+2+...+n。

解题思路：

- 1. 读入 n，初始化 sum=0, i=1
- 2. while i<=n: sum+=i; i++

3. 输出 sum

完整代码：

```
#include <stdio.h>

int main(void) {
    int n, i = 1, sum = 0;
    printf("输入n: ");
    scanf("%d", &n);
    while (i <= n) {
        sum += i;
        i++;
    }
    printf("sum=%d\n", sum);
    return 0;
}
```

要点/易错：循环内必须更新 i，否则死循环。

例题2：菜单至少执行一次

题目：用 do-while 实现：显示菜单，输入 0 退出，其它打印“继续”。

解题思路：菜单适合 do-while——先显示再判断是否退出。

完整代码：

```
#include <stdio.h>

int main(void) {
    int c;
    do {
        printf("1.开始 0.退出\n选择: ");
        scanf("%d", &c);
        if (c != 0) printf("继续...\n");
    } while (c != 0);
    return 0;
}
```

作业练习

先独立完成，再到文末「参考答案」对照。

练习1：输出1到100

- 使用while循环

练习2：计算1到100的和

- 使用while循环

练习3：输入正数验证

- 使用do-while循环确保输入正数

挑战题：

1. 不断输入整数直到输入 0，输出这些数的和与平均值（注意 count==0）
2. 输入一个正整数，统计它有多少位（例如 12345 → 5）

常见错误与调试提示

- **错误：**无限循环
 - **解决：**确保循环中有更新条件的语句，避免死循环
- **错误：**do-while循环末尾忘记加分号
 - **解决：**do-while循环必须以分号结尾
- **错误：**循环条件错误
 - **解决：**仔细检查条件表达式，确保逻辑正确

自主探索

1. 使用while循环计算阶乘
2. 使用do-while循环实现菜单程序
3. 搜索了解循环的时间复杂度概念

参考答案

练习1 参考答案

```
#include <stdio.h>

int main(void) {
    int i = 1;
    while (i <= 100) {
        printf("%d ", i);
        i++;
    }
    printf("\n");
    return 0;
}
```

练习2 参考答案

```
#include <stdio.h>

int main(void) {
    int i = 1, sum = 0;
    while (i <= 100) {
        sum += i;
        i++;
    }
    printf("%d\n", sum);
    return 0;
}
```

练习3 参考答案

```
#include <stdio.h>

int main(void) {
    int num;
    do {
        printf("请输入一个正数: ");
        scanf("%d", &num);
    } while (num <= 0);
    printf("你输入的正数是: %d\n", num);
    return 0;
}
```

挑战题参考答案

```
#include <stdio.h>

int main(void) {
    int x, sum = 0, count = 0;
    printf("输入整数, 0结束: ");
    while (scanf("%d", &x) == 1 && x != 0) {
        sum += x;
        count++;
    }
    if (count == 0) printf("没有有效数据\n");
    else printf("和=%d 平均=%.2f\n", sum, (double)sum / count);

    int n, digits = 0;
    printf("输入正整数统计位数: ");
    scanf("%d", &n);
    if (n == 0) digits = 1;
    else {
        while (n > 0) {
            digits++;
            n /= 10;
        }
    }
    printf("位数=%d\n", digits);
    return 0;
}
```

Day 7: 循环结构（下）

今日目标: 掌握for循环和循环嵌套, 理解break/continue

重点知识点讲解

7.1 for循环

```
for (初始化; 条件; 更新) {  
    // 循环体  
}
```

示例：输出1到10

```
for (int i = 1; i <= 10; i++) {  
    printf("%d ", i);  
}
```

7.2 break语句

跳出当前循环：

```
for (int i = 1; i <= 10; i++) {  
    if (i == 5) break;  
    printf("%d ", i);  
}  
// 输出: 1 2 3 4
```

7.3 continue语句

跳过当前迭代，继续下一次循环：

```
for (int i = 1; i <= 10; i++) {  
    if (i % 2 == 0) continue;  
    printf("%d ", i);  
}  
// 输出: 1 3 5 7 9
```

7.4 循环嵌套

循环中包含另一个循环：

```
// 打印九九乘法表  
for (int i = 1; i <= 9; i++) {  
    for (int j = 1; j <= i; j++) {  
        printf("%d×%d=%d\t", j, i, i * j);  
    }  
    printf("\n");  
}
```

7.5 进阶：break 作用范围与质数判断优化

1. `break` / `continue` 只作用于最近的那一层循环。要跳出多层，通常用标志变量，或把逻辑拆成函数后 `return`。
2. 判断质数时，试除到 `sqrt(n)`（即 `i * i <= n`）即可，不必试到 `n-1`。
3. `for (int i = 1; i <= n; i++)` 里在 C99 才允许在 `for` 内声明 `i`；若编译报错，把 `int i;` 提到循环外，或在 Dev-C++ 编译选项中开启 C99/GNU。

知识点实战例题

例题1: for 打印偶数

题目: 用 for 打印 1 ~ 20 中的偶数; 遇到 16 用 continue 跳过。

解题思路: i 从 1 到 20; 若 i==16 continue; 若 i%2==0 则打印。

完整代码:

```
#include <stdio.h>

int main(void) {
    int i;
    for (i = 1; i <= 20; i++) {
        if (i == 16) continue;
        if (i % 2 == 0) printf("%d ", i);
    }
    printf("\n");
    return 0;
}
```

例题2: 嵌套打印矩形

题目: 输入行数 r、列数 c, 打印 r 行 c 列的  矩形。

解题思路: 外层控行, 内层控列, 行末换行。

完整代码:

```
#include <stdio.h>

int main(void) {
    int r, c, i, j;
    printf("行 列: ");
    scanf("%d %d", &r, &c);
    for (i = 0; i < r; i++) {
        for (j = 0; j < c; j++) printf("*");
        printf("\n");
    }
    return 0;
}
```

作业练习

先独立完成, 再到文末「参考答案」对照。

练习1: 计算阶乘

- 输入一个正整数
- 使用for循环计算阶乘

练习2: 打印金字塔

- 使用嵌套循环打印星号金字塔

练习3: 判断质数

- 输入一个正整数
- 判断是否为质数

挑战题：

1. 打印菱形（由空格和 * 组成，输入奇数行高）
2. 完数判断：一个数若等于其所有真因子之和（如 $6=1+2+3$ ），则输出「完数」；找出 2 ~ 1000 内所有完数

常见错误与调试提示

- **错误：**for循环中初始化表达式执行多次
 - **解决：**初始化表达式只执行一次，不要在其中写需要重复执行的代码
- **错误：**循环嵌套导致逻辑混乱
 - **解决：**外层循环控制行数，内层循环控制列数，使用缩进提高可读性
- **错误：**误用break和continue
 - **解决：**break跳出循环，continue跳过当前迭代，注意区别

自主探索

1. 使用循环嵌套打印菱形
2. 使用循环实现斐波那契数列
3. 搜索了解goto语句的使用场景和争议

参考答案

练习1 参考答案

```
#include <stdio.h>

int main(void) {
    int n, i;
    long long fac = 1;
    printf("输入正整数: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++) fac *= i;
    printf("%d! = %lld\n", n, fac);
    return 0;
}
```

练习2 参考答案

```
#include <stdio.h>

int main(void) {
    int n = 5, i, j, k;
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n - i; j++) printf(" ");
        for (k = 1; k <= 2 * i - 1; k++) printf("*");
        printf("\n");
    }
    return 0;
}
```

练习3 参考答案

```
#include <stdio.h>

int main(void) {
    int num, i, is_prime = 1;
    printf("请输入一个正整数: ");
    scanf("%d", &num);
    if (num <= 1) is_prime = 0;
    else {
        for (i = 2; i * i <= num; i++) {
            if (num % i == 0) { is_prime = 0; break; }
        }
    }
    printf(is_prime ? "%d是质数\n" : "%d不是质数\n", num);
    return 0;
}
```

挑战题参考答案

```
#include <stdio.h>

int main(void) {
    int n, i, j, k;
    printf("输入奇数行高: ");
    scanf("%d", &n);
    int mid = n / 2;
    for (i = 0; i < n; i++) {
        int stars = i <= mid ? (2 * i + 1) : (2 * (n - 1 - i) + 1);
        int spaces = (n - stars) / 2;
        for (j = 0; j < spaces; j++) printf(" ");
        for (k = 0; k < stars; k++) printf("*");
        printf("\n");
    }

    int m, s, d;
    for (m = 2; m <= 1000; m++) {
        s = 0;
        for (d = 1; d < m; d++) if (m % d == 0) s += d;
        if (s == m) printf("%d ", m);
    }
}
```

```
}  
printf("\n");  
return 0;  
}
```

Day 8: 数组

今日目标：理解数组的概念，学会使用数组存储和处理批量数据

重点知识点讲解

8.1 数组的声明与初始化

```
// 声明并初始化  
int scores[5] = {85, 90, 78, 92, 88};  
  
// 省略大小（由初始化列表决定）  
int arr[] = {1, 2, 3, 4, 5};  
  
// 部分初始化（未初始化的元素默认为0）  
int arr[5] = {1, 2, 3};  
  
// 全部初始化为0  
int arr[5] = {0};
```

8.2 数组的访问

数组通过索引访问，索引从0开始：

```
int arr[5] = {10, 20, 30, 40, 50};  
arr[0] = 100; // 修改第一个元素  
printf("%d", arr[2]); // 输出第三个元素：30
```

8.3 数组的遍历

```
int arr[] = {1, 2, 3, 4, 5};  
int len = sizeof(arr) / sizeof(arr[0]);  
  
for (int i = 0; i < len; i++) {  
    printf("%d ", arr[i]);  
}
```

8.4 数组作为函数参数

```
void printArray(int arr[], int len) {
    for (int i = 0; i < len; i++) {
        printf("%d ", arr[i]);
    }
}
```

8.5 进阶：越界、sizeof 失效与二维数组

1. **越界是未定义行为**：`arr[5]` 在长度为 5 的数组上非法，可能当时“看起来正常”，实则埋雷。
2. **函数里不要写 `sizeof(arr)/sizeof(arr[0])` 求长度**：参数 `arr` 已退化成指针，`sizeof(arr)` 往往是指针大小。长度必须单独传入。
3. **二维数组入门**：

```
int a[2][3] = {{1, 2, 3}, {4, 5, 6}};
printf("%d\n", a[1][2]); /* 6 */
```

内存上按行优先连续存放，这为后面理解指针打下基础。

知识点实战例题

例题1：数组求和

题目：定义数组 `{1,3,5,7,9}`，求元素之和。

解题思路：用 `sizeof` 求长度，for 累加。

完整代码：

```
#include <stdio.h>

int main(void) {
    int a[] = {1, 3, 5, 7, 9};
    int len = sizeof(a) / sizeof(a[0]);
    int i, sum = 0;
    for (i = 0; i < len; i++) sum += a[i];
    printf("sum=%d\n", sum);
    return 0;
}
```

例题2：查找元素

题目：在数组中查找目标值，找到打印下标，否则打印 -1。

解题思路：顺序扫描；找到即 break 并记录下标。

完整代码：

```
#include <stdio.h>

int main(void) {
    int a[] = {4, 2, 9, 7, 1};
    int target = 7, i, idx = -1;
    int len = sizeof(a) / sizeof(a[0]);
    for (i = 0; i < len; i++) {
        if (a[i] == target) { idx = i; break; }
    }
    printf("%d\n", idx);
    return 0;
}
```

作业练习

先独立完成，再到文末「参考答案」对照。

练习1：数组求最大值、最小值、平均值

练习2：冒泡排序

- 对数组进行升序排序

练习3：数组逆序

- 将数组元素逆序存放

挑战题：

1. 在有序数组中插入一个新元素，保持有序（注意数组容量）
2. 实现选择排序，并与冒泡排序对比交换次数（可加计数器）

常见错误与调试提示

- **错误：**数组越界访问
 - **解决：**确保索引在有效范围内，避免访问不存在的元素
- **错误：**数组作为函数参数时丢失大小信息
 - **解决：**数组作为参数时会退化为指针，需要额外传递长度参数
- **错误：**使用 `==` 比较数组
 - **解决：**数组名是指针，`==` 比较的是地址，需要逐元素比较

自主探索

1. 实现选择排序算法
2. 实现二分查找算法
3. 搜索了解二维数组的内存布局

参考答案

练习1 参考答案

```
#include <stdio.h>
```

```

int main(void) {
    int a[5], i, max, min;
    double avg = 0;
    printf("输入5个整数: ");
    for (i = 0; i < 5; i++) scanf("%d", &a[i]);
    max = min = a[0];
    for (i = 0; i < 5; i++) {
        if (a[i] > max) max = a[i];
        if (a[i] < min) min = a[i];
        avg += a[i];
    }
    avg /= 5;
    printf("max=%d min=%d avg=%.2f\n", max, min, avg);
    return 0;
}

```

练习2 参考答案

```

#include <stdio.h>

void bubbleSort(int arr[], int len) {
    int i, j, t;
    for (i = 0; i < len - 1; i++)
        for (j = 0; j < len - 1 - i; j++)
            if (arr[j] > arr[j + 1]) {
                t = arr[j]; arr[j] = arr[j + 1]; arr[j + 1] = t;
            }
}

int main(void) {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int len = sizeof(arr) / sizeof(arr[0]), i;
    bubbleSort(arr, len);
    for (i = 0; i < len; i++) printf("%d ", arr[i]);
    printf("\n");
    return 0;
}

```

练习3 参考答案

```

#include <stdio.h>

int main(void) {
    int a[] = {1, 2, 3, 4, 5};
    int len = 5, i, t;
    for (i = 0; i < len / 2; i++) {
        t = a[i]; a[i] = a[len - 1 - i]; a[len - 1 - i] = t;
    }
    for (i = 0; i < len; i++) printf("%d ", a[i]);
    printf("\n");
    return 0;
}

```

挑战题参考答案

```
#include <stdio.h>

int main(void) {
    int a[10] = {1, 3, 5, 7, 9}, n = 5, x = 6, i, j;
    /* 有序插入 */
    for (i = n - 1; i >= 0 && a[i] > x; i--) a[i + 1] = a[i];
    a[i + 1] = x; n++;
    for (i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");

    /* 选择排序 */
    int b[] = {5, 2, 8, 1, 9}, m = 5, min, t;
    for (i = 0; i < m - 1; i++) {
        min = i;
        for (j = i + 1; j < m; j++) if (b[j] < b[min]) min = j;
        t = b[i]; b[i] = b[min]; b[min] = t;
    }
    for (i = 0; i < m; i++) printf("%d ", b[i]);
    printf("\n");
    return 0;
}
```

Day 9：字符串

今日目标：理解字符串的概念，学会使用字符串处理函数

重点知识点讲解

9.1 字符串的定义

在C语言中，字符串是以 '\0' 结尾的字符数组：

```
char str1[6] = {'H', 'e', 'l', 'l', 'o', '\0'};
char str2[] = "Hello"; // 推荐方式
```

9.2 常用字符串函数

函数	功能
strlen(str)	计算字符串长度
strcpy(dest, src)	字符串复制
strcat(dest, src)	字符串拼接
strcmp(str1, str2)	字符串比较

9.3 字符串输入输出

```
char name[50];
scanf("%s", name);           // 输入（遇到空格停止）
fgets(name, sizeof(name), stdin); // 输入整行
printf("%s", name);         // 输出
```

9.4 进阶：'\0'、危险的 gets 与更安全的读入

1. 字符串长度 `strlen` 不含 `'\0'`，但数组空间必须为 `'\0'` 留位置：`"Hi"` 至少要 `char s[3]`。
2. 永远不要使用 `gets`：无法限制长度，极易缓冲区溢出。教材若出现，请改用 `fgets`。
3. `fgets` 会读入换行符，常用处理：

```
char line[100];
if (fgets(line, sizeof(line), stdin)) {
    size_t n = strlen(line);
    if (n > 0 && line[n - 1] == '\n') {
        line[n - 1] = '\0';
    }
}
```

4. 需要从字符串里解析数字时，可了解 `sscanf(line, "%d", &x);`。

知识点实战例题

例题1：手写统计长度

题目：不调用 `strlen`，统计字符串长度。

解题思路：从下标 0 走到 `\0` 为止。

完整代码：

```
#include <stdio.h>

int main(void) {
    char s[] = "Hello";
    int i = 0;
    while (s[i] != '\0') i++;
    printf("len=%d\n", i);
    return 0;
}
```

例题2：fgets 读一行并去掉换行

题目：读入一行文本（可含空格），去掉末尾 `\n` 后输出。

解题思路：`fgets` → 找末尾换行改成 `\0`。

完整代码：


```
#include <stdio.h>
#include <string.h>

int main(void) {
    char line[100];
    printf("输入一行: ");
    if (fgets(line, sizeof(line), stdin)) {
        size_t n = strlen(line);
        if (n > 0 && line[n - 1] == '\n') line[n - 1] = '\0';
        printf("[%s]\n", line);
    }
    return 0;
}
```

作业练习

先独立完成，再到文末「参考答案」对照。

练习1：字符串长度

- 自己实现strlen函数

练习2：字符串反转

练习3：统计字符出现次数

- 输入字符串和目标字符
- 统计该字符在字符串中出现的次数

挑战题：手写 `my_strcmp`（按字典序比较，返回负/零/正），再用它判断用户输入的密码是否等于 `"secret"`

常见错误与调试提示

- **错误：**字符串忘记添加 `'\0'` 结尾符
 - **解决：**字符串必须以 `'\0'` 结尾，否则会导致未定义行为
- **错误：**使用 `==` 比较字符串
 - **解决：**必须使用 `strcmp` 函数比较字符串内容
- **错误：**`strcpy` 导致缓冲区溢出
 - **解决：**确保目标数组足够大，或使用 `strncpy`

自主探索

1. 自己实现strcpy和strcmp函数
2. 实现字符串拼接函数
3. 搜索了解字符串的内存存储方式

参考答案

练习1 参考答案

```
#include <stdio.h>

int my_strlen(const char *str) {
    int len = 0;
    while (str[len] != '\0') len++;
    return len;
}

int main(void) {
    char str[] = "Hello, World!";
    printf("长度: %d\n", my_strlen(str));
    return 0;
}
```

练习2 参考答案

```
#include <stdio.h>

int main(void) {
    char s[100];
    int i, j;
    char t;
    printf("输入字符串: ");
    scanf("%s", s);
    for (i = 0; s[i] != '\0'; i++) { }
    for (j = 0; j < i / 2; j++) {
        t = s[j]; s[j] = s[i - 1 - j]; s[i - 1 - j] = t;
    }
    printf("%s\n", s);
    return 0;
}
```

练习3 参考答案

```
#include <stdio.h>

int main(void) {
    char s[100], ch;
    int i, cnt = 0;
    printf("字符串: ");
    scanf("%s", s);
    printf("目标字符: ");
    scanf(" %c", &ch);
    for (i = 0; s[i] != '\0'; i++) if (s[i] == ch) cnt++;
    printf("出现%d次\n", cnt);
    return 0;
}
```

挑战题参考答案

```
#include <stdio.h>
```

```
int my_strcmp(const char *a, const char *b) {
    while (*a && (*a == *b)) { a++; b++; }
    return (unsigned char)*a - (unsigned char)*b;
}

int main(void) {
    char pwd[50];
    printf("密码: ");
    scanf("%s", pwd);
    if (my_strcmp(pwd, "secret") == 0) printf("正确\n");
    else printf("错误\n");
    return 0;
}
```

Day 10: 函数

今日目标：掌握函数的定义与调用，理解参数传递和递归

重点知识点讲解

10.1 函数定义

```
返回类型 函数名(参数列表) {
    // 函数体
    return 返回值;
}
```

10.2 参数传递方式

值传递：函数对参数的修改不影响实参

```
void increment(int x) {
    x++;
}
```

地址传递：函数对参数的修改会影响实参

```
void increment(int *x) {
    (*x)++;
}
```

10.3 函数原型

函数原型是函数的声明，用于类型检查：

```
int add(int a, int b); // 函数原型

int main() {
    add(3, 5);
    return 0;
}

int add(int a, int b) { // 函数定义
    return a + b;
}
```

10.4 递归函数

函数调用自身，需要终止条件：

```
int factorial(int n) {
    if (n == 0 || n == 1) {
        return 1;
    }
    return n * factorial(n - 1);
}
```

10.5 进阶：形参/实参、值传递局限、递归 vs 循环

1. **实参**是调用时传入的值；**形参**是函数定义中的变量。值传递时形参是实参的副本。
2. 想在函数内修改外部变量，必须传**地址**（指针）——这是后面学指针的动机之一。
3. 能用循环写清的，不必强行递归。递归适合分治、树形结构；注意栈深度，`n` 很大时阶乘递归可能栈溢出，循环更稳。

知识点实战例题

例题1：函数求两数之和

题目：编写 `add(a,b)`，在 `main` 中调用并打印。

解题思路：先声明原型（或定义在 `main` 前）→ 调用 → `return`。

完整代码：

```
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

int main(void) {
    printf("%d\n", add(3, 5));
    return 0;
}
```

例题2：递归求阶乘（理解调用栈）

题目：写 `fac(n)` 递归版，计算 `5!`。

解题思路：边界 $n \leq 1$ 返回 1；否则 $n * \text{fac}(n-1)$ 。

完整代码：

```
#include <stdio.h>

long long fac(int n) {
    if (n <= 1) return 1;
    return n * fac(n - 1);
}

int main(void) {
    printf("%lld\n", fac(5));
    return 0;
}
```

作业练习

先独立完成，再到文末「参考答案」对照。

练习1：计算数组的最大值（使用函数）

练习2：递归计算幂

练习3：二分查找

- 在有序数组中查找目标值

挑战题：

- 写 `gcd(a, b)`：递归版与循环版各一（欧几里得算法）
- 了解汉诺塔：写出 `hanoi(n, from, to, via)` 的递归框架，并对 `n=3` 跟踪打印步骤（理解优先，不强求一次写对）

常见错误与调试提示

- 错误：**函数参数类型不匹配
 - 解决：**确保函数调用时参数类型和数量与定义一致
- 错误：**递归没有终止条件
 - 解决：**递归函数必须有明确的终止条件，否则会导致栈溢出
- 错误：**函数返回值类型不匹配
 - 解决：**确保返回值类型与函数声明一致

自主探索

- 使用递归实现斐波那契数列
- 搜索了解尾递归的概念
- 尝试将递归函数转换为迭代实现

参考答案

练习1 参考答案

```
#include <stdio.h>

int max_of(int arr[], int len) {
    int i, m = arr[0];
    for (i = 1; i < len; i++) if (arr[i] > m) m = arr[i];
    return m;
}

int main(void) {
    int a[] = {3, 9, 2, 7};
    printf("%d\n", max_of(a, 4));
    return 0;
}
```

练习2 参考答案

```
#include <stdio.h>

long long power(int base, int exp) {
    if (exp == 0) return 1;
    return base * power(base, exp - 1);
}

int main(void) {
    printf("%lld\n", power(2, 10));
    return 0;
}
```

练习3 参考答案

```
#include <stdio.h>

int binarySearch(int arr[], int len, int target) {
    int left = 0, right = len - 1, mid;
    while (left <= right) {
        mid = (left + right) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) left = mid + 1;
        else right = mid - 1;
    }
    return -1;
}

int main(void) {
    int arr[] = {1, 3, 5, 7, 9, 11, 13};
    int idx = binarySearch(arr, 7, 7);
    printf("%d\n", idx);
    return 0;
}
```

挑战题参考答案

```
#include <stdio.h>

int gcd_rec(int a, int b) {
    if (b == 0) return a;
    return gcd_rec(b, a % b);
}

int gcd_loop(int a, int b) {
    int t;
    while (b != 0) { t = a % b; a = b; b = t; }
    return a;
}

void hanoi(int n, char from, char to, char via) {
    if (n == 1) {
        printf("%c -> %c\n", from, to);
        return;
    }
    hanoi(n - 1, from, via, to);
    printf("%c -> %c\n", from, to);
    hanoi(n - 1, via, to, from);
}

int main(void) {
    printf("gcd=%d / %d\n", gcd_rec(48, 18), gcd_loop(48, 18));
    hanoi(3, 'A', 'C', 'B');
    return 0;
}
```

Day 11: 指针 (上)

今日目标：理解指针的概念，掌握指针与数组的关系

重点知识点讲解

11.1 指针的概念

指针是一个变量，存储的是另一个变量的内存地址：

```
int num = 42;
int *ptr = &num; // ptr指向num
```

11.2 指针运算符

运算符	名称	作用
&	取地址运算符	获取变量的内存地址
*	解引用运算符	获取指针指向的值

11.3 指针与数组

数组名本质上是指向数组第一个元素的指针：

```
int arr[] = {10, 20, 30};
arr[0] == *arr;    // true
arr[1] == *(arr+1); // true
```

11.4 使用指针遍历数组

```
int arr[] = {1, 2, 3, 4, 5};
int len = sizeof(arr) / sizeof(arr[0]);

for (int *p = arr; p < arr + len; p++) {
    printf("%d ", *p);
}
```

11.5 进阶：指针类型、const 与指针算术

- 指针有类型：`int *` 一次移动 `sizeof(int)` 字节；类型决定解引用时如何解释内存。
- `const` 两种常见写法（含义不同）：
 - `const int *p`：不能通过 `p` 改所指向的值
 - `int *const p`：`p` 本身不能改指向
- `p + 1` 不是地址数字加 1，而是指向下一个同类型元素。
- 数组名多数场合会退化成指针，但数组名不是“可赋值的指针变量”：不能写 `arr = ...`。

知识点实战例题

例题1：指针修改变量

题目：通过指针把变量从 10 改成 20。

解题思路：`p=&x; *p=20;`

完整代码：


```
#include <stdio.h>

int main(void) {
    int x = 10;
    int *p = &x;
    *p = 20;
    printf("%d\n", x);
    return 0;
}
```

例题2：指针遍历数组

题目：用指针打印数组全部元素。

解题思路： `p` 从 `arr` 走到 `arr+len`。

完整代码：

```
#include <stdio.h>

int main(void) {
    int arr[] = {1, 2, 3, 4};
    int *p;
    for (p = arr; p < arr + 4; p++) printf("%d ", *p);
    printf("\n");
    return 0;
}
```

作业练习

先独立完成，再到文末「参考答案」对照。

练习1：使用指针实现数组逆序

练习2：使用指针交换两个整数

练习3：字符串复制（使用指针实现）

挑战题：用指针实现在数组中查找目标值，返回指向该元素的指针；找不到返回 `NULL`，并在 `main` 中安全处理

常见错误与调试提示

- **错误：**空指针解引用
 - **解决：**使用指针前检查是否为 `NULL`
- **错误：**野指针（未初始化的指针）
 - **解决：**初始化指针，避免使用未初始化的指针
- **错误：**指针运算越界
 - **解决：**确保指针操作在有效范围内

自主探索

1. 使用指针实现选择排序

2. 搜索了解指针数组和数组指针的区别
3. 尝试使用指针实现字符串反转

参考答案

练习1 参考答案

```
#include <stdio.h>

void reverse(int *a, int len) {
    int *l = a, *r = a + len - 1, t;
    while (l < r) {
        t = *l; *l = *r; *r = t;
        l++; r--;
    }
}

int main(void) {
    int a[] = {1, 2, 3, 4, 5}, i;
    reverse(a, 5);
    for (i = 0; i < 5; i++) printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

练习2 参考答案

```
#include <stdio.h>

void swap(int *a, int *b) {
    int t = *a; *a = *b; *b = t;
}

int main(void) {
    int x = 3, y = 5;
    swap(&x, &y);
    printf("%d %d\n", x, y);
    return 0;
}
```

练习3 参考答案

```
#include <stdio.h>

void my_strcpy(char *dest, const char *src) {
    while (*src != '\0') {
        *dest = *src;
        dest++; src++;
    }
    *dest = '\0';
}
```

```
int main(void) {
    char src[] = "Hello, World!";
    char dest[20];
    my_strcpy(dest, src);
    printf("%s\n", dest);
    return 0;
}
```

挑战题参考答案

```
#include <stdio.h>

int *find(int *a, int len, int target) {
    int i;
    for (i = 0; i < len; i++)
        if (a[i] == target) return &a[i];
    return NULL;
}

int main(void) {
    int a[] = {4, 2, 9, 7};
    int *p = find(a, 4, 9);
    if (p) printf("found %d\n", *p);
    else printf("NULL\n");
    return 0;
}
```

Day 12: 指针 (下)

今日目标: 掌握动态内存分配, 理解指针的高级应用

重点知识点讲解

12.1 动态内存分配函数

函数	功能
<code>malloc(size)</code>	分配指定大小的内存
<code>calloc(n, size)</code>	分配n个size大小的内存并初始化为0
<code>realloc(ptr, size)</code>	重新分配内存大小
<code>free(ptr)</code>	释放已分配的内存

12.2 动态内存分配示例

```
#include <stdlib.h>
```

```
// 分配单个整数
int *p = (int *)malloc(sizeof(int));
*p = 42;

// 分配数组
int *arr = (int *)malloc(5 * sizeof(int));
for (int i = 0; i < 5; i++) {
    arr[i] = i * 10;
}

// 释放内存
free(p);
free(arr);
```

12.3 常见指针错误

- **空指针解引用**: `int *p = NULL; *p = 10;`
- **野指针**: 未初始化的指针
- **内存泄漏**: 忘记释放动态分配的内存

12.4 进阶: malloc 失败、二次释放与 realloc

```
int *arr = (int *)malloc(n * sizeof(int));
if (arr == NULL) {
    printf("内存分配失败\n");
    return 1;
}
/* 使用 arr ... */
free(arr);
arr = NULL; /* 防止悬空指针被再次使用 */
```

要点:

1. 每次 `malloc` / `calloc` 都要检查是否为 `NULL`。
2. 谁分配谁释放; 同一块内存不要 `free` 两次。
3. `realloc` 可能返回新地址: 应先用临时指针接住, 成功后再赋回, 失败时旧块仍有效。
4. 释放后把指针置 `NULL`, 是简单有效的防护习惯。

知识点实战例题

例题1: malloc 分配单个 int

题目: 动态分配一个 int, 赋值为 42, 打印后释放。

解题思路: malloc → 判空 → 使用 → free → 置 NULL。

完整代码:

```
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int *p = (int *)malloc(sizeof(int));
    if (!p) return 1;
    *p = 42;
    printf("%d\n", *p);
    free(p);
    p = NULL;
    return 0;
}
```

作业练习

先独立完成，再到文末「参考答案」对照。

练习1：动态数组实现

- 创建动态数组，支持添加元素和自动扩容

练习2：动态二维数组

练习3：指针实现链表（单链表基本操作）

挑战题：改造上面的 `addElement`：用临时指针接收 `realloc` 结果；若失败则打印错误且不丢失原数据；在 `createDynamicArray` 中检查每一次分配是否成功

常见错误与调试提示

- **错误：**忘记释放动态分配的内存
 - **解决：**动态分配的内存必须使用`free`释放
- **错误：**重复释放内存
 - **解决：**确保每个`malloc`只对应一个`free`
- **错误：**使用已释放的内存
 - **解决：**释放后将指针置为`NULL`，使用前检查

自主探索

1. 实现一个简单的内存池
2. 搜索了解智能指针的概念
3. 尝试实现双向链表

参考答案

练习1 参考答案（含安全 `realloc` 思路）

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int *data;
```

```

    int size;
    int capacity;
} DynamicArray;

DynamicArray *createDynamicArray(int cap) {
    DynamicArray *arr = (DynamicArray *)malloc(sizeof(DynamicArray));
    if (!arr) return NULL;
    arr->data = (int *)malloc(cap * sizeof(int));
    if (!arr->data) { free(arr); return NULL; }
    arr->size = 0;
    arr->capacity = cap;
    return arr;
}

int addElement(DynamicArray *arr, int element) {
    if (arr->size >= arr->capacity) {
        int newcap = arr->capacity * 2;
        int *tmp = (int *)realloc(arr->data, newcap * sizeof(int));
        if (!tmp) return 0;
        arr->data = tmp;
        arr->capacity = newcap;
    }
    arr->data[arr->size++] = element;
    return 1;
}

void destroyDynamicArray(DynamicArray *arr) {
    free(arr->data);
    free(arr);
}

int main(void) {
    DynamicArray *arr = createDynamicArray(2);
    int i;
    addElement(arr, 10);
    addElement(arr, 20);
    addElement(arr, 30);
    for (i = 0; i < arr->size; i++) printf("%d ", arr->data[i]);
    printf("\n");
    destroyDynamicArray(arr);
    return 0;
}

```

练习2 参考答案

```

#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int r = 2, c = 3, i, j;
    int **a = (int **)malloc(r * sizeof(int *));
    for (i = 0; i < r; i++) a[i] = (int *)malloc(c * sizeof(int));
    for (i = 0; i < r; i++)

```

```

    for (j = 0; j < c; j++) a[i][j] = i * c + j;
    for (i = 0; i < r; i++) {
        for (j = 0; j < c; j++) printf("%d ", a[i][j]);
        printf("\n");
    }
    for (i = 0; i < r; i++) free(a[i]);
    free(a);
    return 0;
}

```

练习3 参考答案

```

#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

int main(void) {
    Node *head = NULL, *p;
    int vals[3] = {1, 2, 3}, i;
    for (i = 2; i >= 0; i--) {
        p = (Node *)malloc(sizeof(Node));
        p->data = vals[i];
        p->next = head;
        head = p;
    }
    for (p = head; p; p = p->next) printf("%d ", p->data);
    printf("\n");
    while (head) {
        p = head; head = head->next; free(p);
    }
    return 0;
}

```

挑战题参考答案：见练习1 中的 `addElement`（临时指针 + 失败不丢数据）。

Day 13: 结构体

今日目标：掌握结构体的定义与使用，学会创建复杂数据结构

重点知识点讲解

13.1 结构体定义

```
struct Person {
    char name[50];
    int age;
    float height;
};
```

13.2 结构体初始化

```
struct Person p1 = {"张三", 25, 1.75f};

struct Person p2 = {
    .name = "李四",
    .age = 30,
    .height = 1.80f
};
```

13.3 结构体访问

```
struct Person p = {"张三", 25, 1.75f};

// 使用.运算符
printf("%s", p.name);

// 使用指针和->运算符
struct Person *ptr = &p;
printf("%d", ptr->age);
```

13.4 结构体数组

```
struct Student students[3] = {
    {101, "张三", 95.5f},
    {102, "李四", 88.0f},
    {103, "王五", 92.3f}
};
```

13.5 typedef的使用

```
typedef struct {
    char name[50];
    int age;
} Person;

Person p = {"张三", 25};
```

13.6 进阶：结构体对齐与传参方式

1. **对齐（了解）**：编译器可能在成员之间插入填充字节，故 `sizeof(struct)` 不一定等于各成员大小之和。可用 `printf("%zu\n", sizeof(Person));` 观察。
2. **传结构体 vs 传指针**：
 - 传结构体：复制整份数据，适合小结构

- 传指针：只传地址，适合大结构或需要在函数内修改原对象

3. 链表节点是结构体 + 指针的经典组合：

```
typedef struct Node {  
    int data;  
    struct Node *next;  
} Node;
```

知识点实战例题

例题1：结构体表示点并计算

题目：定义 `Point{x,y}`，输入两点，输出曼哈顿距离 `|x1-x2|+|y1-y2|`。

解题思路：定义结构体 → 读入 → 计算绝对值差之和。

完整代码：

```
#include <stdio.h>  
#include <stdlib.h>  
  
typedef struct {  
    int x, y;  
} Point;  
  
int main(void) {  
    Point a, b;  
    scanf("%d %d %d %d", &a.x, &a.y, &b.x, &b.y);  
    printf("%d\n", abs(a.x - b.x) + abs(a.y - b.y));  
    return 0;  
}
```

作业练习

先独立完成，再到文末「参考答案」对照。

练习1：学生信息管理系统

- 使用结构体数组存储学生信息
- 实现添加、查询功能

练习2：日期结构体

- 定义日期结构体
- 实现日期比较功能

练习3：通讯录管理

- 定义联系人结构体
- 实现添加、查找、删除功能

挑战题：

1. 对学生数组按成绩降序排序（冒泡即可）
2. 用 `Node` 实现：头插法插入 3 个节点并遍历打印（为后续数据结构打基础）

常见错误与调试提示

- **错误：** 结构体成员访问错误
 - **解决：** 使用 `.` 访问结构体变量，使用 `->` 访问结构体指针
- **错误：** 结构体数组越界
 - **解决：** 确保索引在有效范围内
- **错误：** 结构体赋值错误
 - **解决：** 同类型结构体可以直接赋值，不同类型需要逐成员赋值

自主探索

1. 实现二叉树节点结构
2. 使用共用体实现类型转换
3. 搜索了解位域的使用场景

参考答案

练习1 参考答案（精简版添加+显示）

```
#include <stdio.h>

#define MAX 100

typedef struct {
    int id;
    char name[50];
    float score;
} Student;

int main(void) {
    Student s[MAX];
    int n = 0, op, i, id;
    do {
        printf("1加2查3列0退: ");
        scanf("%d", &op);
        if (op == 1 && n < MAX) {
            scanf("%d %s %f", &s[n].id, s[n].name, &s[n].score);
            n++;
        } else if (op == 2) {
            scanf("%d", &id);
            for (i = 0; i < n; i++)
                if (s[i].id == id) printf("%s %.1f\n", s[i].name, s[i].score);
        } else if (op == 3) {
            for (i = 0; i < n; i++)
                printf("%d %s %.1f\n", s[i].id, s[i].name, s[i].score);
        }
    } while (op != 0);
    return 0;
}
```

练习2 参考答案

```

#include <stdio.h>

typedef struct { int y, m, d; } Date;

int cmp(Date a, Date b) {
    if (a.y != b.y) return a.y - b.y;
    if (a.m != b.m) return a.m - b.m;
    return a.d - b.d;
}

int main(void) {
    Date a = {2026, 7, 26}, b = {2026, 1, 1};
    int c = cmp(a, b);
    if (c > 0) printf("a较晚\n");
    else if (c < 0) printf("a较早\n");
    else printf("相同\n");
    return 0;
}

```

练习3 参考答案 (通讯录核心)

```

#include <stdio.h>
#include <string.h>

typedef struct {
    char name[30];
    char phone[20];
} Contact;

int main(void) {
    Contact c[50];
    int n = 0, op, i;
    char key[30];
    do {
        printf("1加2查3删0退: ");
        scanf("%d", &op);
        if (op == 1) {
            scanf("%s %s", c[n].name, c[n].phone); n++;
        } else if (op == 2) {
            scanf("%s", key);
            for (i = 0; i < n; i++)
                if (strcmp(c[i].name, key) == 0) printf("%s\n", c[i].phone);
        } else if (op == 3) {
            scanf("%s", key);
            for (i = 0; i < n; i++)
                if (strcmp(c[i].name, key) == 0) {
                    c[i] = c[n - 1]; n--; break;
                }
        }
    } while (op != 0);
    return 0;
}

```

挑战题参考答案

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int id; char name[20]; float score;
} Stu;

typedef struct Node {
    int data; struct Node *next;
} Node;

int main(void) {
    Stu s[3] = {{1, "A", 90}, {2, "B", 70}, {3, "C", 85}};
    int i, j; Stu t;
    for (i = 0; i < 2; i++)
        for (j = 0; j < 2 - i; j++)
            if (s[j].score < s[j+1].score) { t = s[j]; s[j] = s[j+1]; s[j+1] = t; }
    for (i = 0; i < 3; i++) printf("%s %.0f\n", s[i].name, s[i].score);

    Node *head = NULL, *p;
    int v;
    for (v = 1; v <= 3; v++) {
        p = (Node *)malloc(sizeof(Node));
        p->data = v; p->next = head; head = p;
    }
    for (p = head; p; p = p->next) printf("%d ", p->data);
    printf("\n");
    return 0;
}
```

Day 14: 综合实战

今日目标：完成三个实战项目，综合运用所学知识

综合项目实施思路

项目1 简易计算器 — 实现思路

1. 用死循环打印菜单
2. `switch` 分支处理加减乘除/退出
3. 除法前判断除数是否为 0
4. (加分) 用变量 `last` 保存上次结果，支持连续运算

项目2 学生成绩管理 — 实现思路

1. 定义 `Student` 结构体数组 + `count`

2. 菜单驱动：增删改查、列表、平均分
3. 查找用 for + 学号比较；删除用“末尾覆盖”
4. （加分）按成绩冒泡排序后显示

项目3 图书管理 — 实现思路

1. 定义 `Book`（编号、书名、作者、库存）
2. 借阅前检查库存>0；归还库存+1
3. 查询可按编号；加分可用书名关键字（循环/`strstr`）

实战项目1：简易计算器

功能要求：

1. **基本运算**：支持加、减、乘、除四则运算
2. **连续计算**：支持连续输入多个运算
3. **退出功能**：输入特定命令退出程序
4. **菜单界面**：清晰的操作菜单

输入输出示例：

```
===== 简易计算器 =====
```

1. 加法
2. 减法
3. 乘法
4. 除法
5. 退出

```
请选择操作(1-5): 1
```

```
请输入第一个数: 10
```

```
请输入第二个数: 5
```

```
计算结果: 10 + 5 = 15
```

```
===== 简易计算器 =====
```

1. 加法
2. 减法
3. 乘法
4. 除法
5. 退出

```
请选择操作(1-5): 5
```

```
感谢使用计算器！
```

技术要点：

- switch语句处理运算选择
- 循环实现连续计算
- 错误处理（如除数为0）

加分项（挑战）：

- 支持“上一次结果继续运算”（例如先算出 15，再输入 `* 2` 得到 30）
- 或增加 `%`（整数取余）运算，并处理好类型

自我评估检查清单：

- ☐ 能否正确实现四则运算

- ☐ 能否处理除数为0的情况
- ☐ 能否实现连续计算
- ☐ 界面是否友好清晰
- ☐ （加分）是否完成连续运算或取余

实战项目2：学生成绩管理系统

功能要求：

1. **添加学生**：输入学号、姓名、成绩
2. **查询学生**：根据学号或姓名查询
3. **修改成绩**：修改指定学生的成绩
4. **删除学生**：删除指定学生
5. **显示所有学生**：按表格形式显示
6. **计算平均分**：计算所有学生的平均分

输入输出示例：

===== 学生成绩管理系统 =====

1. 添加学生
2. 查询学生
3. 修改成绩
4. 删除学生
5. 显示所有学生
6. 计算平均分
7. 退出

请选择操作(1-7)：1

请输入学生ID：101

请输入学生姓名：张三

请输入学生成绩：95

学生添加成功！

===== 学生成绩管理系统 =====

1. 添加学生
2. 查询学生
3. 修改成绩
4. 删除学生
5. 显示所有学生
6. 计算平均分
7. 退出

请选择操作(1-7)：5

学生列表：

ID	姓名	成绩
101	张三	95.0

===== 学生成绩管理系统 =====

1. 添加学生
2. 查询学生
3. 修改成绩
4. 删除学生

```
5. 显示所有学生
6. 计算平均分
7. 退出
请选择操作(1-7): 7
感谢使用学生成绩管理系统!
```

技术要点:

- 结构体数组存储学生信息
- 函数封装各个功能
- 循环菜单

加分项 (挑战):

- 增加「按成绩排序显示」功能 (升序或降序)
- 查询时同时支持按学号、按姓名 (姓名可用 `strcmp`)

自我评估检查清单:

- ☐ 能否正确添加学生信息
- ☐ 能否根据学号查询学生
- ☐ 能否修改和删除学生信息
- ☐ 能否正确计算平均分
- ☐ 程序是否健壮 (处理边界情况)
- ☐ (加分) 是否完成排序或双条件查询

实战项目3: 图书管理系统

功能要求:

1. **添加图书**: 输入编号、书名、作者、库存
2. **查询图书**: 根据编号或书名查询
3. **借阅图书**: 借阅指定图书 (库存减1)
4. **归还图书**: 归还指定图书 (库存加1)
5. **显示所有图书**: 按表格形式显示

输入输出示例:

```
===== 图书管理系统 =====
1. 添加图书
2. 查询图书
3. 借阅图书
4. 归还图书
5. 显示所有图书
6. 退出
请选择操作(1-6): 1
请输入图书编号: 1001
请输入书名: C语言程序设计
请输入作者: 谭浩强
请输入库存: 10
图书添加成功!
```

```
===== 图书管理系统 =====
1. 添加图书
2. 查询图书
3. 借阅图书
4. 归还图书
5. 显示所有图书
6. 退出
请选择操作(1-6): 3
请输入要借阅的图书编号: 1001
借阅成功! 当前库存: 9
```

```
===== 图书管理系统 =====
1. 添加图书
2. 查询图书
3. 借阅图书
4. 归还图书
5. 显示所有图书
6. 退出
请选择操作(1-6): 5
```

图书列表:

编号	书名	作者	库存
1001	C语言程序设计	谭浩强	9

```
===== 图书管理系统 =====
1. 添加图书
2. 查询图书
3. 借阅图书
4. 归还图书
5. 显示所有图书
6. 退出
请选择操作(1-6): 6
感谢使用图书管理系统!
```

技术要点:

- 结构体数组存储图书信息
- 函数实现各个操作
- 完整的菜单系统

加分项 (挑战) :

- 库存为 0 时借阅应明确提示「无库存」, 不能变成负数
- 增加「按书名模糊查找」: 书名包含某关键字即命中 (可用循环比较字符, 或了解 `strstr`)

自我评估检查清单:

- ☐ 能否正确添加图书信息
- ☐ 能否根据编号查询图书
- ☐ 借阅功能是否正确 (库存减1)
- ☐ 归还功能是否正确 (库存加1)

- ☐ 能否处理库存为0的情况
- ☐ 程序是否有良好的用户体验
- ☐ （加分）无库存提示或书名查找是否完成

参考答案（核心模块）

项目1 参考实现

```
#include <stdio.h>

int main(void) {
    int op;
    double a, b, last = 0;
    int has_last = 0;
    while (1) {
        printf("1+ 2- 3* 4/ 5连续用上次 0退: ");
        scanf("%d", &op);
        if (op == 0) break;
        if (op == 5) {
            if (!has_last) { printf("尚无结果\n"); continue; }
            printf("上次=%.2f, 输入运算符和数 (如 * 2) : ", last);
            {
                char o; double x;
                scanf(" %c %lf", &o, &x);
                if (o == '+') last += x;
                else if (o == '-') last -= x;
                else if (o == '*') last *= x;
                else if (o == '/' && x != 0) last /= x;
                else { printf("错误\n"); continue; }
                printf("结果=%.2f\n", last);
            }
            continue;
        }
        printf("两数: ");
        scanf("%lf %lf", &a, &b);
        switch (op) {
            case 1: last = a + b; break;
            case 2: last = a - b; break;
            case 3: last = a * b; break;
            case 4:
                if (b == 0) { printf("除数0\n"); continue; }
                last = a / b; break;
            default: printf("无效\n"); continue;
        }
        has_last = 1;
        printf("结果=%.2f\n", last);
    }
    return 0;
}
```

项目2 参考实现（核心）

```

#include <stdio.h>
#include <string.h>

#define MAX 100
typedef struct { int id; char name[50]; float score; } Student;

int main(void) {
    Student s[MAX];
    int n = 0, op, i, id;
    float sum;
    while (1) {
        printf("1加2查3改4删5列6均7排序0退: ");
        scanf("%d", &op);
        if (op == 0) break;
        if (op == 1 && n < MAX) {
            scanf("%d %s %f", &s[n].id, s[n].name, &s[n].score); n++;
        } else if (op == 2) {
            scanf("%d", &id);
            for (i = 0; i < n; i++) if (s[i].id == id)
                printf("%s %.1f\n", s[i].name, s[i].score);
        } else if (op == 3) {
            scanf("%d", &id);
            for (i = 0; i < n; i++) if (s[i].id == id) scanf("%f", &s[i].score);
        } else if (op == 4) {
            scanf("%d", &id);
            for (i = 0; i < n; i++) if (s[i].id == id) { s[i] = s[n-1]; n--; break; }
        } else if (op == 5) {
            for (i = 0; i < n; i++) printf("%d %s %.1f\n", s[i].id, s[i].name, s[i].score);
        } else if (op == 6) {
            for (sum = 0, i = 0; i < n; i++) sum += s[i].score;
            printf(n ? "avg=%.2f\n" : "空\n", n ? sum / n : 0);
        } else if (op == 7) {
            int j; Student t;
            for (i = 0; i < n - 1; i++)
                for (j = 0; j < n - 1 - i; j++)
                    if (s[j].score < s[j+1].score) { t = s[j]; s[j] = s[j+1]; s[j+1] = t; }
            printf("已按成绩降序\n");
        }
    }
    return 0;
}

```

项目3 参考实现（核心）

```

#include <stdio.h>
#include <string.h>

#define MAX 100
typedef struct {
    int id; char title[50]; char author[30]; int stock;
} Book;

```

```

int main(void) {
    Book b[MAX];
    int n = 0, op, i, id;
    char key[50];
    while (1) {
        printf("1加2查3借4还5列0退: ");
        scanf("%d", &op);
        if (op == 0) break;
        if (op == 1 && n < MAX) {
            scanf("%d %s %s %d", &b[n].id, b[n].title, b[n].author, &b[n].stock); n++;
        } else if (op == 2) {
            scanf("%s", key);
            for (i = 0; i < n; i++)
                if (strstr(b[i].title, key)) printf("%d %s 库存%d\n", b[i].id, b[i].title,
b[i].stock);
        } else if (op == 3) {
            scanf("%d", &id);
            for (i = 0; i < n; i++) if (b[i].id == id) {
                if (b[i].stock > 0) { b[i].stock--; printf("借阅成功, 库存%d\n", b[i].stock); }
                else printf("无库存\n");
            }
        } else if (op == 4) {
            scanf("%d", &id);
            for (i = 0; i < n; i++) if (b[i].id == id) { b[i].stock++; printf("归还成功\n"); }
        } else if (op == 5) {
            for (i = 0; i < n; i++)
                printf("%d %s %s %d\n", b[i].id, b[i].title, b[i].author, b[i].stock);
        }
    }
    return 0;
}

```

📖 附录：系统教程全文（原 guides）

以下保留原 `guides/` 目录全部 15 篇专栏正文，与上文「两周学习计划」互补：计划侧重按天练习与作业，附录侧重系统讲解与延伸阅读。学习计划表中的「参考教程」已改为文内锚点。

附录目录

- [01 C语言入门：为什么它是编程的基石](#)
- [02 C语言开发环境搭建：从安装到Hello World](#)
- [03 Hello World背后：C语言程序的执行原理](#)
- [04 数据类型与变量：编程的基本积木](#)
- [05 运算符与表达式：程序的计算逻辑](#)
- [06 流程控制：让程序学会做选择](#)
- [07 循环结构：重复执行的艺术](#)
- [08 数组与字符串：批量数据处理](#)
- [09 函数：代码复用的利器](#)

- [10 指针：C语言的灵魂](#)
- [11 内存管理：堆与栈的奥秘](#)
- [12 结构体与共用体：自定义数据类型](#)
- [13 文件操作：数据的持久化存储](#)
- [14 综合实战：实现一个简易计算器](#)
- [15 C语言学习路线与资源推荐](#)

01 C语言入门：为什么它是编程的基石

从"Hello World"开始的编程之旅

在当今这个AI横行的时代，你可能会疑惑：为什么还要学习一门诞生于上世纪70年代的编程语言？Python、JavaScript这些现代语言看起来更简单、更强大，为什么还要从C语言开始？

这正是我想要与你探讨的问题。C语言不仅仅是一门编程语言，它是编程世界的基石，是理解计算机底层原理的钥匙。

一、C语言的前世今生

1.1 诞生背景：一个时代的产物

C语言诞生于1972年的贝尔实验室，由丹尼斯·里奇（Dennis Ritchie）为了开发UNIX操作系统而设计。当时的计算机世界正处于从大型机向小型机过渡的时期，急需一门既高效又灵活的编程语言。

历史节点：

- 1969年：UNIX系统开始开发
- 1972年：C语言诞生
- 1978年：《The C Programming Language》出版（K&R C）
- 1989年：ANSI C标准发布（C89）
- 1999年：C99标准发布
- 2011年：C11标准发布

1.2 设计哲学：简洁与效率的完美平衡

C语言的设计理念可以概括为：**相信程序员**。它给予程序员极大的自由，同时也要求程序员对自己的行为负责。

```
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

这是经典的"Hello World"程序，短短几行代码却蕴含着深刻的设计思想：

- **简洁性**：没有冗余的语法，一切都是必要的
- **高效性**：直接操作内存，执行效率接近汇编语言
- **可移植性**：相同的代码可以在不同平台上编译运行

1.3 影响力：改变世界的编程语言

C语言的影响力远超你的想象：

领域	应用举例
操作系统	Linux、Windows、macOS内核
嵌入式系统	单片机、物联网设备
编译器	GCC、Clang等编译器本身
数据库	MySQL、PostgreSQL
游戏引擎	Unity、Unreal Engine核心模块
网络协议	TCP/IP协议栈实现

二、为什么C语言是编程的基石

2.1 理解计算机的本质

学习C语言就像是学习计算机的"母语"。它让你直接与计算机对话，理解程序是如何在硬件上运行的。

内存管理的直观感受：

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = (int *)malloc(sizeof(int));
    *ptr = 42;
    printf("变量的值: %d\n", *ptr);
    printf("变量的地址: %p\n", ptr);
    free(ptr);
    return 0;
}
```

这段代码展示了C语言最核心的特性——直接操作内存地址。你可以清晰地看到：

- 如何向操作系统申请内存
- 如何通过指针访问内存中的数据
- 如何释放不再使用的内存

2.2 培养良好的编程习惯

C语言是一门"严格"的语言，它不会为你做太多自动处理。这种"严格"恰恰是培养良好编程习惯的最佳方式：

1. **类型安全意识**：必须明确变量类型，不能随意转换
2. **内存管理能力**：手动分配和释放内存，培养资源管理意识
3. **代码规范**：简洁的语法要求程序员写出清晰可读的代码

2.3 为学习其他语言打下基础

几乎所有现代编程语言都借鉴了C语言的语法：

```
// C语言
for (int i = 0; i < 10; i++) {
    printf("%d\n", i);
}
```

```
// Java (类C语法)
for (int i = 0; i < 10; i++) {
    System.out.println(i);
}
```

```
// JavaScript (类C语法)
for (let i = 0; i < 10; i++) {
    console.log(i);
}
```

学会C语言后，学习其他语言会变得事半功倍。

2.4 AI时代的编程基础

在AI和机器学习飞速发展的今天，你可能会认为Python已经足够了。但事实是：

1. **深度学习框架的底层**：PyTorch、TensorFlow的核心都是用C/C++实现的
2. **性能优化**：对于需要极致性能的场景，C语言仍然是首选
3. **理解原理**：只有理解底层实现，才能更好地使用高级框架

三、C语言的核心特点

3.1 高效性

C语言被称为"高级语言中的汇编"，它允许程序员直接操作硬件，执行效率极高。

3.2 可移植性

C语言的编译器几乎存在于所有平台上，相同的代码可以在不同操作系统上编译运行。

3.3 灵活性

C语言给予程序员极大的自由度，可以进行底层操作，也可以进行高层抽象。

3.4 简洁性

C语言只有32个关键字，语法简洁明了，易于学习和掌握。

四、学习C语言的正确姿势

4.1 环境准备

- **编译器**：推荐使用GCC或Clang
- **编辑器**：VS Code、Vim、Emacs
- **调试工具**：GDB

4.2 学习路径

第一阶段：基础语法

- ├─ 数据类型与变量
- ├─ 运算符与表达式
- ├─ 流程控制 (if-else, switch)
- └─ 循环结构 (for, while, do-while)

第二阶段：核心概念

- ├─ 数组与字符串
- ├─ 函数定义与调用
- ├─ 指针的概念与应用
- └─ 结构体与共用体

第三阶段：进阶内容

- ├─ 内存管理 (malloc/free)
- ├─ 文件操作
- ├─ 预处理指令
- └─ 模块化编程

4.3 实践建议

1. **动手写代码**：理论学习必须配合大量练习
2. **调试能力**：学会使用调试工具排查问题
3. **阅读优秀代码**：学习他人的编程风格和技巧
4. **参与开源项目**：从实际项目中学习

五、实战练习

练习1：理解程序执行流程

运行以下代码，观察输出结果：

```
#include <stdio.h>

int main() {
    printf("Step 1: 程序开始\n");

    int a = 10;
    printf("Step 2: 变量a = %d\n", a);

    a = a + 5;
    printf("Step 3: 变量a = %d\n", a);

    printf("Step 4: 程序结束\n");
    return 0;
}
```

练习2：计算圆的面积

编写程序，输入半径，计算并输出圆的面积。

```
#include <stdio.h>
```

```
#define PI 3.14159

int main() {
    float radius, area;

    printf("请输入圆的半径: ");
    scanf("%f", &radius);

    area = PI * radius * radius;
    printf("圆的面积为: %.2f\n", area);

    return 0;
}
```

思考：为什么要使用 `float` 类型而不是 `int`？

六、总结与延伸

本节重点回顾

1. C语言诞生于1972年，是为开发UNIX操作系统而设计的
2. C语言的核心特点：高效、可移植、灵活、简洁
3. 学习C语言可以帮助理解计算机底层原理
4. C语言是学习其他编程语言的基础

下节预告

下一篇文章我们将学习《开发环境搭建：从0到1配置你的第一个C语言项目》，带你一步步配置开发环境，写出你的第一个C语言程序。

📌 知识点卡片

- C语言诞生于1972年的贝尔实验室
- 核心特点：高效、可移植、灵活、简洁
- 学习C语言是理解计算机原理的最佳途径
- 几乎所有现代操作系统都使用C语言编写

💡 小技巧

- 养成良好的代码风格，使用有意义的变量名
- 编写代码时添加适当的注释
- 学会使用调试工具，这是程序员必备技能

⚠️ 注意事项

- C语言不提供自动内存管理，需要手动释放内存
- 指针操作需要格外小心，避免内存错误
- 注意数据类型的范围，避免溢出问题

如果你有任何问题或想法，欢迎在评论区留言讨论！下一篇见！

02 C语言开发环境搭建：从安装到Hello World

准备好你的编程战场

在上一篇文章中，我们了解了C语言的重要性和学习价值。现在，是时候动手搭建我们的开发环境了。一个好的开发环境是高效编程的基础，让我们一起从0到1配置一个专业的C语言开发环境。

一、开发环境核心组件

1.1 核心组件介绍

一个完整的C语言开发环境需要以下几个组件：

组件	作用	推荐工具
编译器	将C代码转换为机器可执行文件	GCC、Clang、MSVC
编辑器	编写和编辑代码	VS Code、Vim、Emacs
调试器	调试和排查程序错误	GDB、LLDB
构建工具	管理项目编译流程	Make、CMake

1.2 选择合适的工具

编译器选择建议：

- Windows**：MinGW-w64 (GCC) 或MSVC
- macOS**：Xcode Command Line Tools (Clang)
- Linux**：GCC (通常预装)

编辑器选择建议：

- 初学者**：VS Code (界面友好，插件丰富)
- 进阶用户**：Vim/Emacs (高度可定制，效率高)

二、Windows环境配置

2.1 安装MinGW-w64

MinGW-w64是Windows平台上最流行的GCC编译器套件。

安装步骤：

1. 下载MinGW-w64

- 访问官网：<https://sourceforge.net/projects/mingw-w64/>
- 选择适合你系统的版本 (32位或64位)
- 推荐使用在线安装器：`mingw-w64-install.exe`

2. 配置安装选项

- **Architecture:** x86_64 (64位系统) 或 i686 (32位系统)
- **Thread model:** posix
- **Exception model:** seh (64位) 或 sjlj (32位)
- **Build revision:** 保持默认

3. 添加到系统环境变量

- 找到MinGW-w64的安装目录, 通常是 `C:\mingw64\bin`
- 打开"系统属性" → "高级" → "环境变量"
- 在"系统变量"的"Path"中添加上述路径
- 重启终端使配置生效

4. 验证安装

```
gcc --version
g++ --version
gdb --version
```

2.2 安装VS Code

1. 下载VS Code

- 访问官网: <https://code.visualstudio.com/>
- 下载并安装适合Windows的版本

2. 安装C/C++插件

- 打开VS Code, 点击左侧扩展图标
- 搜索"Extension Pack for C++"并安装
- 这会自动安装C/C++、C/C++ Themes等必要插件

2.3 配置VS Code

1. 创建工作目录

```
mkdir C:\CProjects\HelloWorld
cd C:\CProjects\HelloWorld
```

2. 创建tasks.json (编译配置)

- 在VS Code中按 `Ctrl+Shift+B`
- 选择"创建tasks.json文件"
- 选择"Others"模板
- 修改为以下内容:

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "gcc",
      "type": "shell",
      "command": "gcc",
      "args": [
        "-g",
        "${file}",

```

```

        "-o",
        "${fileDirname}\\${fileBasenameNoExtension}.exe"
    ],
    "group": {
        "kind": "build",
        "isDefault": true
    },
    "problemMatcher": ["$gcc"]
  }
]
}

```

3. 创建launch.json (调试配置)

- 点击左侧"运行和调试"图标
- 选择"创建launch.json文件"
- 选择"GDB/LLDB"环境
- 修改为以下内容:

```

{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "gcc - 生成和调试活动文件",
      "type": "cppvsdbg",
      "request": "launch",
      "program": "${fileDirname}\\${fileBasenameNoExtension}.exe",
      "args": [],
      "stopAtEntry": false,
      "cwd": "${fileDirname}",
      "environment": [],
      "externalConsole": true,
      "preLaunchTask": "gcc"
    }
  ]
}

```

三、macOS环境配置

3.1 安装Xcode Command Line Tools

macOS自带Clang编译器，只需安装Command Line Tools即可。

```

# 安装Command Line Tools
xcode-select --install

# 验证安装
clang --version
gcc --version # 这会指向Clang

```

3.2 安装VS Code

1. 下载VS Code

- 访问官网: <https://code.visualstudio.com/>
- 下载并安装

2. 安装C/C++插件

- 搜索"Extension Pack for C++"并安装

3.3 配置VS Code

1. 创建工作目录

```
mkdir ~/CProjects/HelloWorld  
cd ~/CProjects/HelloWorld
```

2. 创建tasks.json

```
{  
  "version": "2.0.0",  
  "tasks": [  
    {  
      "label": "clang",  
      "type": "shell",  
      "command": "clang",  
      "args": [  
        "-g",  
        "${file}",  
        "-o",  
        "${fileDirname}/${fileBasenameNoExtension}"  
      ],  
      "group": {  
        "kind": "build",  
        "isDefault": true  
      },  
      "problemMatcher": ["$gcc"]  
    }  
  ]  
}
```

3. 创建launch.json

```
{  
  "version": "0.2.0",  
  "configurations": [  
    {  
      "name": "clang - 生成和调试活动文件",  
      "type": "lldb",  
      "request": "launch",  
      "program": "${fileDirname}/${fileBasenameNoExtension}",  
      "args": [],  
      "cwd": "${fileDirname}",  
      "preLaunchTask": "clang"  
    }  
  ]  
}
```

```
]
}
```

四、Linux环境配置

4.1 安装GCC

大多数Linux发行版都预装了GCC，如果没有：

Debian/Ubuntu系列：

```
sudo apt update
sudo apt install gcc g++ gdb make
```

Fedora/RHEL系列：

```
sudo dnf install gcc gcc-c++ gdb make
```

验证安装：

```
gcc --version
gdb --version
```

4.2 安装VS Code

```
# Debian/Ubuntu
sudo apt install code

# Fedora
sudo dnf install code
```

或者从官网下载.deb或.rpm包安装。

4.3 配置VS Code

配置方式与macOS类似，使用gcc作为编译器。

五、编写第一个C语言程序

5.1 创建Hello World程序

在VS Code中创建新文件 `hello.c`：

```
#include <stdio.h>

int main() {
    // 输出Hello World
    printf("Hello, World!\n");

    // 输出中文
    printf("你好, 世界! \n");

    return 0;
}
```

5.2 编译运行

方法一：使用VS Code快捷键

- 按 `Ctrl+Shift+B` 编译
- 按 `F5` 调试运行

方法二：使用终端命令

```
# 编译
gcc hello.c -o hello

# 运行 (Windows)
hello.exe

# 运行 (macOS/Linux)
./hello
```

5.3 预期输出

```
Hello, World!
你好, 世界!
```

六、常见问题解决

6.1 编译器找不到

问题： `'gcc'` 不是内部或外部命令，也不是可运行的程序

解决：

- 检查MinGW-w64是否正确安装
- 确保 `bin` 目录已添加到系统Path
- 重启终端或VS Code

6.2 中文输出乱码

问题： 中文输出显示为乱码

解决：

- Windows: 确保终端使用UTF-8编码

```
chcp 65001
```

- macOS/Linux: 通常默认支持UTF-8

6.3 VS Code调试时无法找到程序

问题: 调试时提示"无法找到程序"

解决:

- 确保tasks.json和launch.json配置正确
- 确保先编译成功再调试
- 检查program路径是否正确

七、实战练习

练习1: 配置开发环境

按照教程配置你的开发环境, 确保能够编译和运行Hello World程序。

练习2: 输出个人信息

修改Hello World程序, 输出你的姓名、学号和专业:

```
#include <stdio.h>

int main() {
    printf("===== 个人信息 =====\n");
    printf("姓名: 张三\n");
    printf("学号: 2024001001\n");
    printf("专业: 计算机科学与技术\n");
    printf("===== \n");
    return 0;
}
```

要求:

1. 创建新文件 `info.c`
2. 编译并运行
3. 截图保存运行结果

练习3: 计算两个数的和

编写程序, 输入两个整数, 计算并输出它们的和:

```
#include <stdio.h>

int main() {
    int a, b, sum;

    printf("请输入第一个整数: ");
```

```
scanf("%d", &a);

printf("请输入第二个整数: ");
scanf("%d", &b);

sum = a + b;
printf("两数之和为: %d\n", sum);

return 0;
}
```

思考: `scanf` 函数中的 `&` 符号是什么意思?

八、总结与延伸

本节重点回顾

1. C语言开发环境需要编译器、编辑器、调试器和构建工具
2. Windows推荐使用MinGW-w64 + VS Code
3. macOS使用Xcode Command Line Tools + VS Code
4. Linux通常预装GCC, 直接使用即可
5. 学会使用tasks.json和launch.json配置编译和调试

下节预告

下一篇文章我们将学习《Hello World背后: C语言程序的执行原理》, 深入理解程序从源码到运行的全过程。

📌 知识点卡片

- 核心组件: 编译器(GCC/Clang)、编辑器(VS Code)、调试器(GDB)
- Windows推荐MinGW-w64, macOS使用Clang, Linux使用GCC
- VS Code需要配置tasks.json (编译) 和launch.json (调试)
- 编译命令: `gcc 源文件.c -o 输出文件`
- 运行命令: `./输出文件` (Linux/macOS) 或 `输出文件.exe` (Windows)

💡 小技巧

- 使用 `gcc -g` 生成调试信息, 便于GDB调试
- 养成使用 `-Wall` 参数的习惯, 开启更多警告信息
- 学习使用Makefile管理复杂项目

⚠️ 注意事项

- 确保编译器路径已添加到系统环境变量
- 注意代码文件的编码格式 (推荐UTF-8)
- 调试前必须先编译成功

如果你在配置过程中遇到问题, 欢迎在评论区留言! 下一篇见!

03 Hello World背后：C语言程序的执行原理

揭开程序运行的神秘面纱

在上一篇文章中，我们成功编译并运行了第一个C语言程序。当你看到屏幕上输出"Hello, World!"时，是否好奇这背后发生了什么？从你编写的源代码到最终的运行结果，中间经历了哪些神奇的过程？

这正是我们今天要探讨的话题——C语言程序的执行原理。

一、程序的生命周期

一个C语言程序从诞生到运行，需要经历以下几个阶段：

源代码(.c) → 预处理 → 编译 → 汇编 → 链接 → 可执行文件 → 加载运行

1.1 完整的编译流程

```
hello.c
  ↓
预处理 (cpp)
  ↓
hello.i (预处理后的C代码)
  ↓
编译 (cc1)
  ↓
hello.s (汇编代码)
  ↓
汇编 (as)
  ↓
hello.o (目标文件/对象文件)
  ↓
链接 (ld)
  ↓
hello.exe (可执行文件)
  ↓
加载运行 (操作系统)
```

1.2 GCC编译过程详解

使用GCC编译时，可以通过不同参数查看各个阶段的输出：

```
# 1. 预处理阶段 (生成.i文件)
gcc -E hello.c -o hello.i

# 2. 编译阶段 (生成.s汇编文件)
gcc -S hello.c -o hello.s

# 3. 汇编阶段 (生成.o目标文件)
gcc -c hello.c -o hello.o

# 4. 链接阶段 (生成可执行文件)
```

```
gcc hello.o -o hello

# 一步完成所有阶段
gcc hello.c -o hello
```

二、预处理阶段

2.1 预处理器的作用

预处理阶段由预处理器(cpp)完成，主要处理以#开头的预处理指令：

```
// 原始代码
#include <stdio.h>

#define PI 3.14159

int main() {
    printf("PI = %f\n", PI);
    return 0;
}
```

预处理后的代码（简化版）：

```
// stdio.h的内容被展开
extern int printf(const char *__format, ...);

int main() {
    printf("PI = %f\n", 3.14159);
    return 0;
}
```

2.2 常见预处理指令

指令	作用	示例
#include	包含头文件	#include <stdio.h>
#define	宏定义	#define MAX 100
#ifdef/#endif	条件编译	#ifdef DEBUG ... #endif
#ifndef/#endif	如果未定义则编译	#ifndef NULL ... #endif
#undef	取消宏定义	#undef MAX
#pragma	编译器指令	#pragma once

三、编译阶段

3.1 编译器的工作

编译阶段将预处理后的C代码转换为汇编代码。编译器(cc1)会进行：

1. **词法分析**：将源代码分解为token（关键字、标识符、常量等）
2. **语法分析**：检查语法是否正确，构建语法树
3. **语义分析**：检查语义是否正确（如类型匹配）
4. **优化**：优化代码以提高性能
5. **代码生成**：生成汇编代码

3.2 汇编代码示例

以下是 `hello.c` 编译后生成的汇编代码（x86-64 Linux）：

```
.file "hello.c"
.text
.section .rodata
.LC0:
.string "Hello, World!"
.text
.globl main
.type main, @function
main:
.LFB0:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
movl $.LC0, %edi
call puts
movl $0, %eax
popq %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc
.LFE0:
.size main, .-main
.ident "GCC: (Ubuntu 9.4.0-1ubuntu1~20.04.1) 9.4.0"
.section .note.GNU-stack,"",@progbits
```

四、汇编阶段

4.1 汇编器的工作

汇编阶段将汇编代码转换为机器码（二进制），生成目标文件(.o)。

目标文件包含：

- **代码段(.text)**：存放程序指令
- **数据段(.data)**：存放已初始化的全局变量和静态变量
- **BSS段(.bss)**：存放未初始化的全局变量和静态变量（运行时初始化为0）

- **符号表**：存放函数和变量的名称及其地址

4.2 目标文件的结构

ELF Header	文件头信息
.text section	代码段（程序指令）
.data section	数据段（已初始化数据）
.bss section	BSS段（未初始化数据）
Symbol Table	符号表
Relocation Table	重定位表

五、链接阶段

5.1 链接器的工作

链接阶段将多个目标文件和库文件合并，生成可执行文件。链接器(ld)主要完成：

1. **符号解析**：解析目标文件中引用的外部符号（如 `printf`）
2. **重定位**：将符号引用替换为实际地址

5.2 静态链接与动态链接

静态链接：将库代码直接复制到可执行文件中

```
gcc -static hello.c -o hello_static
```

动态链接：运行时才加载库文件

```
gcc hello.c -o hello_dynamic # 默认方式
```

链接方式	优点	缺点
静态链接	运行时不依赖外部库，移植性好	可执行文件体积大，更新困难
动态链接	可执行文件体积小，便于更新	运行时需要依赖库文件

六、程序的加载与运行

6.1 操作系统的加载过程

当你运行可执行文件时，操作系统会：

1. **创建进程**：为程序分配独立的内存空间
2. **加载代码**：将可执行文件的代码段加载到内存
3. **加载数据**：将数据段和BSS段加载到内存
4. **设置栈**：为程序栈分配空间
5. **跳转到main函数**：开始执行程序

6.2 程序的内存布局

一个运行中的C程序在内存中通常分为以下几个区域：



6.3 main函数的执行

`main` 函数是程序的入口，但并不是真正的起点。在 `main` 函数执行前，还有一段启动代码(`_start`)负责：

1. 初始化运行时环境
2. 处理命令行参数
3. 调用 `main` 函数
4. 处理 `main` 函数的返回值

```
// 简化的启动代码逻辑
int _start() {
    // 初始化
    // ...

    // 获取命令行参数
    int argc = ...;
    char **argv = ...;

    // 调用main函数
    int ret = main(argc, argv);

    // 退出程序
    exit(ret);
}
```

七、深入理解printf的工作原理

7.1 printf函数的执行流程

当你调用 `printf("Hello, World!\n")` 时：

1. `printf` 函数接收到字符串地址
2. 遍历字符串中的每个字符
3. 遇到普通字符直接输出
4. 遇到格式说明符（如 `%d`、`%f`）则解析对应参数并格式化输出
5. 遇到 `\n` 换行符时刷新输出缓冲区

7.2 缓冲区机制

C语言的标准输出默认是**行缓冲**的：

- 遇到换行符 `\n` 时刷新缓冲区
- 缓冲区满时自动刷新
- 程序结束时自动刷新

```
#include <stdio.h>

int main() {
    printf("Hello"); // 不会立即输出，留在缓冲区
    printf(" ");      // 不会立即输出
    printf("World!\n"); // 遇到换行符，刷新缓冲区，输出"Hello World!"

    // 或者手动刷新
    printf("Hello");
    fflush(stdout); // 手动刷新缓冲区

    return 0;
}
```

八、实战练习

练习1：观察编译过程

使用以下命令观察编译的各个阶段：

```
# 1. 查看预处理结果
gcc -E hello.c -o hello.i
cat hello.i

# 2. 生成汇编代码
gcc -S hello.c -o hello.s
cat hello.s

# 3. 生成目标文件
gcc -c hello.c -o hello.o

# 4. 查看目标文件信息
```

```
objdump -h hello.o # 查看段信息
nm hello.o         # 查看符号表

# 5. 链接生成可执行文件
gcc hello.o -o hello

# 6. 查看可执行文件信息
objdump -h hello   # 查看段信息
ldd hello          # 查看动态链接库依赖
```

练习2: 理解内存布局

编写程序观察变量在内存中的位置:

```
#include <stdio.h>

int global_var;           // 在.bss段 (未初始化)
int global_init = 100;    // 在.data段 (已初始化)

int main() {
    int local_var;        // 在栈上
    static int static_var; // 在.bss段
    static int static_init = 200; // 在.data段
    int *heap_var = (int *)malloc(sizeof(int)); // 在堆上

    printf("全局变量 (未初始化) : %p\n", &global_var);
    printf("全局变量 (已初始化) : %p\n", &global_init);
    printf("静态变量 (未初始化) : %p\n", &static_var);
    printf("静态变量 (已初始化) : %p\n", &static_init);
    printf("局部变量: %p\n", &local_var);
    printf("堆变量: %p\n", heap_var);

    free(heap_var);
    return 0;
}
```

观察输出: 比较不同类型变量的地址, 理解内存布局。

练习3: 理解缓冲区

运行以下程序, 观察输出顺序:

```
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Hello");
    printf(" ");
    sleep(2); // 暂停2秒
    printf("World!\n");

    return 0;
}
```

思考：为什么"Hello "会在2秒后才和"World!"一起输出？

九、总结与延伸

本节重点回顾

1. C语言程序的编译流程：预处理→编译→汇编→链接
2. 预处理器处理 `#` 开头的指令，展开头文件和宏
3. 编译器将C代码转换为汇编代码
4. 汇编器将汇编代码转换为机器码（目标文件）
5. 链接器合并目标文件和库，生成可执行文件
6. 程序运行时的内存布局：栈、堆、.data、.bss、.text

下节预告

下一篇文章我们将学习《数据类型与变量：编程的基本积木》，深入理解C语言的数据类型系统和变量的使用方法。

📌 知识点卡片

- 编译流程：预处理(.i) → 编译(.s) → 汇编(.o) → 链接(可执行文件)
- 预处理器指令：`#include`、`#define`、`#ifdef`等
- 目标文件包含：`.text`（代码）、`.data`（已初始化数据）、`.bss`（未初始化数据）
- 链接方式：静态链接（复制库代码）、动态链接（运行时加载）
- 内存布局：栈（局部变量）、堆（动态分配）、`.data`、`.bss`、`.text`

💡 小技巧

- 使用 `gcc -E` 查看预处理结果
- 使用 `nm` 查看目标文件的符号表
- 使用 `ldd` 查看动态链接库依赖
- 使用 `objdump` 查看可执行文件的段信息

⚠️ 注意事项

- 全局变量和静态变量在程序启动时就已分配内存
- 局部变量在函数调用时分配在栈上，函数返回时释放
- 堆内存需要手动分配(`malloc`)和释放(`free`)
- 标准输出默认是行缓冲，遇到 `\n` 才刷新

如果你对程序执行原理有任何疑问，欢迎在评论区留言讨论！下一篇见！

04 数据类型与变量：编程的基本积木

构建程序的基石

在上一篇文章中，我们深入了解了C语言程序的执行原理。现在，让我们回到编程的基础——数据类型与变量。它们是构建任何程序的基本积木，就像盖房子需要砖块一样，编写程序需要数据类型和变量来存储和处理信息。

一、数据类型的概念

1.1 什么是数据类型

数据类型决定了数据的存储方式和可以进行的操作。C语言是一种**强类型语言**，每个变量在声明时必须指定其类型。

```
int age = 25;           // 整数类型
float height = 1.75;    // 单精度浮点类型
char grade = 'A';       // 字符类型
```

1.2 为什么需要数据类型

- 1. **内存分配**：不同类型占用不同大小的内存空间
- 2. **操作限制**：不同类型支持不同的操作（如整数可以做除法，字符串可以拼接）
- 3. **类型安全**：防止不合法的操作（如不能将字符串和整数直接相加）

二、C语言的基本数据类型

2.1 基本类型分类

C语言的基本数据类型可以分为四大类：

类别	类型	关键字	典型用途
整型	字符型	char	存储单个字符
	短整型	short	存储较小范围的整数
	整型	int	存储普通整数
	长整型	long	存储较大范围的整数
	长长整型	long long	存储非常大的整数
浮点型	单精度浮点	float	存储小数（精度较低）
	双精度浮点	double	存储小数（精度较高）
	长双精度浮点	long double	存储高精度小数
指针类型	指针	*	存储内存地址
空类型	空	void	表示无类型

2.2 整型数据类型

整型用于存储整数，可以是正数、负数或零。

```
#include <stdio.h>
```

```

int main() {
    char c = 'A';           // 字符型，实际存储ASCII码值
    short s = 32767;        // 短整型，范围约-32768到32767
    int i = 2147483647;      // 整型，范围约-21亿到21亿
    long l = 9223372036854775807L; // 长整型，L后缀表示长整型
    long long ll = 9223372036854775807LL; // 长长整型

    printf("char: %c (%d)\n", c, (int)c);    // 输出字符和ASCII码
    printf("short: %d\n", s);
    printf("int: %d\n", i);
    printf("long: %ld\n", l);
    printf("long long: %lld\n", ll);

    return 0;
}

```

2.3 浮点型数据类型

浮点型用于存储带有小数部分的数值。

```

#include <stdio.h>

int main() {
    float f = 3.14f;        // 单精度浮点，f后缀表示float
    double d = 3.1415926535; // 双精度浮点，默认是double
    long double ld = 3.141592653589793238L; // 长双精度

    printf("float: %.6f\n", f);
    printf("double: %.10f\n", d);
    printf("long double: %.15Lf\n", ld);

    return 0;
}

```

2.4 字符类型

字符类型 `char` 用于存储单个字符，实际上存储的是该字符的ASCII码值。

```

#include <stdio.h>

int main() {
    char letter = 'A';      // 存储字符'A'
    char digit = '5';       // 存储字符'5'
    char newline = '\n';    // 转义字符，换行符

    printf("字母: %c (ASCII: %d)\n", letter, letter);
    printf("数字字符: %c (ASCII: %d)\n", digit, digit);

    // 字符运算
    char lowercase = letter + 32; // 'A' + 32 = 'a'
    printf("小写字母: %c\n", lowercase);

    return 0;
}

```

```
}
```

2.5 转义字符

转义字符用于表示一些特殊字符：

转义字符	含义
<code>\n</code>	换行符
<code>\t</code>	制表符
<code>\r</code>	回车符
<code>\\</code>	反斜杠
<code>\'</code>	单引号
<code>\"</code>	双引号
<code>\0</code>	空字符（字符串结束标志）

三、变量的定义与使用

3.1 变量的声明与初始化

变量声明：告诉编译器变量的名称和类型

变量初始化：给变量赋初始值

```
#include <stdio.h>

int main() {
    // 声明并初始化
    int age = 25;
    float weight = 65.5f;
    char gender = 'M';

    // 先声明后赋值
    int score;
    score = 95;

    // 多个变量同时声明
    int a = 1, b = 2, c = 3;

    printf("年龄: %d\n", age);
    printf("体重: %.1fkg\n", weight);
    printf("性别: %c\n", gender);
    printf("分数: %d\n", score);

    return 0;
}
```

3.2 变量命名规则

1. 必须以字母或下划线开头
2. 只能包含字母、数字和下划线
3. 不能是C语言的关键字
4. 区分大小写

```
// 合法的变量名
int age;
int my_age;
int _age;
int Age;
int age2;

// 不合法的变量名
// int 1age;    // 不能以数字开头
// int my-age;  // 不能包含连字符
// int int;     // 不能是关键字
// int My Age;  // 不能包含空格
```

3.3 变量的作用域

作用域是指变量可以被访问的范围。

```
#include <stdio.h>

int global_var = 100; // 全局变量，整个程序都可以访问

int main() {
    int local_var = 200; // 局部变量，仅在main函数内有效

    printf("全局变量: %d\n", global_var);
    printf("局部变量: %d\n", local_var);

    {
        int block_var = 300; // 块级变量，仅在花括号内有效
        printf("块级变量: %d\n", block_var);
    }
    // printf("块级变量: %d\n", block_var); // 错误! block_var在此不可访问

    return 0;
}
```

3.4 变量的存储类型

C语言提供了存储类型修饰符来控制变量的存储方式：

修饰符	含义
auto	自动变量（默认），存储在栈上
static	静态变量，存储在静态存储区，程序生命周期内保持存在
extern	外部变量，引用其他文件中定义的变量
register	寄存器变量，建议编译器将变量存储在寄存器中

```
#include <stdio.h>

void counter() {
    auto int auto_count = 0;    // 每次调用都重新初始化为0
    static int static_count = 0; // 只初始化一次，值会保留

    auto_count++;
    static_count++;

    printf("auto_count: %d, static_count: %d\n", auto_count, static_count);
}

int main() {
    counter(); // auto_count: 1, static_count: 1
    counter(); // auto_count: 1, static_count: 2
    counter(); // auto_count: 1, static_count: 3

    return 0;
}
```

四、数据类型的大小与范围

4.1 使用sizeof获取类型大小

sizeof 运算符用于获取变量或类型所占的字节数。

```
#include <stdio.h>

int main() {
    printf("char: %zu bytes\n", sizeof(char));
    printf("short: %zu bytes\n", sizeof(short));
    printf("int: %zu bytes\n", sizeof(int));
    printf("long: %zu bytes\n", sizeof(long));
    printf("long long: %zu bytes\n", sizeof(long long));
    printf("float: %zu bytes\n", sizeof(float));
    printf("double: %zu bytes\n", sizeof(double));
    printf("long double: %zu bytes\n", sizeof(long double));

    return 0;
}
```

```
}
```

4.2 常见数据类型的大小和范围

类型	大小（字节）	范围
<code>char</code>	1	-128 到 127 或 0 到 255（取决于是否有符号）
<code>short</code>	2	-32768 到 32767
<code>int</code>	4	-2147483648 到 2147483647
<code>long</code>	4 或 8	取决于系统
<code>long long</code>	8	-9223372036854775808 到 9223372036854775807
<code>float</code>	4	约 $\pm 3.4 \times 10^38$
<code>double</code>	8	约 $\pm 1.7 \times 10^{308}$

五、类型转换

5.1 隐式类型转换

编译器自动进行的类型转换：

```
#include <stdio.h>

int main() {
    int i = 10;
    float f = 3.14f;

    // 隐式转换: int -> float
    float result = i + f; // i被转换为float
    printf("result: %f\n", result); // 输出13.140000

    // 隐式转换: float -> int (截断小数部分)
    int truncated = i + f; // 结果13.14被截断为13
    printf("truncated: %d\n", truncated); // 输出13

    return 0;
}
```

5.2 显式类型转换（强制转换）

程序员明确指定的类型转换：

```
#include <stdio.h>

int main() {
    float pi = 3.14159f;
```

```

// 显式转换: float -> int
int integer_part = (int)pi;
printf("整数部分: %d\n", integer_part); // 输出3

// 显式转换: int -> float
int num = 5;
float decimal = (float)num / 2; // 结果是2.5而不是2
printf("5/2 = %f\n", decimal); // 输出2.500000

return 0;
}

```

六、常量

6.1 字面常量

直接写在代码中的值:

```

#include <stdio.h>

int main() {
    printf("整数常量: %d\n", 123);
    printf("浮点常量: %f\n", 3.14);
    printf("字符常量: %c\n", 'A');
    printf("字符串常量: %s\n", "Hello");

    return 0;
}

```

6.2 使用#define定义常量

```

#include <stdio.h>

#define PI 3.14159
#define MAX_SIZE 100
#define AUTHOR "张三"

int main() {
    float radius = 5.0f;
    float area = PI * radius * radius;

    printf("圆的面积: %.2f\n", area);
    printf("最大尺寸: %d\n", MAX_SIZE);
    printf("作者: %s\n", AUTHOR);

    return 0;
}

```

6.3 使用const定义常量

```
#include <stdio.h>

int main() {
    const float PI = 3.14159;
    const int MAX_SIZE = 100;

    float radius = 5.0f;
    float area = PI * radius * radius;

    printf("圆的面积: %.2f\n", area);

    // PI = 3.14; // 错误! const变量不能被修改

    return 0;
}
```

6.4 #define与const的区别

特性	#define	const
处理阶段	预处理阶段	编译阶段
类型检查	无类型检查	有类型检查
内存占用	不占用内存（宏替换）	占用内存
作用域	文件作用域	块作用域

七、实战练习

练习1: 计算圆的周长和面积

```
#include <stdio.h>

#define PI 3.14159

int main() {
    float radius;

    printf("请输入圆的半径: ");
    scanf("%f", &radius);

    float circumference = 2 * PI * radius;
    float area = PI * radius * radius;

    printf("圆的周长: %.2f\n", circumference);
    printf("圆的面积: %.2f\n", area);

    return 0;
}
```


练习2：温度转换

编写程序将华氏温度转换为摄氏温度：

```
#include <stdio.h>

int main() {
    float fahrenheit;

    printf("请输入华氏温度: ");
    scanf("%f", &fahrenheit);

    float celsius = (fahrenheit - 32) * 5 / 9;

    printf("%.1f华氏度 = %.1f摄氏度\n", fahrenheit, celsius);

    return 0;
}
```

练习3：变量作用域实验

```
#include <stdio.h>

int x = 10; // 全局变量

void func1() {
    int x = 20; // 局部变量，隐藏全局变量
    printf("func1: x = %d\n", x);
}

void func2() {
    printf("func2: x = %d\n", x); // 使用全局变量
}

int main() {
    printf("main: x = %d\n", x); // 使用全局变量
    func1();
    func2();

    {
        int x = 30; // 块级变量
        printf("block: x = %d\n", x);
    }

    printf("main: x = %d\n", x); // 回到全局变量

    return 0;
}
```

思考：预测程序的输出结果，并解释原因。

八、总结与延伸

本节重点回顾

1. C语言的基本数据类型：整型(char, short, int, long, long long)、浮点型(float, double, long double)
2. 变量的声明、初始化和命名规则
3. 变量的作用域：全局变量、局部变量、块级变量
4. 存储类型：auto、static、extern、register
5. 使用sizeof获取类型大小
6. 类型转换：隐式转换和显式转换
7. 常量：字面常量、#define宏常量、const常量

下节预告

下一篇文章我们将学习《运算符与表达式：程序的计算逻辑》，深入理解C语言中的各种运算符和表达式的计算规则。

📌 知识点卡片

- 基本数据类型：char(1字节)、short(2字节)、int(4字节)、long(4/8字节)、long long(8字节)、float(4字节)、double(8字节)
- 变量命名：字母/下划线开头，只能包含字母、数字、下划线，区分大小写
- 作用域：全局变量（整个程序）、局部变量（函数内）、块级变量（花括号内）
- static变量：只初始化一次，值在函数调用间保持
- const变量：只读，不能被修改
- sizeof：获取类型或变量的字节数

💡 小技巧

- 初始化变量是良好的编程习惯，避免使用未初始化的变量
- 使用const代替#define可以获得类型检查
- 注意隐式类型转换可能导致的精度损失
- 对于需要频繁访问的变量，可以使用register修饰符

⚠️ 注意事项

- 不同系统上数据类型的大小可能不同
- char类型可能是有符号或无符号的，取决于编译器
- 浮点数存在精度问题，比较时要小心
- 不要在头文件中定义全局变量，使用extern声明

如果你对数据类型和变量有任何疑问，欢迎在评论区留言讨论！下一篇见！

05 运算符与表达式：程序的计算逻辑

让程序学会计算

在上一篇文章中，我们学习了数据类型和变量的基本概念。现在，让我们更进一步——学习运算符和表达式。运算符就像是程序的“动词”，它们告诉计算机如何处理数据；而表达式则是由运算符和操作数组成的“句子”，用来表达计算逻辑。

一、运算符的分类

C语言提供了丰富的运算符，可以分为以下几大类：

类别	运算符	示例
算术运算符	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code>	<code>a + b</code> , <code>a % b</code>
关系运算符	<code>></code> , <code><</code> , <code>>=</code> , <code><=</code> , <code>==</code> , <code>!=</code>	<code>a > b</code> , <code>a != 0</code>
逻辑运算符	<code>&&</code> , <code> </code> , <code>!</code>	<code>a && b</code> , <code>!flag</code>
赋值运算符	<code>=</code> , <code>+=</code> , <code>-=</code> , <code>*=</code> , <code>/=</code>	<code>a = 10</code> , <code>a += 5</code>
位运算符	<code>&</code> , <code> </code> , <code>^</code> , <code>~</code> , <code><<</code> , <code>>></code>	<code>a & b</code> , <code>a << 2</code>
自增自减运算符	<code>++</code> , <code>--</code>	<code>a++</code> , <code>++b</code>
条件运算符	<code>?:</code>	<code>a > b ? a : b</code>
逗号运算符	<code>,</code>	<code>a = 1, b = 2</code>

二、算术运算符

2.1 基本算术运算

```
#include <stdio.h>

int main() {
    int a = 10, b = 3;

    printf("a = %d, b = %d\n", a, b);
    printf("a + b = %d\n", a + b);    // 加法
    printf("a - b = %d\n", a - b);    // 减法
    printf("a * b = %d\n", a * b);    // 乘法
    printf("a / b = %d\n", a / b);    // 除法（整数除法）
    printf("a %% b = %d\n", a % b);   // 取余（模运算）

    return 0;
}
```

输出结果：

```
a = 10, b = 3
a + b = 13
a - b = 7
a * b = 30
a / b = 3        // 整数除法，结果取整
a % b = 1        // 10除以3余1
```

2.2 整数除法与浮点除法

```
#include <stdio.h>

int main() {
    int a = 10, b = 3;
    float c = 10.0f, d = 3.0f;

    printf("整数除法: %d / %d = %d\n", a, b, a / b);
    printf("浮点除法: %.1f / %.1f = %.6f\n", c, d, c / d);

    // 使用强制转换实现浮点除法
    printf("强制转换: %d / %d = %.6f\n", a, b, (float)a / b);

    return 0;
}
```

输出结果:

```
整数除法: 10 / 3 = 3
浮点除法: 10.0 / 3.0 = 3.333333
强制转换: 10 / 3 = 3.333333
```

2.3 取余运算的特性

```
#include <stdio.h>

int main() {
    printf("7 %% 3 = %d\n", 7 % 3);    // 结果: 1
    printf("-7 %% 3 = %d\n", -7 % 3); // 结果: -1
    printf("7 %% -3 = %d\n", 7 % -3); // 结果: 1
    printf("-7 %% -3 = %d\n", -7 % -3); // 结果: -1

    return 0;
}
```

注意: 取余运算的结果符号与被除数相同。

三、关系运算符

3.1 比较运算

关系运算符用于比较两个值，返回布尔值（0表示假，非0表示真）。

```
#include <stdio.h>

int main() {
    int a = 10, b = 5;

    printf("a > b: %d\n", a > b);    // 1 (真)
```

```
printf("a < b: %d\n", a < b);    // 0 (假)
printf("a >= b: %d\n", a >= b); // 1 (真)
printf("a <= b: %d\n", a <= b); // 0 (假)
printf("a == b: %d\n", a == b); // 0 (假)
printf("a != b: %d\n", a != b); // 1 (真)

return 0;
}
```

3.2 常见错误：赋值与相等判断

```
#include <stdio.h>

int main() {
    int x = 5;

    // 错误：使用了赋值运算符=而不是相等运算符==
    if (x = 10) {
        printf("x被赋值为10，条件为真\n");
    }

    // 正确：使用相等运算符==
    if (x == 10) {
        printf("x等于10\n");
    } else {
        printf("x不等于10，x = %d\n", x);
    }

    return 0;
}
```

输出结果：

```
x被赋值为10，条件为真
x等于10
```

提示：为了避免这种错误，可以将常量放在前面：`if (10 == x)`，这样如果误写为`=`会产生编译错误。

四、逻辑运算符

4.1 逻辑运算的真值表

运算符	含义	真值表
&&	逻辑与	两个操作数都为真时结果为真
	逻辑或	任意一个操作数为真时结果为真
!	逻辑非	取反

4.2 逻辑运算示例

```
#include <stdio.h>

int main() {
    int a = 5, b = 0, c = -3;

    // 逻辑与：两个条件都为真
    printf("(a > 0) && (b > 0): %d\n", (a > 0) && (b > 0)); // 0

    // 逻辑或：只要有一个为真
    printf("(a > 0) || (b > 0): %d\n", (a > 0) || (b > 0)); // 1

    // 逻辑非：取反
    printf("!(a > 0): %d\n", !(a > 0)); // 0
    printf("!(b > 0): %d\n", !(b > 0)); // 1

    // 组合逻辑
    printf("(a > 0) && (c < 0): %d\n", (a > 0) && (c < 0)); // 1

    return 0;
}
```

4.3 短路求值

C语言的逻辑运算符支持**短路求值**：

```
#include <stdio.h>

int main() {
    int a = 5, b = 0;

    // 逻辑与的短路求值
    // 第一个条件为假，第二个条件不会被执行
    if ((a < 0) && (++b > 0)) {
        printf("条件为真\n");
    }
    printf("b = %d\n", b); // b = 0, ++b没有执行

    // 逻辑或的短路求值
    // 第一个条件为真，第二个条件不会被执行
    if ((a > 0) || (++b > 0)) {
        printf("条件为真\n");
    }
    printf("b = %d\n", b); // b = 0, ++b仍然没有执行

    return 0;
}
```

五、赋值运算符

5.1 基本赋值

```
int a = 10;    // 直接赋值
int b = a;     // 将a的值赋给b
```

5.2 复合赋值运算符

```
#include <stdio.h>

int main() {
    int a = 10;

    a += 5;    // 等价于 a = a + 5, 结果: 15
    printf("a += 5: %d\n", a);

    a -= 3;    // 等价于 a = a - 3, 结果: 12
    printf("a -= 3: %d\n", a);

    a *= 2;    // 等价于 a = a * 2, 结果: 24
    printf("a *= 2: %d\n", a);

    a /= 4;    // 等价于 a = a / 4, 结果: 6
    printf("a /= 4: %d\n", a);

    a %= 4;    // 等价于 a = a % 4, 结果: 2
    printf("a %= 4: %d\n", a);

    return 0;
}
```

六、自增自减运算符

6.1 前缀与后缀的区别

```
#include <stdio.h>

int main() {
    int a = 5, b = 5;
    int c, d;

    // 前缀自增: 先加1, 再使用
    c = ++a;
    printf("前缀自增: a = %d, c = %d\n", a, c); // a=6, c=6

    // 后缀自增: 先使用, 再加1
    d = b++;
    printf("后缀自增: b = %d, d = %d\n", b, d); // b=6, d=5

    return 0;
}
```

6.2 在表达式中的注意事项

```
#include <stdio.h>

int main() {
    int x = 5;

    // 不要在同一个表达式中多次修改同一个变量
    int result = x++ + ++x; // 行为未定义!

    printf("x = %d, result = %d\n", x, result);

    return 0;
}
```

警告：在同一个表达式中多次修改同一个变量会导致未定义行为，不同编译器可能产生不同结果。

七、位运算符

7.1 位运算的基本概念

位运算符直接操作二进制位：

```
#include <stdio.h>

int main() {
    unsigned char a = 0b00111000; // 56
    unsigned char b = 0b00011100; // 28

    printf("a = %d (0b%08b)\n", a, a);
    printf("b = %d (0b%08b)\n", b, b);
    printf("a & b = %d (0b%08b)\n", a & b, a & b); // 按位与
    printf("a | b = %d (0b%08b)\n", a | b, a | b); // 按位或
    printf("a ^ b = %d (0b%08b)\n", a ^ b, a ^ b); // 按位异或
    printf("~a = %d (0b%08b)\n", ~a & 0xFF, ~a & 0xFF); // 按位取反
    printf("a << 2 = %d (0b%08b)\n", a << 2, a << 2); // 左移
    printf("a >> 2 = %d (0b%08b)\n", a >> 2, a >> 2); // 右移

    return 0;
}
```

输出结果：

```
a = 56 (0b00111000)
b = 28 (0b00011100)
a & b = 24 (0b00011000)
a | b = 60 (0b00111100)
a ^ b = 36 (0b00100100)
~a = 199 (0b11000111)
a << 2 = 224 (0b11100000)
a >> 2 = 14 (0b00001110)
```


7.2 位运算的实用技巧

```
#include <stdio.h>

int main() {
    int x = 10;

    // 判断奇偶性
    if (x & 1) {
        printf("%d 是奇数\n", x);
    } else {
        printf("%d 是偶数\n", x);
    }

    // 交换两个数（不使用临时变量）
    int a = 5, b = 7;
    printf("交换前: a = %d, b = %d\n", a, b);
    a = a ^ b;
    b = a ^ b;
    a = a ^ b;
    printf("交换后: a = %d, b = %d\n", a, b);

    // 快速计算2的幂次
    printf("2^5 = %d\n", 1 << 5); // 32

    return 0;
}
```

八、条件运算符（三元运算符）

8.1 语法与使用

```
#include <stdio.h>

int main() {
    int a = 10, b = 20;

    // 求最大值
    int max = (a > b) ? a : b;
    printf("最大值: %d\n", max); // 20

    // 求最小值
    int min = (a < b) ? a : b;
    printf("最小值: %d\n", min); // 10

    // 判断是否为偶数
    printf("%d是%s\n", a, (a % 2 == 0) ? "偶数" : "奇数"); // 偶数

    return 0;
}
```

8.2 嵌套使用

```
#include <stdio.h>

int main() {
    int score = 85;

    char grade = (score >= 90) ? 'A' :
                 (score >= 80) ? 'B' :
                 (score >= 70) ? 'C' :
                 (score >= 60) ? 'D' : 'F';

    printf("成绩等级: %c\n", grade); // B

    return 0;
}
```

九、运算符的优先级与结合性

9.1 优先级表（从高到低）

优先级	运算符	结合性
1	<code>()</code>	左到右
2	<code>!</code> , <code>~</code> , <code>++</code> , <code>--</code> , <code>(类型)</code>	右到左
3	<code>*</code> , <code>/</code> , <code>%</code>	左到右
4	<code>+</code> , <code>-</code>	左到右
5	<code><<</code> , <code>>></code>	左到右
6	<code><</code> , <code><=</code> , <code>></code> , <code>>=</code>	左到右
7	<code>==</code> , <code>!=</code>	左到右
8	<code>&</code>	左到右
9	<code>^</code>	左到右
10	<code>\ </code>	左到右
11	<code>&&</code>	左到右
12	<code>\ \ </code>	左到右
13	<code>? :</code>	右到左
14	<code>=</code> , <code>+=</code> , <code>-=</code> , ...	右到左
15	<code>,</code>	左到右

9.2 优先级示例

```
#include <stdio.h>

int main() {
    int a = 10, b = 3, c = 2;

    // 没有括号, 按优先级计算
    int result1 = a - b * c; // 等价于 a - (b * c) = 10 - 6 = 4
    printf("a - b * c = %d\n", result1);

    // 使用括号改变优先级
    int result2 = (a - b) * c; // 等价于 (10 - 3) * 2 = 14
    printf("(a - b) * c = %d\n", result2);

    // 复合表达式
    int x = 5, y = 3;
    int result3 = x > y && x + y > 5; // 等价于 (x > y) && ((x + y) > 5)
    printf("result3 = %d\n", result3); // 1

    return 0;
}
```

十、实战练习

练习1: 判断闰年

```
#include <stdio.h>

int main() {
    int year;

    printf("请输入年份: ");
    scanf("%d", &year);

    // 闰年判断规则: 能被4整除但不能被100整除, 或能被400整除
    int is_leap = (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);

    if (is_leap) {
        printf("%d年是闰年\n", year);
    } else {
        printf("%d年不是闰年\n", year);
    }

    return 0;
}
```

练习2: 计算BMI指数

```
#include <stdio.h>
```

```

int main() {
    float weight, height, bmi;

    printf("请输入体重 (kg) : ");
    scanf("%f", &weight);

    printf("请输入身高 (m) : ");
    scanf("%f", &height);

    bmi = weight / (height * height);

    printf("你的BMI指数为: %.1f\n", bmi);

    // 判断BMI等级
    printf("BMI等级: ");
    if (bmi < 18.5) {
        printf("偏瘦\n");
    } else if (bmi < 24) {
        printf("正常\n");
    } else if (bmi < 28) {
        printf("超重\n");
    } else {
        printf("肥胖\n");
    }

    return 0;
}

```

练习3: 位运算实现权限控制

```

#include <stdio.h>

#define READ  1 << 0  // 0b0001
#define WRITE 1 << 1  // 0b0010
#define EXEC  1 << 2  // 0b0100

int main() {
    int permission = READ | WRITE; // 0b0011

    // 添加执行权限
    permission |= EXEC; // 0b0111
    printf("添加执行权限后: 0b%04b\n", permission);

    // 移除写权限
    permission &= ~WRITE; // 0b0101
    printf("移除写权限后: 0b%04b\n", permission);

    // 检查是否有读权限
    if (permission & READ) {
        printf("有读权限\n");
    }

    // 检查是否有写权限
}

```

```
if (permission & WRITE) {
    printf("有写权限\n");
} else {
    printf("没有写权限\n");
}

return 0;
}
```

思考：位运算为什么适合用于权限控制？

十一、总结与延伸

本节重点回顾

1. **算术运算符**：`+`, `-`, `*`, `/`, `%`（注意整数除法和取余的特性）
2. **关系运算符**：`>`, `<`, `>=`, `<=`, `==`, `!=`（注意 `=` 和 `==` 的区别）
3. **逻辑运算符**：`&&`, `||`, `!`（支持短路求值）
4. **赋值运算符**：`=`, `+=`, `-=`, `*=`, `/=`
5. **自增自减运算符**：`++`, `--`（前缀和后缀的区别）
6. **位运算符**：`&`, `|`, `^`, `~`, `<<`, `>>`（直接操作二进制位）
7. **条件运算符**：`? :`（简洁的条件判断）
8. **运算符优先级**：使用括号可以改变运算顺序

下节预告

下一篇文章我们将学习《流程控制：让程序学会做选择》，深入理解if-else、switch等条件语句的使用方法。

📌 知识点卡片

- 整数除法会截断小数部分，使用强制转换可以实现浮点除法
- 取余运算的结果符号与被除数相同
- 逻辑运算符支持短路求值，可能导致部分表达式不执行
- 位运算符直接操作二进制位，常用于底层操作和权限控制
- 三元运算符是唯一的三目运算符，可以简化简单的条件判断
- 当不确定优先级时，使用括号明确表达意图

💡 小技巧

- 用 `& 1` 判断奇偶性比取余运算更快
- 使用 `<<` 和 `>>` 进行乘除2的幂次运算
- 将常量放在 `==` 前面可以避免误写为 `=`
- 复杂表达式使用括号提高可读性

⚠️ 注意事项

- 避免在同一表达式中多次修改同一变量
- 浮点数比较要小心精度问题
- 位运算操作的是补码形式
- 负数右移可能是算术右移（符号位扩展）

06 流程控制：让程序学会做选择

赋予程序决策能力

在上一篇文章中，我们学习了运算符和表达式的使用。现在，让我们进一步学习流程控制语句。流程控制就像是程序的"决策系统"，它让程序能够根据不同的条件执行不同的代码，从而实现复杂的逻辑功能。

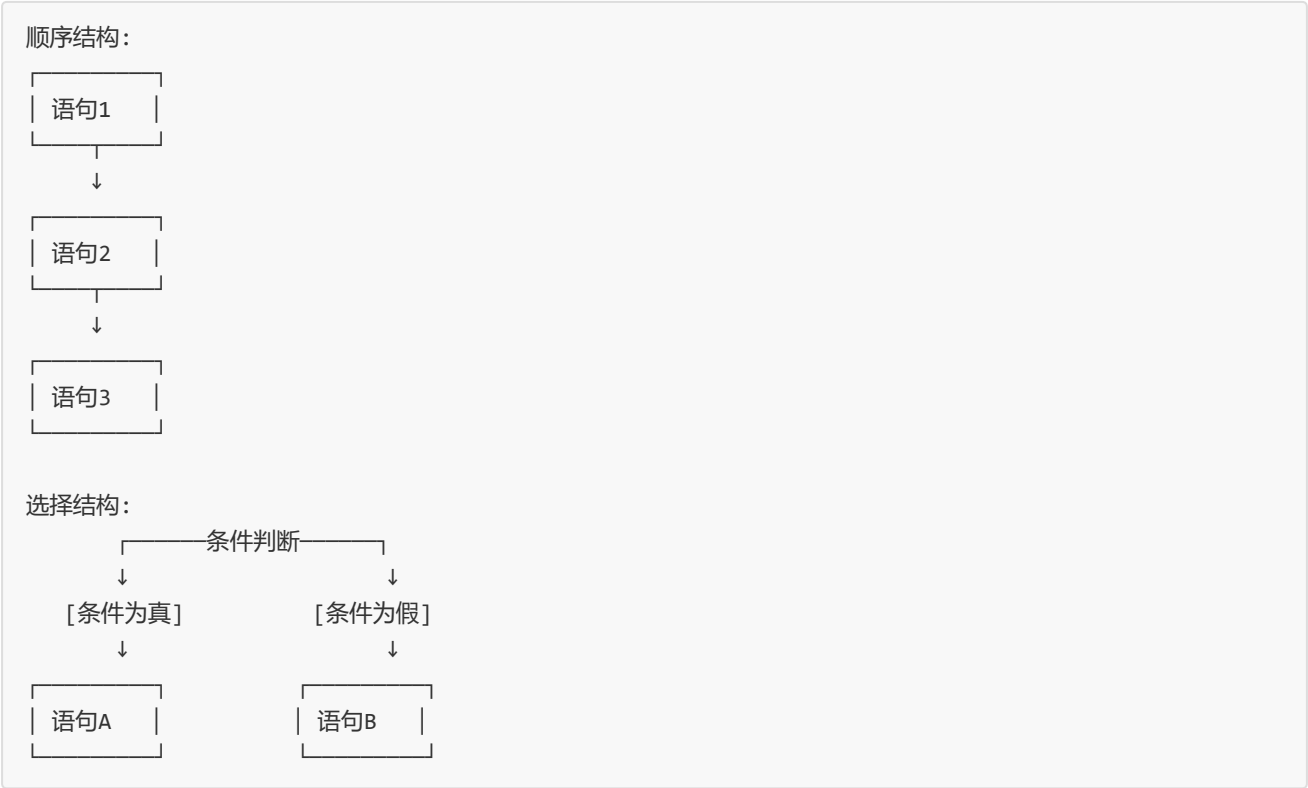
一、流程控制的基本概念

1.1 什么是流程控制

流程控制是指程序执行的顺序和路径。C语言提供了三种基本的流程控制结构：

结构	描述	关键字
顺序结构	按顺序执行	默认
选择结构	根据条件选择执行	if, else, switch
循环结构	重复执行	for, while, do-while

1.2 流程图表示



二、if语句

2.1 基本if语句

```
#include <stdio.h>

int main() {
    int score;

    printf("请输入你的成绩: ");
    scanf("%d", &score);

    if (score >= 60) {
        printf("恭喜你, 及格了! \n");
    }

    printf("程序结束\n");

    return 0;
}
```

2.2 if-else语句

```
#include <stdio.h>

int main() {
    int score;

    printf("请输入你的成绩: ");
    scanf("%d", &score);

    if (score >= 60) {
        printf("恭喜你, 及格了! \n");
    } else {
        printf("很遗憾, 不及格。 \n");
    }

    printf("程序结束\n");

    return 0;
}
```

2.3 嵌套if-else语句

```
#include <stdio.h>

int main() {
    int score;

    printf("请输入你的成绩: ");
    scanf("%d", &score);

    if (score >= 90) {
        printf("优秀! \n");
    }
}
```

```
    } else if (score >= 80) {  
        printf("良好! \n");  
    } else if (score >= 70) {  
        printf("中等! \n");  
    } else if (score >= 60) {  
        printf("及格! \n");  
    } else {  
        printf("不及格! \n");  
    }  
  
    return 0;  
}
```

2.4 if语句的常见错误

```
#include <stdio.h>  
  
int main() {  
    int x = 5;  
  
    // 错误: if后面多了分号  
    if (x > 0); {  
        printf("x大于0\n"); // 这个块总是会执行  
    }  
  
    // 正确写法  
    if (x > 0) {  
        printf("x大于0\n");  
    }  
  
    return 0;  
}
```

三、switch语句

3.1 基本switch语句

```
#include <stdio.h>  
  
int main() {  
    int day;  
  
    printf("请输入星期几(1-7): ");  
    scanf("%d", &day);  
  
    switch (day) {  
        case 1:  
            printf("星期一\n");  
            break;  
        case 2:  
            printf("星期二\n");  
            break;  
        // ...  
    }  
}
```



```

        break;
    case 3:
        printf("星期三\n");
        break;
    case 4:
        printf("星期四\n");
        break;
    case 5:
        printf("星期五\n");
        break;
    case 6:
        printf("星期六\n");
        break;
    case 7:
        printf("星期日\n");
        break;
    default:
        printf("输入错误! \n");
}

return 0;
}

```

3.2 switch语句的执行流程

```

switch(表达式)
|
|→ case 常量1:
|   语句1
|   break; ← 跳出switch
|
|→ case 常量2:
|   语句2
|   break;
|
|→ ...
|
|→ default:
|   默认语句

```

3.3 switch语句中的case穿透

```

#include <stdio.h>

int main() {
    int grade;

    printf("请输入成绩等级(A-E): ");
    scanf("%c", &grade);

    switch (grade) {

        case 'A':

```

```
        case 'B':
        case 'C':
            printf("成绩合格\n");
            break;
        case 'D':
        case 'E':
            printf("成绩不合格\n");
            break;
        default:
            printf("输入错误\n");
    }

    return 0;
}
```

3.4 if-else vs switch

特性	if-else	switch
条件类型	任意表达式	只能是整型或字符型常量
判断方式	连续条件判断	直接跳转
可读性	适合复杂条件	适合多分支选择
效率	逐个判断	跳转表，效率更高

四、条件运算符（三元运算符）

4.1 简化的条件判断

```
#include <stdio.h>

int main() {
    int a = 10, b = 20;

    // 使用三元运算符求最大值
    int max = (a > b) ? a : b;
    printf("最大值: %d\n", max);

    // 使用三元运算符输出结果
    printf("较大的数是: %d\n", (a > b) ? a : b);

    return 0;
}
```

4.2 嵌套三元运算符

```
#include <stdio.h>
```

```
int main() {
    int score = 85;

    char grade = (score >= 90) ? 'A' :
                 (score >= 80) ? 'B' :
                 (score >= 70) ? 'C' :
                 (score >= 60) ? 'D' : 'F';

    printf("成绩等级: %c\n", grade);

    return 0;
}
```

五、实战练习

练习1: 判断奇偶性

```
#include <stdio.h>

int main() {
    int num;

    printf("请输入一个整数: ");
    scanf("%d", &num);

    if (num % 2 == 0) {
        printf("%d是偶数\n", num);
    } else {
        printf("%d是奇数\n", num);
    }

    return 0;
}
```

练习2: 判断闰年

```
#include <stdio.h>

int main() {
    int year;

    printf("请输入年份: ");
    scanf("%d", &year);

    if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)) {
        printf("%d年是闰年\n", year);
    } else {
        printf("%d年不是闰年\n", year);
    }

    return 0;
}
```

```
}
```

练习3：简单计算器

```
#include <stdio.h>

int main() {
    float num1, num2;
    char op;

    printf("请输入表达式 (如: 3 + 5) : ");
    scanf("%f %c %f", &num1, &op, &num2);

    switch (op) {
        case '+':
            printf("%.2f + %.2f = %.2f\n", num1, num2, num1 + num2);
            break;
        case '-':
            printf("%.2f - %.2f = %.2f\n", num1, num2, num1 - num2);
            break;
        case '*':
            printf("%.2f * %.2f = %.2f\n", num1, num2, num1 * num2);
            break;
        case '/':
            if (num2 != 0) {
                printf("%.2f / %.2f = %.2f\n", num1, num2, num1 / num2);
            } else {
                printf("错误: 除数不能为0\n");
            }
            break;
        default:
            printf("错误: 未知运算符\n");
    }

    return 0;
}
```

练习4：学生成绩管理

```
#include <stdio.h>

int main() {
    int choice;

    while (1) {
        printf("\n===== 学生成绩管理系统 =====\n");
        printf("1. 添加成绩\n");
        printf("2. 查询成绩\n");
        printf("3. 修改成绩\n");
        printf("4. 删除成绩\n");
        printf("5. 退出系统\n");

        printf("请输入选择(1-5): ");
    }
}
```

```
scanf("%d", &choice);

switch (choice) {
    case 1:
        printf("执行添加成绩功能\n");
        break;
    case 2:
        printf("执行查询成绩功能\n");
        break;
    case 3:
        printf("执行修改成绩功能\n");
        break;
    case 4:
        printf("执行删除成绩功能\n");
        break;
    case 5:
        printf("感谢使用，再见！\n");
        return 0;
    default:
        printf("输入错误，请重新输入\n");
}

return 0;
}
```

六、总结与延伸

本节重点回顾

1. **流程控制结构**：顺序结构、选择结构、循环结构
2. **if语句**：基本if、if-else、嵌套if-else
3. **switch语句**：多分支选择，使用break跳出，支持case穿透
4. **三元运算符**：简化的条件判断，语法：`条件 ? 表达式1 : 表达式2`

下节预告

下一篇文章我们将学习《循环结构：重复执行的艺术》，深入理解for、while、do-while等循环语句的使用方法。

📌 知识点卡片

- if语句用于条件判断，else处理相反情况
- switch语句适合多分支选择，case必须是常量表达式
- break语句用于跳出switch或循环
- default语句处理未匹配的情况
- 三元运算符可以简化简单的if-else判断

💡 小技巧

- 当条件较多时，switch语句比if-else更高效
- 使用缩进提高代码可读性

- 在复杂条件中使用括号明确优先级
- default语句可以省略，但建议保留以处理意外情况

⚠️ 注意事项

- if后面不要误加分号
- switch的表达式必须是整型或字符型
- 不要忘记在case末尾添加break，否则会发生case穿透
- 三元运算符不要嵌套过深，影响可读性

如果你对流程控制有任何疑问，欢迎在评论区留言讨论！下一篇见！

07 循环结构：重复执行的艺术

让程序学会重复

在上一篇文章中，我们学习了流程控制中的选择结构。现在，让我们来学习另一种重要的流程控制结构——循环结构。循环结构让程序能够重复执行一段代码，这是实现复杂逻辑的基础。

一、循环的基本概念

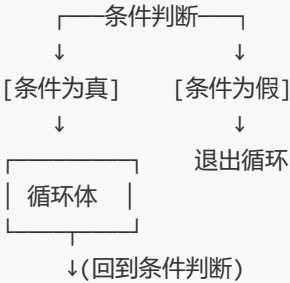
1.1 什么是循环

循环是指程序根据一定的条件，重复执行一段代码的结构。C语言提供了三种循环语句：

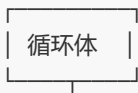
循环类型	关键字	特点
while循环	<code>while</code>	先判断条件，再执行循环体
do-while循环	<code>do-while</code>	先执行循环体，再判断条件
for循环	<code>for</code>	将初始化、条件、更新放在一起

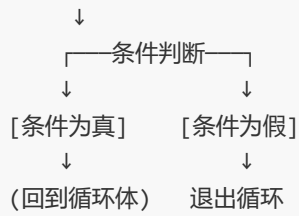
1.2 循环的流程图

while循环流程：



do-while循环流程：





二、while循环

2.1 基本语法

```
while (条件表达式) {  
    // 循环体  
}
```

2.2 示例：输出1到10

```
#include <stdio.h>  
  
int main() {  
    int i = 1;  
  
    while (i <= 10) {  
        printf("%d ", i);  
        i++; // 避免无限循环  
    }  
    printf("\n");  
  
    return 0;  
}
```

输出结果：

```
1 2 3 4 5 6 7 8 9 10
```

2.3 示例：计算1到100的和

```
#include <stdio.h>  
  
int main() {  
    int i = 1;  
    int sum = 0;  
  
    while (i <= 100) {  
        sum += i; // sum = sum + i  
        i++;  
    }  
  
    printf("1到100的和: %d\n", sum);  
}
```

```
    return 0;
}
```

输出结果:

1到100的和: 5050

三、do-while循环

3.1 基本语法

```
do {
    // 循环体
} while (条件表达式);
```

3.2 示例：至少执行一次

```
#include <stdio.h>

int main() {
    int num;

    do {
        printf("请输入一个正数: ");
        scanf("%d", &num);
    } while (num <= 0);

    printf("你输入的正数是: %d\n", num);

    return 0;
}
```

3.3 while vs do-while

特性	while	do-while
执行顺序	先判断后执行	先执行后判断
循环体执行次数	0次或多次	至少1次
适用场景	不确定是否需要执行	至少需要执行一次

四、for循环

4.1 基本语法


```
for (初始化表达式; 条件表达式; 更新表达式) {  
    // 循环体  
}
```

4.2 示例：输出1到10

```
#include <stdio.h>  
  
int main() {  
    for (int i = 1; i <= 10; i++) {  
        printf("%d ", i);  
    }  
    printf("\n");  
  
    return 0;  
}
```

4.3 示例：计算阶乘

```
#include <stdio.h>  
  
int main() {  
    int n;  
    printf("请输入一个正整数: ");  
    scanf("%d", &n);  
  
    int factorial = 1;  
    for (int i = 1; i <= n; i++) {  
        factorial *= i;  
    }  
  
    printf("%d的阶乘是: %d\n", n, factorial);  
  
    return 0;  
}
```

4.4 for循环的灵活性

```
#include <stdio.h>  
  
int main() {  
    // 多个变量初始化  
    for (int i = 0, j = 10; i <= j; i++, j--) {  
        printf("i=%d, j=%d\n", i, j);  
    }  
  
    return 0;  
}
```

输出结果：

```
i=0, j=10
i=1, j=9
i=2, j=8
i=3, j=7
i=4, j=6
i=5, j=5
```

五、循环控制语句

5.1 break语句

`break` 用于跳出当前循环：

```
#include <stdio.h>

int main() {
    for (int i = 1; i <= 10; i++) {
        if (i == 5) {
            break; // 跳出循环
        }
        printf("%d ", i);
    }
    printf("\n");

    return 0;
}
```

输出结果：

```
1 2 3 4
```

5.2 continue语句

`continue` 用于跳过当前迭代，继续下一次循环：

```
#include <stdio.h>

int main() {
    for (int i = 1; i <= 10; i++) {
        if (i % 2 == 0) {
            continue; // 跳过偶数
        }
        printf("%d ", i);
    }
    printf("\n");

    return 0;
}
```

输出结果：

```
1 3 5 7 9
```

5.3 goto语句

`goto` 可以跳转到指定标签：

```
#include <stdio.h>

int main() {
    int i = 1;

start:
    if (i <= 5) {
        printf("%d ", i);
        i++;
        goto start; // 跳转到start标签
    }
    printf("\n");

    return 0;
}
```

注意： `goto` 语句容易导致代码混乱，建议谨慎使用。

六、嵌套循环

6.1 示例：打印九九乘法表

```
#include <stdio.h>

int main() {
    for (int i = 1; i <= 9; i++) {
        for (int j = 1; j <= i; j++) {
            printf("%d×%d=%d\t", j, i, i * j);
        }
        printf("\n");
    }

    return 0;
}
```

输出结果：

```
1×1=1
1×2=2   2×2=4
1×3=3   2×3=6   3×3=9
1×4=4   2×4=8   3×4=12   4×4=16
1×5=5   2×5=10   3×5=15   4×5=20   5×5=25
1×6=6   2×6=12   3×6=18   4×6=24   5×6=30   6×6=36
1×7=7   2×7=14   3×7=21   4×7=28   5×7=35   6×7=42   7×7=49
1×8=8   2×8=16   3×8=24   4×8=32   5×8=40   6×8=48   7×8=56   8×8=64
1×9=9   2×9=18   3×9=27   4×9=36   5×9=45   6×9=54   7×9=63   8×9=72   9×9=81
```

6.2 示例：打印金字塔

```
#include <stdio.h>

int main() {
    int n = 5;

    for (int i = 1; i <= n; i++) {
        // 打印空格
        for (int j = n - i; j > 0; j--) {
            printf(" ");
        }
        // 打印星号
        for (int k = 1; k <= 2 * i - 1; k++) {
            printf("*");
        }
        printf("\n");
    }

    return 0;
}
```

输出结果：

```
  *
 ***
*****
*****
*****
```

七、实战练习

练习1：输出斐波那契数列

```
#include <stdio.h>

int main() {
    int n;
    printf("请输入斐波那契数列的项数：");
    scanf("%d", &n);
```

```

int a = 0, b = 1;

for (int i = 1; i <= n; i++) {
    printf("%d ", a);
    int temp = a + b;
    a = b;
    b = temp;
}
printf("\n");

return 0;
}

```

练习2: 判断质数

```

#include <stdio.h>

int main() {
    int num;
    printf("请输入一个正整数: ");
    scanf("%d", &num);

    if (num <= 1) {
        printf("%d不是质数\n", num);
        return 0;
    }

    int is_prime = 1;
    for (int i = 2; i * i <= num; i++) {
        if (num % i == 0) {
            is_prime = 0;
            break;
        }
    }

    if (is_prime) {
        printf("%d是质数\n", num);
    } else {
        printf("%d不是质数\n", num);
    }

    return 0;
}

```

练习3: 求最大公约数

```

#include <stdio.h>

int main() {
    int a, b;

    printf("请输入两个正整数: ");
}

```

```
scanf("%d %d", &a, &b);

// 使用辗转相除法
while (b != 0) {
    int temp = b;
    b = a % b;
    a = temp;
}

printf("最大公约数是: %d\n", a);

return 0;
}
```

练习4: 打印菱形

```
#include <stdio.h>

int main() {
    int n = 5;

    // 上半部分
    for (int i = 1; i <= n; i++) {
        for (int j = n - i; j > 0; j--) {
            printf(" ");
        }
        for (int k = 1; k <= 2 * i - 1; k++) {
            printf("*");
        }
        printf("\n");
    }

    // 下半部分
    for (int i = n - 1; i >= 1; i--) {
        for (int j = n - i; j > 0; j--) {
            printf(" ");
        }
        for (int k = 1; k <= 2 * i - 1; k++) {
            printf("*");
        }
        printf("\n");
    }

    return 0;
}
```

八、总结与延伸

本节重点回顾

1. **while**循环: 先判断后执行, 适合不确定循环次数的情况
2. **do-while**循环: 先执行后判断, 至少执行一次

3. **for循环**：将初始化、条件、更新放在一起，适合确定循环次数的情况
4. **break语句**：跳出当前循环
5. **continue语句**：跳过当前迭代，继续下一次循环
6. **嵌套循环**：循环中包含循环，用于处理二维数据

下节预告

下一篇文章我们将学习《数组与字符串：批量数据处理》，深入理解数组和字符串的使用方法。

📌 知识点卡片

- while循环： `while(条件) { 循环体 }`
- do-while循环： `do { 循环体 } while(条件);`
- for循环： `for(初始化; 条件; 更新) { 循环体 }`
- break：跳出当前循环
- continue：跳过当前迭代
- 嵌套循环：循环中可以嵌套另一个循环

💡 小技巧

- 选择循环类型时，确定次数用for，不确定用while，至少执行一次用do-while
- 使用循环时一定要确保有退出条件，避免无限循环
- 嵌套循环中，外层循环控制行数，内层循环控制列数
- 对于复杂的循环逻辑，可以使用辅助变量提高可读性

⚠️ 注意事项

- for循环中的初始化表达式只执行一次
- do-while循环末尾必须有分号
- 避免在循环中修改循环变量以外的变量，容易导致逻辑混乱
- goto语句容易破坏代码结构，尽量使用结构化控制语句

如果你对循环结构有任何疑问，欢迎在评论区留言讨论！下一篇见！

08 数组与字符串：批量数据处理

管理一组数据

在上一篇文章中，我们学习了循环结构的使用。现在，让我们来学习数组和字符串。数组是一种用于存储多个相同类型数据的容器，而字符串是字符数组的一种特殊形式。它们是处理批量数据的基础工具。

一、数组的概念

1.1 什么是数组

数组是一组具有相同数据类型的元素的集合，这些元素在内存中连续存储。

```
#include <stdio.h>
```

```

int main() {
    // 定义一个包含5个整数的数组
    int scores[5];

    // 初始化数组元素
    scores[0] = 85;
    scores[1] = 90;
    scores[2] = 78;
    scores[3] = 92;
    scores[4] = 88;

    // 访问数组元素
    printf("第一个学生的成绩: %d\n", scores[0]);
    printf("第三个学生的成绩: %d\n", scores[2]);

    return 0;
}

```

1.2 数组的声明与初始化

```

#include <stdio.h>

int main() {
    // 方式1: 先声明后初始化
    int arr1[5];
    arr1[0] = 1;
    arr1[1] = 2;
    arr1[2] = 3;
    arr1[3] = 4;
    arr1[4] = 5;

    // 方式2: 声明时初始化
    int arr2[5] = {1, 2, 3, 4, 5};

    // 方式3: 省略数组大小 (由初始化列表决定)
    int arr3[] = {1, 2, 3, 4, 5};

    // 方式4: 部分初始化 (未初始化的元素默认为0)
    int arr4[5] = {1, 2, 3}; // arr4[3] = 0, arr4[4] = 0

    // 方式5: 全部初始化为0
    int arr5[5] = {0};

    return 0;
}

```

1.3 数组的内存布局

数组元素在内存中是连续存储的:


```
#include <stdio.h>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};

    for (int i = 0; i < 5; i++) {
        printf("arr[%d] = %d, 地址 = %p\n", i, arr[i], &arr[i]);
    }

    return 0;
}
```

输出结果示例：

```
arr[0] = 10, 地址 = 0x7ffeefbfff540
arr[1] = 20, 地址 = 0x7ffeefbfff544
arr[2] = 30, 地址 = 0x7ffeefbfff548
arr[3] = 40, 地址 = 0x7ffeefbfff54c
arr[4] = 50, 地址 = 0x7ffeefbfff550
```

注意：每个int元素占用4字节，所以相邻元素地址相差4。

二、数组的基本操作

2.1 数组的遍历

```
#include <stdio.h>

int main() {
    int arr[] = {10, 20, 30, 40, 50};
    int len = sizeof(arr) / sizeof(arr[0]); // 计算数组长度

    // 使用for循环遍历
    printf("数组元素: ");
    for (int i = 0; i < len; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    // 使用while循环遍历
    printf("数组元素: ");
    int j = 0;
    while (j < len) {
        printf("%d ", arr[j]);
        j++;
    }
    printf("\n");

    return 0;
}
```

2.2 数组的常见操作

```
#include <stdio.h>

int main() {
    int arr[] = {5, 2, 8, 1, 9, 3};
    int len = sizeof(arr) / sizeof(arr[0]);

    // 1. 求最大值
    int max = arr[0];
    for (int i = 1; i < len; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    printf("最大值: %d\n", max);

    // 2. 求最小值
    int min = arr[0];
    for (int i = 1; i < len; i++) {
        if (arr[i] < min) {
            min = arr[i];
        }
    }
    printf("最小值: %d\n", min);

    // 3. 求和
    int sum = 0;
    for (int i = 0; i < len; i++) {
        sum += arr[i];
    }
    printf("总和: %d\n", sum);

    // 4. 求平均值
    float avg = (float)sum / len;
    printf("平均值: %.2f\n", avg);

    return 0;
}
```

2.3 数组作为函数参数

```
#include <stdio.h>

// 数组作为函数参数
void printArray(int arr[], int len) {
    for (int i = 0; i < len; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

// 修改数组元素
```

```

void doubleArray(int arr[], int len) {
    for (int i = 0; i < len; i++) {
        arr[i] *= 2;
    }
}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int len = sizeof(arr) / sizeof(arr[0]);

    printf("原数组: ");
    printArray(arr, len);

    doubleArray(arr, len);

    printf("翻倍后: ");
    printArray(arr, len);

    return 0;
}

```

输出结果:

```

原数组: 1 2 3 4 5
翻倍后: 2 4 6 8 10

```

三、二维数组

3.1 二维数组的声明与初始化

```

#include <stdio.h>

int main() {
    // 声明一个3行4列的二维数组
    int matrix[3][4];

    // 初始化二维数组
    int matrix2[3][4] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };

    // 部分初始化
    int matrix3[3][4] = {
        {1, 2},
        {3},
        {4, 5, 6}
    };

    // 省略行号 (由初始化列表决定)
}

```

```

int matrix4[][4] = {
    {1, 2, 3, 4},
    {5, 6, 7, 8}
};

return 0;
}

```

3.2 二维数组的访问

```

#include <stdio.h>

int main() {
    int matrix[3][4] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };

    // 遍历二维数组
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 4; j++) {
            printf("%d\t", matrix[i][j]);
        }
        printf("\n");
    }

    return 0;
}

```

输出结果:

1	2	3	4
5	6	7	8
9	10	11	12

3.3 二维数组的内存布局

二维数组在内存中是按行优先顺序存储的:

```

#include <stdio.h>

int main() {
    int arr[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };

    printf("数组元素及其地址: \n");
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            printf("arr[%d][%d] = %d, 地址 = %p\n", i, j, arr[i][j], &arr[i][j]);
        }
    }
}

```

```
    }  
}  
  
return 0;  
}
```

四、字符串的概念

4.1 字符串的定义

在C语言中，字符串是一个以 `'\0'`（空字符）结尾的字符数组。

```
#include <stdio.h>  
  
int main() {  
    // 方式1: 字符数组形式  
    char str1[6] = {'H', 'e', 'l', 'l', 'o', '\0'};  
  
    // 方式2: 字符串字面量 (推荐)  
    char str2[] = "Hello";  
  
    // 方式3: 字符指针  
    char *str3 = "Hello";  
  
    printf("str1: %s\n", str1);  
    printf("str2: %s\n", str2);  
    printf("str3: %s\n", str3);  
  
    return 0;  
}
```

4.2 字符串的长度

```
#include <stdio.h>  
  
// 自定义字符串长度函数  
int my_strlen(const char *str) {  
    int len = 0;  
    while (str[len] != '\0') {  
        len++;  
    }  
    return len;  
}  
  
int main() {  
    char str[] = "Hello, World!";  
  
    // 使用自定义函数  
    printf("字符串长度: %d\n", my_strlen(str));  
  
    // 使用标准库函数
```

```
#include <string.h>
printf("字符串长度: %zu\n", strlen(str));

return 0;
}
```

五、字符串处理函数

5.1 常用字符串函数

```
#include <stdio.h>
#include <string.h>

int main() {
    char str1[20] = "Hello";
    char str2[20] = "World";
    char str3[40];

    // 1. strcpy: 字符串复制
    strcpy(str3, str1);
    printf("strcpy: %s\n", str3); // Hello

    // 2. strcat: 字符串拼接
    strcat(str1, " ");
    strcat(str1, str2);
    printf("strcat: %s\n", str1); // Hello World

    // 3. strcmp: 字符串比较
    int result = strcmp("Apple", "Banana");
    printf("strcmp: %d\n", result); // 负数 (Apple < Banana)

    // 4. strchr: 查找字符
    char *pos = strchr("Hello, World!", 'W');
    printf("strchr: %s\n", pos); // World!

    // 5. strstr: 查找子字符串
    char *sub = strstr("Hello, World!", "World");
    printf("strstr: %s\n", sub); // World!

    return 0;
}
```

5.2 字符串输入输出

```
#include <stdio.h>

int main() {
    char name[50];
    char address[100];

    // 1. 使用scanf输入 (遇到空格停止)
```

```
printf("请输入你的姓名: ");
scanf("%s", name);
printf("你好, %s! \n", name);

// 2. 使用gets输入 (读取整行, 包括空格)
printf("请输入你的地址: ");
// gets(address); // 注意: gets已废弃, 不安全

// 3. 使用fgets输入 (推荐)
printf("请输入你的地址: ");
fgets(address, sizeof(address), stdin);
printf("你的地址: %s", address);

return 0;
}
```

六、实战练习

练习1: 数组排序 (冒泡排序)

```
#include <stdio.h>

void bubbleSort(int arr[], int len) {
    for (int i = 0; i < len - 1; i++) {
        for (int j = 0; j < len - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                // 交换
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int len = sizeof(arr) / sizeof(arr[0]);

    printf("排序前: ");
    for (int i = 0; i < len; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    bubbleSort(arr, len);

    printf("排序后: ");
    for (int i = 0; i < len; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
```

```
    return 0;
}
```

练习2: 字符串反转

```
#include <stdio.h>
#include <string.h>

void reverseString(char str[]) {
    int len = strlen(str);
    int start = 0;
    int end = len - 1;

    while (start < end) {
        // 交换字符
        char temp = str[start];
        str[start] = str[end];
        str[end] = temp;

        start++;
        end--;
    }
}

int main() {
    char str[] = "Hello, World!";

    printf("原字符串: %s\n", str);

    reverseString(str);

    printf("反转后: %s\n", str);

    return 0;
}
```

练习3: 二维数组转置

```
#include <stdio.h>

#define ROWS 3
#define COLS 4

void transpose(int matrix[ROWS][COLS], int result[COLS][ROWS]) {
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            result[j][i] = matrix[i][j];
        }
    }
}
```



```

int main() {
    int matrix[ROWS][COLS] = {
        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };
    int result[COLS][ROWS];

    printf("原矩阵: \n");
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            printf("%d\t", matrix[i][j]);
        }
        printf("\n");
    }

    transpose(matrix, result);

    printf("\n转置后: \n");
    for (int i = 0; i < COLS; i++) {
        for (int j = 0; j < ROWS; j++) {
            printf("%d\t", result[i][j]);
        }
        printf("\n");
    }

    return 0;
}

```

练习4: 统计字符串中字符出现次数

```

#include <stdio.h>
#include <string.h>

int main() {
    char str[100];
    char target;
    int count = 0;

    printf("请输入一个字符串: ");
    fgets(str, sizeof(str), stdin);

    printf("请输入要统计的字符: ");
    scanf("%c", &target);

    // 统计字符出现次数
    for (int i = 0; str[i] != '\0'; i++) {
        if (str[i] == target) {
            count++;
        }
    }

    printf("字符 '%c' 在字符串中出现了%d次\n", target, count);
}

```

```
    return 0;
}
```

七、总结与延伸

本节重点回顾

1. **数组**：相同类型元素的集合，连续存储
2. **数组声明**：`type name[size]`
3. **数组初始化**：`int arr[] = {1, 2, 3};`
4. **数组遍历**：使用for循环，通过索引访问
5. **二维数组**：`type name[rows][cols]`
6. **字符串**：以 `'\0'` 结尾的字符数组
7. **字符串函数**：`strlen`，`strcpy`，`strcat`，`strcmp` 等

下节预告

下一篇文章我们将学习《函数：代码复用的利器》，深入理解函数的定义、调用和参数传递。

📌 知识点卡片

- 数组：`type name[size]`，元素连续存储
- 数组长度：`sizeof(arr) / sizeof(arr[0])`
- 二维数组：`type name[rows][cols]`
- 字符串：以 `'\0'` 结尾的字符数组
- 常用字符串函数：`strlen`，`strcpy`，`strcat`，`strcmp`
- 数组作为函数参数时会退化为指针

💡 小技巧

- 数组索引从0开始
- 初始化数组时可以省略大小，由初始化列表决定
- 使用 `sizeof(arr) / sizeof(arr[0])` 计算数组长度
- 字符串输入推荐使用 `fgets`，避免 `gets`（不安全）
- 字符串比较必须使用 `strcmp`，不能直接用 `==`

⚠️ 注意事项

- 不要越界访问数组元素
- 字符串必须以 `'\0'` 结尾
- 数组作为函数参数时，需要额外传递长度信息
- `strcpy` 和 `strcat` 可能导致缓冲区溢出，注意目标数组大小

如果你对数组和字符串有任何疑问，欢迎在评论区留言讨论！下一篇见！

09 函数：代码复用的利器

组织代码的基本单元

在上一篇文章中，我们学习了数组和字符串的使用。现在，让我们来学习函数。函数是C语言中组织代码的基本单元，它允许我们将一段代码封装起来，通过名字调用执行，实现代码复用和模块化编程。

一、函数的概念

1.1 什么是函数

函数是一段完成特定功能的代码块，可以通过函数名调用执行。

```
#include <stdio.h>

// 定义一个计算平方的函数
int square(int num) {
    return num * num;
}

int main() {
    int result = square(5); // 调用函数
    printf("5的平方是: %d\n", result);

    return 0;
}
```

1.2 函数的组成部分

```
返回类型 函数名(参数列表) {
    // 函数体
    return 返回值;
}
```

组成部分	说明	示例
返回类型	函数返回值的类型	int, float, void
函数名	函数的名称，用于调用	square, add, printArray
参数列表	传递给函数的值	(int num), (int a, int b)
函数体	函数执行的代码	变量声明、语句、控制结构
返回值	通过 return 语句返回	return result;

二、函数的定义与调用

2.1 无参数无返回值函数

```

#include <stdio.h>

// 无参数无返回值
void sayHello() {
    printf("Hello, World!\n");
}

int main() {
    sayHello(); // 调用函数
    sayHello(); // 再次调用

    return 0;
}

```

2.2 有参数无返回值函数

```

#include <stdio.h>

// 有参数无返回值
void printMessage(char *msg) {
    printf("消息: %s\n", msg);
}

void printNumbers(int start, int end) {
    for (int i = start; i <= end; i++) {
        printf("%d ", i);
    }
    printf("\n");
}

int main() {
    printMessage("这是一个测试消息");
    printNumbers(1, 10);

    return 0;
}

```

2.3 有参数有返回值函数

```

#include <stdio.h>

// 计算两个数的和
int add(int a, int b) {
    return a + b;
}

// 计算两个数的最大值
int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {

```

```
int sum = add(5, 3);
printf("5 + 3 = %d\n", sum);

int maximum = max(10, 15);
printf("最大值: %d\n", maximum);

return 0;
}
```

三、函数参数传递

3.1 值传递

```
#include <stdio.h>

void increment(int x) {
    x++;
    printf("函数内: x = %d\n", x);
}

int main() {
    int num = 5;
    printf("调用前: num = %d\n", num);

    increment(num);

    printf("调用后: num = %d\n", num); // num仍然是5

    return 0;
}
```

输出结果:

```
调用前: num = 5
函数内: x = 6
调用后: num = 5
```

3.2 地址传递 (指针传递)

```
#include <stdio.h>

void increment(int *x) {
    (*x)++;
    printf("函数内: *x = %d\n", *x);
}

int main() {
    int num = 5;
    printf("调用前: num = %d\n", num);

    increment(&num); // 传递地址
}
```

```
    printf("调用后: num = %d\n", num); // num变成6

    return 0;
}
```

输出结果:

```
调用前: num = 5
函数内: *x = 6
调用后: num = 6
```

3.3 数组作为参数

```
#include <stdio.h>

void printArray(int arr[], int len) {
    for (int i = 0; i < len; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

void modifyArray(int arr[], int len) {
    for (int i = 0; i < len; i++) {
        arr[i] *= 2;
    }
}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int len = sizeof(arr) / sizeof(arr[0]);

    printf("原数组: ");
    printArray(arr, len);

    modifyArray(arr, len);

    printf("修改后: ");
    printArray(arr, len);

    return 0;
}
```

四、函数原型

4.1 什么是函数原型

函数原型是函数的声明，告诉编译器函数的名称、返回类型和参数类型。

```
#include <stdio.h>
```

```
// 函数原型
int add(int a, int b);
void printHello();

int main() {
    int result = add(3, 5);
    printf("结果: %d\n", result);

    printHello();

    return 0;
}

// 函数定义
int add(int a, int b) {
    return a + b;
}

void printHello() {
    printf("Hello!\n");
}
```

4.2 函数原型的作用

1. **类型检查**: 编译器可以检查参数类型和数量是否正确
2. **代码组织**: 可以将函数定义放在文件末尾, 提高代码可读性
3. **头文件声明**: 在头文件中声明函数, 供其他文件使用

五、递归函数

5.1 什么是递归

递归是指函数调用自身的编程技巧。

```
#include <stdio.h>

// 计算阶乘的递归函数
int factorial(int n) {
    if (n == 0 || n == 1) {
        return 1; // 递归终止条件
    }
    return n * factorial(n - 1); // 递归调用
}

int main() {
    int num = 5;
    printf("%d的阶乘是: %d\n", num, factorial(num));

    return 0;
}
```

执行过程：

```
factorial(5) = 5 * factorial(4)
factorial(4) = 4 * factorial(3)
factorial(3) = 3 * factorial(2)
factorial(2) = 2 * factorial(1)
factorial(1) = 1
```

5.2 递归的条件

1. **递归终止条件**：必须有一个明确的终止条件
2. **递归调用**：函数必须调用自身
3. **收敛性**：每次递归调用都应向终止条件靠近

5.3 递归示例：斐波那契数列

```
#include <stdio.h>

int fibonacci(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    printf("斐波那契数列前10项: ");
    for (int i = 0; i < 10; i++) {
        printf("%d ", fibonacci(i));
    }
    printf("\n");

    return 0;
}
```

六、函数的作用域

6.1 局部变量与全局变量

```
#include <stdio.h>

int global_var = 100; // 全局变量

void test() {
    int local_var = 200; // 局部变量
    printf("全局变量: %d\n", global_var);
    printf("局部变量: %d\n", local_var);
}

int main() {
    printf("全局变量: %d\n", global_var);

    // printf("局部变量: %d\n", local_var); // 错误! 局部变量不可访问
}
```



```
    test();

    return 0;
}
```

6.2 静态变量

```
#include <stdio.h>

void counter() {
    static int count = 0; // 静态变量, 只初始化一次
    count++;
    printf("count = %d\n", count);
}

int main() {
    counter(); // count = 1
    counter(); // count = 2
    counter(); // count = 3

    return 0;
}
```

七、实战练习

练习1: 计算数组的最大值

```
#include <stdio.h>

int findMax(int arr[], int len) {
    int max = arr[0];
    for (int i = 1; i < len; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    return max;
}

int main() {
    int arr[] = {5, 2, 8, 1, 9, 3};
    int len = sizeof(arr) / sizeof(arr[0]);

    printf("数组的最大值: %d\n", findMax(arr, len));

    return 0;
}
```

练习2: 字符串长度

```

#include <stdio.h>

int my_strlen(const char *str) {
    int len = 0;
    while (str[len] != '\0') {
        len++;
    }
    return len;
}

int main() {
    char str[] = "Hello, World!";
    printf("字符串长度: %d\n", my_strlen(str));

    return 0;
}

```

练习3: 递归计算幂

```

#include <stdio.h>

int power(int base, int exp) {
    if (exp == 0) {
        return 1;
    }
    return base * power(base, exp - 1);
}

int main() {
    printf("2^3 = %d\n", power(2, 3));
    printf("5^4 = %d\n", power(5, 4));

    return 0;
}

```

练习4: 二分查找

```

#include <stdio.h>

int binarySearch(int arr[], int len, int target) {
    int left = 0;
    int right = len - 1;

    while (left <= right) {
        int mid = (left + right) / 2;

        if (arr[mid] == target) {
            return mid; // 找到目标, 返回索引
        } else if (arr[mid] < target) {
            left = mid + 1; // 在右半部分查找
        } else {
            right = mid - 1; // 在左半部分查找
        }
    }
    return -1;
}

```

```
    }  
}  
  
return -1; // 未找到  
}  
  
int main() {  
    int arr[] = {1, 3, 5, 7, 9, 11, 13};  
    int len = sizeof(arr) / sizeof(arr[0]);  
  
    int target = 7;  
    int index = binarySearch(arr, len, target);  
  
    if (index != -1) {  
        printf("找到%d, 索引为%d\n", target, index);  
    } else {  
        printf("未找到%d\n", target);  
    }  
  
    return 0;  
}
```

八、总结与延伸

本节重点回顾

1. **函数定义**：返回类型 函数名(参数列表) { 函数体 }
2. **函数调用**：通过函数名传递参数调用
3. **参数传递**：值传递和地址传递
4. **函数原型**：声明函数接口，用于类型检查
5. **递归函数**：函数调用自身，需要终止条件
6. **变量作用域**：局部变量、全局变量、静态变量

下节预告

下一篇文章我们将学习《指针：C语言的灵魂》，深入理解指针的概念和使用方法。

📌 知识点卡片

- 函数定义：返回类型 函数名(参数列表) { 函数体 }
- 值传递：函数对参数的修改不影响实参
- 地址传递：函数对参数的修改会影响实参
- 函数原型：返回类型 函数名(参数类型列表);
- 递归：函数调用自身，需要终止条件
- 静态变量：只初始化一次，值在函数调用间保持

💡 小技巧

- 将常用功能封装成函数，提高代码复用性
- 使用函数原型可以将函数定义放在文件末尾
- 递归函数必须有明确的终止条件

- 数组作为参数时会退化为指针，需要额外传递长度

⚠ 注意事项

- 函数参数的类型和数量必须与调用时匹配
- 递归深度过大可能导致栈溢出
- 全局变量会增加程序耦合度，应谨慎使用
- 静态变量的生命周期是整个程序运行期间

如果你对函数有任何疑问，欢迎在评论区留言讨论！下一篇见！

10 指针：C语言的灵魂

深入内存的世界

在上一篇文章中，我们学习了函数的使用。现在，让我们来学习C语言中最核心、最强大的概念——指针。指针是C语言的灵魂，它让我们能够直接操作内存地址，实现高效的数据访问和内存管理。

一、指针的概念

1.1 什么是指针

指针是一个变量，它存储的是另一个变量的内存地址。

```
#include <stdio.h>

int main() {
    int num = 42;
    int *ptr = &num; // ptr是指向num的指针

    printf("变量num的值: %d\n", num);
    printf("变量num的地址: %p\n", &num);
    printf("指针ptr的值: %p\n", ptr);
    printf("指针ptr指向的值: %d\n", *ptr);

    return 0;
}
```

输出结果：

```
变量num的值: 42
变量num的地址: 0x7ffeefbff54c
指针ptr的值: 0x7ffeefbff54c
指针ptr指向的值: 42
```

1.2 指针的声明

```
// 基本语法
类型 *指针变量名;

// 示例
int *p;      // 指向int的指针
float *f;    // 指向float的指针
char *c;     // 指向char的指针
int **pp;    // 指向指针的指针 (二级指针)
```

1.3 指针运算符

运算符	名称	作用
&	取地址运算符	获取变量的内存地址
*	解引用运算符	获取指针指向的值

```
#include <stdio.h>

int main() {
    int a = 10;
    int *p = &a;

    printf("a = %d\n", a);      // 直接访问变量
    printf("&a = %p\n", &a);    // 获取变量地址
    printf("p = %p\n", p);     // 指针存储的地址
    printf("*p = %d\n", *p);   // 通过指针访问变量

    return 0;
}
```

二、指针与数组

2.1 数组名与指针

数组名本质上是指向数组第一个元素的指针。

```
#include <stdio.h>

int main() {
    int arr[] = {10, 20, 30, 40, 50};

    printf("arr = %p\n", arr);
    printf("&arr[0] = %p\n", &arr[0]);

    printf("arr[0] = %d\n", arr[0]);
    printf("*arr = %d\n", *arr);

    printf("arr[1] = %d\n", arr[1]);
}
```

```
    printf("*(arr + 1) = %d\n", *(arr + 1));

    return 0;
}
```

2.2 使用指针遍历数组

```
#include <stdio.h>

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int len = sizeof(arr) / sizeof(arr[0]);

    // 使用数组下标
    printf("使用下标遍历: ");
    for (int i = 0; i < len; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    // 使用指针
    printf("使用指针遍历: ");
    int *p = arr;
    for (int i = 0; i < len; i++) {
        printf("%d ", *(p + i));
    }
    printf("\n");

    // 使用指针移动
    printf("使用指针移动遍历: ");
    for (int *p = arr; p < arr + len; p++) {
        printf("%d ", *p);
    }
    printf("\n");

    return 0;
}
```

三、指针与函数

3.1 指针作为函数参数

```
#include <stdio.h>

// 使用指针交换两个整数
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
```

```
int main() {
    int x = 5, y = 10;

    printf("交换前: x = %d, y = %d\n", x, y);

    swap(&x, &y);

    printf("交换后: x = %d, y = %d\n", x, y);

    return 0;
}
```

3.2 指针作为函数返回值

```
#include <stdio.h>

int *createArray(int size) {
    // 使用动态内存分配
    int *arr = (int *)malloc(size * sizeof(int));

    for (int i = 0; i < size; i++) {
        arr[i] = i + 1;
    }

    return arr;
}

int main() {
    int *arr = createArray(5);

    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    free(arr); // 释放动态分配的内存

    return 0;
}
```

四、指针的高级用法

4.1 指针数组

```
#include <stdio.h>

int main() {
    char *names[] = {"张三", "李四", "王五", "赵六"};

    for (int i = 0; i < 4; i++) {
        printf("names[%d] = %s\n", i, names[i]);
    }

    return 0;
}
```

4.2 指向指针的指针（二级指针）

```
#include <stdio.h>

int main() {
    int num = 100;
    int *p = &num;      // 一级指针
    int **pp = &p;       // 二级指针

    printf("num = %d\n", num);
    printf("*p = %d\n", *p);
    printf("**pp = %d\n", **pp);

    **pp = 200; // 通过二级指针修改原始变量
    printf("修改后 num = %d\n", num);

    return 0;
}
```

4.3 指针与字符串

```
#include <stdio.h>

int main() {
    char str[] = "Hello";
    char *p = str;

    // 遍历字符串
    while (*p != '\0') {
        printf("%c ", *p);
        p++;
    }
    printf("\n");

    return 0;
}
```

五、指针与内存管理

5.1 动态内存分配

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    // 分配单个整数
    int *p1 = (int *)malloc(sizeof(int));
    *p1 = 42;
    printf("p1 = %d\n", *p1);

    // 分配数组
    int *arr = (int *)malloc(5 * sizeof(int));
    for (int i = 0; i < 5; i++) {
        arr[i] = i * 10;
    }

    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    // 释放内存
    free(p1);
    free(arr);

    return 0;
}
```

5.2 calloc与realloc

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    // calloc: 分配并初始化为0
    int *arr1 = (int *)calloc(5, sizeof(int));
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr1[i]); // 输出: 0 0 0 0 0
    }
    printf("\n");

    // realloc: 重新分配内存
    int *arr2 = (int *)malloc(3 * sizeof(int));
    arr2[0] = 1; arr2[1] = 2; arr2[2] = 3;

    arr2 = (int *)realloc(arr2, 5 * sizeof(int));
    arr2[3] = 4; arr2[4] = 5;

    for (int i = 0; i < 5; i++) {
        printf("%d ", arr2[i]); // 输出: 1 2 3 4 5
    }
    printf("\n");
}
```

```
    free(arr1);
    free(arr2);

    return 0;
}
```

六、指针的常见错误

6.1 空指针解引用

```
#include <stdio.h>

int main() {
    int *p = NULL;
    // *p = 10; // 错误! 空指针解引用会导致程序崩溃

    // 正确做法: 检查指针是否为空
    if (p != NULL) {
        *p = 10;
    }

    return 0;
}
```

6.2 野指针

```
#include <stdio.h>

int main() {
    int *p; // 未初始化的指针, 指向随机地址
    // *p = 10; // 错误! 野指针可能导致未知行为

    // 正确做法: 初始化指针
    int num = 0;
    p = &num;
    *p = 10;

    return 0;
}
```

6.3 内存泄漏

```
#include <stdio.h>
#include <stdlib.h>

void func() {
    int *p = (int *)malloc(sizeof(int));
    *p = 10;
    // 忘记释放内存!
}
```

```
int main() {
    func();
    // p指向的内存永远无法释放，造成内存泄漏

    return 0;
}
```

七、实战练习

练习1: 指针实现数组逆序

```
#include <stdio.h>

void reverseArray(int arr[], int len) {
    int *start = arr;
    int *end = arr + len - 1;

    while (start < end) {
        int temp = *start;
        *start = *end;
        *end = temp;

        start++;
        end--;
    }
}

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int len = sizeof(arr) / sizeof(arr[0]);

    printf("原数组: ");
    for (int i = 0; i < len; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    reverseArray(arr, len);

    printf("逆序后: ");
    for (int i = 0; i < len; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

练习2: 字符串复制函数

```

#include <stdio.h>

void my_strcpy(char *dest, const char *src) {
    while (*src != '\0') {
        *dest = *src;
        dest++;
        src++;
    }
    *dest = '\0'; // 添加字符串结束符
}

int main() {
    char src[] = "Hello, World!";
    char dest[20];

    my_strcpy(dest, src);
    printf("复制结果: %s\n", dest);

    return 0;
}

```

练习3: 动态二维数组

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    int rows = 3, cols = 4;

    // 分配二维数组
    int **matrix = (int **)malloc(rows * sizeof(int *));
    for (int i = 0; i < rows; i++) {
        matrix[i] = (int *)malloc(cols * sizeof(int));
    }

    // 填充数据
    int count = 1;
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            matrix[i][j] = count++;
        }
    }

    // 输出
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%d\t", matrix[i][j]);
        }
        printf("\n");
    }

    // 释放内存
}

```

```

    for (int i = 0; i < rows; i++) {
        free(matrix[i]);
    }
    free(matrix);

    return 0;
}

```

练习4: 指针实现链表

```

#include <stdio.h>
#include <stdlib.h>

// 链表节点结构
typedef struct Node {
    int data;
    struct Node *next;
} Node;

// 创建新节点
Node* createNode(int data) {
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

// 打印链表
void printList(Node *head) {
    Node *current = head;
    while (current != NULL) {
        printf("%d -> ", current->data);
        current = current->next;
    }
    printf("NULL\n");
}

int main() {
    // 创建链表
    Node *head = createNode(1);
    head->next = createNode(2);
    head->next->next = createNode(3);
    head->next->next->next = createNode(4);

    printf("链表内容: ");
    printList(head);

    return 0;
}

```

八、总结与延伸

本节重点回顾

1. **指针声明**: `类型 *指针变量名;`
2. **取地址运算符**: `&` 获取变量地址
3. **解引用运算符**: `*` 获取指针指向的值
4. **数组与指针**: 数组名是指向第一个元素的指针
5. **指针作为函数参数**: 实现引用传递
6. **动态内存分配**: `malloc`, `calloc`, `realloc`, `free`
7. **指针数组**: 存储多个指针的数组
8. **二级指针**: 指向指针的指针

下节预告

下一篇文章我们将学习《内存管理：堆与栈的奥秘》，深入理解内存的分配和管理机制。

📌 知识点卡片

- 指针: 存储另一个变量地址的变量
- 声明: `int *p;`
- 取地址: `&变量名`
- 解引用: `*指针名`
- 数组名: 指向第一个元素的指针
- 动态分配: `malloc`, `calloc`, `realloc`, `free`
- 空指针: `NULL`, 不指向任何有效地址

💡 小技巧

- 使用指针可以实现高效的数组遍历
- 指针作为函数参数可以修改实参
- 动态内存分配需要手动释放
- 二级指针常用于二维数组和链表操作

⚠️ 注意事项

- 不要解引用空指针
- 不要使用未初始化的野指针
- 动态分配的内存必须释放, 避免内存泄漏
- 指针运算时要注意边界检查

如果你对指针有任何疑问, 欢迎在评论区留言讨论! 下一篇见!

11 内存管理：堆与栈的奥秘

深入理解程序的内存布局

在上一篇文章中, 我们学习了指针的概念和使用。现在, 让我们深入探讨C语言中的内存管理机制, 特别是堆 (Heap) 和栈 (Stack) 这两种重要的内存区域。理解它们的区别和使用场景, 是成为优秀C程序员的关键。

一、程序的内存布局

1.1 内存区域划分

一个运行中的C程序在内存中通常分为以下几个区域：



1.2 内存区域详解

区域	存储内容	分配方式	生命周期
栈	局部变量、函数参数、返回地址	自动分配	函数调用开始时分配，结束时释放
堆	动态分配的数据	手动分配	从 <code>malloc</code> 到 <code>free</code>
数据段	已初始化全局变量、静态变量	编译时分配	整个程序运行期间
BSS段	未初始化全局变量、静态变量	编译时分配	整个程序运行期间
代码段	程序指令、常量	编译时分配	整个程序运行期间

二、栈的概念

2.1 什么是栈

栈是一种**后进先出 (LIFO)** 的数据结构，用于存储函数调用时的临时数据。

```
#include <stdio.h>

void func(int a, int b) {
    int x = a + b;    // 局部变量x在栈上
    printf("x = %d\n", x);
} // x自动从栈上释放

int main() {
    func(3, 5);
    return 0;
}
```

2.2 栈的特点

1. **自动管理**：栈上的变量在函数调用时自动分配，函数返回时自动释放
2. **大小有限**：栈的大小是固定的（通常几MB），超出会导致栈溢出
3. **效率高**：栈操作非常快，因为不需要复杂的内存管理
4. **后进先出**：最后入栈的数据最先出栈

2.3 栈溢出示例

```
#include <stdio.h>

void recursiveFunc(int n) {
    char buffer[1024]; // 每次递归分配1KB
    printf("递归深度: %d\n", n);
    recursiveFunc(n + 1); // 无限递归
}

int main() {
    recursiveFunc(1);
    return 0;
}
```

运行结果：程序会因为栈溢出而崩溃，因为每次递归都会在栈上分配内存，最终超出栈的容量。

三、堆的概念

3.1 什么是堆

堆是一块用于**动态内存分配**的内存区域，程序可以在运行时请求和释放内存。

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    // 在堆上分配一个整数
    int *p = (int *)malloc(sizeof(int));
    *p = 42;
    printf("p = %d\n", *p);
}
```



```
// 在堆上分配一个数组
int *arr = (int *)malloc(5 * sizeof(int));
for (int i = 0; i < 5; i++) {
    arr[i] = i * 10;
}

// 手动释放内存
free(p);
free(arr);

return 0;
}
```

3.2 堆的特点

- 1. **手动管理**：需要使用 malloc / calloc / realloc 分配， free 释放
- 2. **大小灵活**：堆的大小受限于系统可用内存
- 3. **效率较低**：堆操作需要查找可用内存块，比栈慢
- 4. **碎片化**：频繁分配和释放可能导致内存碎片化

四、堆与栈的对比

4.1 核心区别

特性	栈 (Stack)	堆 (Heap)
分配方式	自动分配	手动分配
释放方式	自动释放	手动释放
大小限制	较小 (几MB)	较大 (系统内存)
效率	高	较低
内存布局	连续、向下增长	不连续、向上增长
碎片化	不会碎片化	可能碎片化
生命周期	函数调用期间	从分配到释放

4.2 如何选择

```
#include <stdio.h>
#include <stdlib.h>

// 小数据、临时使用 → 使用栈
int calculateSum(int a, int b) {
    int result = a + b; // 栈上分配
    return result;
}
```

```
// 大数据、需要持久使用 → 使用堆
int* createArray(int size) {
    int *arr = (int *)malloc(size * sizeof(int)); // 堆上分配
    return arr;
}

int main() {
    int small = calculateSum(3, 5); // 栈变量

    int *large = createArray(10000); // 堆数组
    // 使用...
    free(large); // 必须手动释放

    return 0;
}
```

五、动态内存分配函数

5.1 malloc

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    // 分配单个元素
    int *p = (int *)malloc(sizeof(int));
    if (p == NULL) {
        printf("内存分配失败\n");
        return 1;
    }
    *p = 100;
    printf("*p = %d\n", *p);
    free(p);

    // 分配数组
    int *arr = (int *)malloc(5 * sizeof(int));
    if (arr == NULL) {
        printf("内存分配失败\n");
        return 1;
    }
    for (int i = 0; i < 5; i++) {
        arr[i] = i;
    }
    free(arr);

    return 0;
}
```

5.2 calloc

```
#include <stdio.h>
```

```
#include <stdlib.h>

int main() {
    // calloc会将内存初始化为0
    int *arr = (int *)calloc(5, sizeof(int));

    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]); // 输出: 0 0 0 0 0
    }
    printf("\n");

    free(arr);

    return 0;
}
```

5.3 realloc

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr = (int *)malloc(3 * sizeof(int));
    arr[0] = 1; arr[1] = 2; arr[2] = 3;

    // 扩展数组大小
    arr = (int *)realloc(arr, 5 * sizeof(int));
    arr[3] = 4; arr[4] = 5;

    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]); // 输出: 1 2 3 4 5
    }
    printf("\n");

    free(arr);

    return 0;
}
```

六、常见内存问题

6.1 内存泄漏

```
#include <stdio.h>
#include <stdlib.h>

void badFunc() {
    int *p = (int *)malloc(sizeof(int));
    *p = 42;
    // 忘记释放! 造成内存泄漏
}
```

```
int main() {
    for (int i = 0; i < 1000000; i++) {
        badFunc(); // 每次调用都会泄漏内存
    }
    return 0;
}
```

6.2 重复释放

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p = (int *)malloc(sizeof(int));
    free(p);
    // free(p); // 错误! 重复释放会导致未定义行为

    return 0;
}
```

6.3 使用已释放的内存

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p = (int *)malloc(sizeof(int));
    *p = 42;

    free(p);
    // printf("%d\n", *p); // 错误! 使用已释放的内存

    return 0;
}
```

6.4 缓冲区溢出

```
#include <stdio.h>
#include <string.h>

int main() {
    char buffer[10];
    // strcpy(buffer, "Hello, World!"); // 错误! 缓冲区溢出

    // 安全做法
    strncpy(buffer, "Hello, World!", sizeof(buffer) - 1);
    buffer[sizeof(buffer) - 1] = '\0';

    return 0;
}
```

七、内存管理最佳实践

7.1 始终检查malloc返回值

```
int *p = (int *)malloc(sizeof(int));
if (p == NULL) {
    printf("内存分配失败\n");
    return 1;
}
```

7.2 配对使用malloc和free

```
void func() {
    int *data = (int *)malloc(100 * sizeof(int));
    if (data == NULL) return;

    // 使用data...

    free(data); // 确保释放
}
```

7.3 使用智能指针模式（模拟）

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int *data;
    int size;
} Array;

Array* createArray(int size) {
    Array *arr = (Array *)malloc(sizeof(Array));
    arr->data = (int *)malloc(size * sizeof(int));
    arr->size = size;
    return arr;
}

void destroyArray(Array *arr) {
    free(arr->data);
    free(arr);
}

int main() {
    Array *arr = createArray(10);
    // 使用arr...
    destroyArray(arr); // 一次性释放所有资源

    return 0;
}
```

八、实战练习

练习1: 动态数组实现

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int *data;
    int size;
    int capacity;
} DynamicArray;

DynamicArray* createDynamicArray(int initialCapacity) {
    DynamicArray *arr = (DynamicArray *)malloc(sizeof(DynamicArray));
    arr->data = (int *)malloc(initialCapacity * sizeof(int));
    arr->size = 0;
    arr->capacity = initialCapacity;
    return arr;
}

void addElement(DynamicArray *arr, int element) {
    if (arr->size >= arr->capacity) {
        // 扩展容量
        arr->capacity *= 2;
        arr->data = (int *)realloc(arr->data, arr->capacity * sizeof(int));
    }
    arr->data[arr->size++] = element;
}

void destroyDynamicArray(DynamicArray *arr) {
    free(arr->data);
    free(arr);
}

int main() {
    DynamicArray *arr = createDynamicArray(2);

    addElement(arr, 10);
    addElement(arr, 20);
    addElement(arr, 30); // 触发扩容

    for (int i = 0; i < arr->size; i++) {
        printf("%d ", arr->data[i]); // 输出: 10 20 30
    }
    printf("\n");

    destroyDynamicArray(arr);

    return 0;
}
```

练习2: 内存池实现

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define BLOCK_SIZE 1024

typedef struct MemoryBlock {
    struct MemoryBlock *next;
    char data[BLOCK_SIZE - sizeof(struct MemoryBlock *)];
} MemoryBlock;

typedef struct {
    MemoryBlock *head;
} MemoryPool;

MemoryPool* createMemoryPool() {
    MemoryPool *pool = (MemoryPool *)malloc(sizeof(MemoryPool));
    pool->head = NULL;
    return pool;
}

void* poolAlloc(MemoryPool *pool, size_t size) {
    if (size > BLOCK_SIZE - sizeof(MemoryBlock *)) {
        return NULL; // 超过块大小
    }

    MemoryBlock *block = (MemoryBlock *)malloc(BLOCK_SIZE);
    block->next = pool->head;
    pool->head = block;

    return block->data;
}

void destroyMemoryPool(MemoryPool *pool) {
    MemoryBlock *current = pool->head;
    while (current != NULL) {
        MemoryBlock *next = current->next;
        free(current);
        current = next;
    }
    free(pool);
}

int main() {
    MemoryPool *pool = createMemoryPool();

    char *str = (char *)poolAlloc(pool, 100);
    strcpy(str, "Hello, Memory Pool!");
    printf("%s\n", str);

    destroyMemoryPool(pool);

    return 0;
}

```

```
}
```

练习3: 自定义malloc调试器

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

static size_t totalAllocated = 0;
static size_t totalFreed = 0;

void* debugMalloc(size_t size, const char *file, int line) {
    void *ptr = malloc(size);
    if (ptr != NULL) {
        totalAllocated += size;
        printf("[MALLOC] %s:%d - Allocated %zu bytes at %p\n",
            file, line, size, ptr);
    }
    return ptr;
}

void debugFree(void *ptr, const char *file, int line) {
    if (ptr != NULL) {
        // 简化处理: 实际中需要记录分配大小
        totalFreed += sizeof(ptr);
        printf("[FREE] %s:%d - Freed memory at %p\n", file, line, ptr);
        free(ptr);
    }
}

#define malloc(size) debugMalloc(size, __FILE__, __LINE__)
#define free(ptr) debugFree(ptr, __FILE__, __LINE__)

int main() {
    int *p = (int *)malloc(sizeof(int));
    *p = 42;

    int *arr = (int *)malloc(5 * sizeof(int));

    free(p);
    free(arr);

    printf("Total allocated: %zu bytes\n", totalAllocated);
    printf("Total freed: %zu bytes\n", totalFreed);

    return 0;
}
```

九、总结与延伸

本节重点回顾

1. **内存布局**：栈、堆、数据段、BSS段、代码段
2. **栈**：自动分配释放，大小有限，效率高
3. **堆**：手动分配释放，大小灵活，效率较低
4. **动态分配函数**：`malloc`，`calloc`，`realloc`，`free`
5. **常见问题**：内存泄漏、重复释放、使用已释放内存、缓冲区溢出
6. **最佳实践**：检查返回值、配对使用、使用RAII模式

下节预告

下一篇文章我们将学习《结构体与共用体：自定义数据类型》，深入理解如何创建复杂的数据结构。

📌 知识点卡片

- 栈：自动管理，后进先出，大小有限
- 堆：手动管理，灵活大小，需要配对free
- 内存区域：栈、堆、数据段、BSS段、代码段
- 动态分配：`malloc/calloc/realloc/free`
- 常见问题：内存泄漏、重复释放、野指针

💡 小技巧

- 小数据用栈，大数据用堆
- 始终检查malloc返回值
- 使用宏封装内存分配，便于调试
- 考虑使用内存池减少碎片化

⚠️ 注意事项

- 不要忘记释放堆内存
- 不要重复释放同一内存
- 不要使用已释放的内存
- 注意缓冲区溢出问题

如果你对内存管理有任何疑问，欢迎在评论区留言讨论！下一篇见！

12 结构体与共用体：自定义数据类型

创建复杂的数据结构

在上一篇文章中，我们学习了内存管理的知识。现在，让我们来学习C语言中用于创建复杂数据结构的工具——结构体和共用体。它们允许我们将不同类型的数据组合在一起，创建自定义的数据类型。

一、结构体的概念

1.1 什么是结构体

结构体是一种用户定义的数据类型，可以将不同类型的数据成员组合在一起。

```
#include <stdio.h>
```

```

#include <string.h>

// 定义结构体
struct Person {
    char name[50];
    int age;
    float height;
};

int main() {
    // 声明并初始化结构体变量
    struct Person p1;
    strcpy(p1.name, "张三");
    p1.age = 25;
    p1.height = 1.75f;

    // 访问结构体成员
    printf("姓名: %s\n", p1.name);
    printf("年龄: %d\n", p1.age);
    printf("身高: %.2f\n", p1.height);

    return 0;
}

```

1.2 结构体的定义

```

// 语法
struct 结构体名 {
    数据类型 成员名1;
    数据类型 成员名2;
    // ...
};

// 示例
struct Student {
    int id;
    char name[50];
    float score;
};

struct Point {
    int x;
    int y;
};

```

1.3 结构体的初始化

```

#include <stdio.h>
#include <string.h>

struct Person {
    char name[50];

```

```

    int age;
    float height;
};

int main() {
    // 方式1: 逐个成员初始化
    struct Person p1;
    strcpy(p1.name, "张三");
    p1.age = 25;
    p1.height = 1.75f;

    // 方式2: 使用初始化列表
    struct Person p2 = {"李四", 30, 1.80f};

    // 方式3: 指定成员初始化 (C99标准)
    struct Person p3 = {
        .name = "王五",
        .height = 1.72f,
        .age = 28
    };

    return 0;
}

```

二、结构体的使用

2.1 结构体数组

```

#include <stdio.h>

struct Student {
    int id;
    char name[50];
    float score;
};

int main() {
    // 定义结构体数组
    struct Student students[3] = {
        {101, "张三", 95.5f},
        {102, "李四", 88.0f},
        {103, "王五", 92.3f}
    };

    // 遍历结构体数组
    for (int i = 0; i < 3; i++) {
        printf("学生%d: \n", i + 1);
        printf("  ID: %d\n", students[i].id);
        printf("  姓名: %s\n", students[i].name);
        printf("  成绩: %.1f\n", students[i].score);
    }
}

```

```
    return 0;
}
```

2.2 结构体指针

```
#include <stdio.h>
#include <string.h>

struct Person {
    char name[50];
    int age;
};

int main() {
    struct Person p = {"张三", 25};
    struct Person *ptr = &p;

    // 使用指针访问结构体成员
    printf("姓名: %s\n", ptr->name);
    printf("年龄: %d\n", ptr->age);

    // 修改结构体成员
    strcpy(ptr->name, "李四");
    ptr->age = 30;

    printf("修改后: \n");
    printf("姓名: %s\n", p.name);
    printf("年龄: %d\n", p.age);

    return 0;
}
```

2.3 结构体作为函数参数

```
#include <stdio.h>

struct Point {
    int x;
    int y;
};

// 值传递
void printPoint(struct Point p) {
    printf("Point: (%d, %d)\n", p.x, p.y);
}

// 指针传递
void movePoint(struct Point *p, int dx, int dy) {
    p->x += dx;
    p->y += dy;
}
```

```
int main() {
    struct Point pt = {3, 4};

    printPoint(pt);
    movePoint(&pt, 2, -1);
    printPoint(pt);

    return 0;
}
```

三、嵌套结构体

3.1 结构体嵌套

```
#include <stdio.h>

// 定义日期结构体
struct Date {
    int year;
    int month;
    int day;
};

// 定义学生结构体, 包含日期结构体
struct Student {
    char name[50];
    struct Date birthday;
    float score;
};

int main() {
    struct Student s = {
        "张三",
        {2000, 5, 15},
        95.5f
    };

    printf("姓名: %s\n", s.name);
    printf("生日: %d年%d月%d日\n", s.birthday.year, s.birthday.month, s.birthday.day);
    printf("成绩: %.1f\n", s.score);

    return 0;
}
```

3.2 结构体链表

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// 定义链表节点
```

```

typedef struct Node {
    int data;
    struct Node *next;
} Node;

// 创建新节点
Node* createNode(int data) {
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

// 打印链表
void printList(Node *head) {
    Node *current = head;
    while (current != NULL) {
        printf("%d -> ", current->data);
        current = current->next;
    }
    printf("NULL\n");
}

int main() {
    // 创建链表
    Node *head = createNode(1);
    head->next = createNode(2);
    head->next->next = createNode(3);

    printf("链表: ");
    printList(head);

    return 0;
}

```

四、共用体的概念

4.1 什么是共用体

共用体（Union）是一种特殊的数据类型，所有成员共享同一块内存空间。

```

#include <stdio.h>

// 定义共用体
union Data {
    int i;
    float f;
    char str[20];
};

int main() {
    union Data data;
}

```

```
printf("共用体大小: %zu bytes\n", sizeof(data));

// 使用整型
data.i = 100;
printf("data.i = %d\n", data.i);

// 使用浮点型 (会覆盖之前的值)
data.f = 3.14f;
printf("data.f = %.2f\n", data.f);
printf("data.i = %d\n", data.i); // 值已改变

// 使用字符串
strcpy(data.str, "Hello");
printf("data.str = %s\n", data.str);
printf("data.i = %d\n", data.i); // 值已改变

return 0;
}
```

4.2 共用体的特点

特性	结构体 (struct)	共用体 (union)
内存分配	成员各自独立分配内存	成员共享同一块内存
内存大小	所有成员大小之和	最大成员的大小
成员访问	可以同时访问所有成员	同一时刻只能访问一个成员
用途	表示一个对象的多个属性	节省内存，类型转换

五、typedef的使用

5.1 简化结构体定义

```
#include <stdio.h>

// 使用typedef简化结构体定义
typedef struct {
    char name[50];
    int age;
    float height;
} Person; // Person现在是一个类型名

int main() {
    // 直接使用Person作为类型
    Person p = {"张三", 25, 1.75f};
    printf("姓名: %s, 年龄: %d, 身高: %.2f\n", p.name, p.age, p.height);

    return 0;
}
```

```
}
```

5.2 typedef的常见用法

```
#include <stdio.h>

// 为基本类型创建别名
typedef int Integer;
typedef float Real;

// 为指针类型创建别名
typedef int* IntPtr;
typedef char* String;

// 为数组类型创建别名
typedef int IntArray[10];

int main() {
    Integer num = 42;
    Real pi = 3.14f;
    String str = "Hello";
    IntArray arr = {1, 2, 3, 4, 5};

    printf("num = %d\n", num);
    printf("pi = %.2f\n", pi);
    printf("str = %s\n", str);

    return 0;
}
```

六、位域

6.1 什么是位域

位域允许我们按位来分配结构体成员的内存空间。

```
#include <stdio.h>

// 使用位域定义结构体
struct Flags {
    unsigned int is_active : 1;    // 1位
    unsigned int is_admin : 1;    // 1位
    unsigned int has_access : 1;  // 1位
    unsigned int reserved : 29;   // 29位 (总共32位)
};

int main() {
    struct Flags f;
    printf("Flags大小: %zu bytes\n", sizeof(f));

    f.is_active = 1;
}
```



```

    f.is_admin = 0;
    f.has_access = 1;

    printf("is_active = %u\n", f.is_active);
    printf("is_admin = %u\n", f.is_admin);
    printf("has_access = %u\n", f.has_access);

    return 0;
}

```

6.2 位域的应用场景

```

#include <stdio.h>

// 颜色表示 (使用位域)
struct Color {
    unsigned int red : 8;
    unsigned int green : 8;
    unsigned int blue : 8;
    unsigned int alpha : 8;
};

int main() {
    struct Color c = {255, 128, 64, 255};

    printf("红色: %u\n", c.red);
    printf("绿色: %u\n", c.green);
    printf("蓝色: %u\n", c.blue);
    printf("透明度: %u\n", c.alpha);

    // 将结构体转换为32位整数
    unsigned int color_value = *(unsigned int *)&c;
    printf("颜色值: 0x%08X\n", color_value);

    return 0;
}

```

七、枚举类型

7.1 什么是枚举

枚举 (Enumeration) 是一种用户定义的整数类型，用于表示一组命名常量。

```

#include <stdio.h>

// 定义枚举类型
enum Weekday {
    Monday,    // 0
    Tuesday,   // 1
    Wednesday, // 2
    Thursday,  // 3
}

```

```

    Friday,    // 4
    Saturday,  // 5
    Sunday     // 6
};

int main() {
    enum Weekday today = Wednesday;

    printf("今天是星期%d\n", today);

    // switch语句中使用枚举
    switch (today) {
        case Monday:
        case Tuesday:
        case Wednesday:
        case Thursday:
        case Friday:
            printf("工作日\n");
            break;
        case Saturday:
        case Sunday:
            printf("周末\n");
            break;
    }

    return 0;
}

```

7.2 枚举的自定义值

```

#include <stdio.h>

// 自定义枚举值
enum Status {
    ERROR = -1,
    SUCCESS = 0,
    WARNING = 1,
    INFO = 2
};

// 带位运算的枚举
enum Permission {
    READ = 1 << 0,    // 0001
    WRITE = 1 << 1,    // 0010
    EXECUTE = 1 << 2 // 0100
};

int main() {
    printf("ERROR = %d\n", ERROR);
    printf("SUCCESS = %d\n", SUCCESS);

    // 使用位运算组合权限
    int permissions = READ | WRITE;
}

```

```
printf("权限组合: %d\n", permissions);

if (permissions & READ) {
    printf("有读权限\n");
}

return 0;
}
```

八、实战练习

练习1: 学生信息管理系统

```
#include <stdio.h>
#include <string.h>

#define MAX_STUDENTS 100

struct Student {
    int id;
    char name[50];
    float score;
};

void addStudent(struct Student students[], int *count) {
    if (*count >= MAX_STUDENTS) {
        printf("学生数量已达上限\n");
        return;
    }

    struct Student s;
    printf("请输入学生ID: ");
    scanf("%d", &s.id);
    printf("请输入学生姓名: ");
    scanf("%s", s.name);
    printf("请输入学生成绩: ");
    scanf("%f", &s.score);

    students[*count] = s;
    (*count)++;

    printf("学生信息添加成功\n");
}

void printStudents(struct Student students[], int count) {
    printf("\n学生列表: \n");
    printf("ID\t姓名\t成绩\n");
    for (int i = 0; i < count; i++) {
        printf("%d\t%s\t%.1f\n", students[i].id, students[i].name, students[i].score);
    }
}
```

```

int main() {
    struct Student students[MAX_STUDENTS];
    int count = 0;
    int choice;

    do {
        printf("\n学生信息管理系统\n");
        printf("1. 添加学生\n");
        printf("2. 显示学生列表\n");
        printf("3. 退出\n");
        printf("请选择: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                addStudent(students, &count);
                break;
            case 2:
                printStudents(students, count);
                break;
            case 3:
                printf("退出系统\n");
                break;
            default:
                printf("无效选择\n");
        }
    } while (choice != 3);

    return 0;
}

```

练习2: 二叉树节点结构

```

#include <stdio.h>
#include <stdlib.h>

typedef struct TreeNode {
    int data;
    struct TreeNode *left;
    struct TreeNode *right;
} TreeNode;

TreeNode* createNode(int data) {
    TreeNode *node = (TreeNode *)malloc(sizeof(TreeNode));
    node->data = data;
    node->left = NULL;
    node->right = NULL;
    return node;
}

void inorderTraversal(TreeNode *root) {
    if (root == NULL) return;
    inorderTraversal(root->left);
}

```

```

    printf("%d ", root->data);
    inorderTraversal(root->right);
}

int main() {
    // 创建二叉树
    TreeNode *root = createNode(1);
    root->left = createNode(2);
    root->right = createNode(3);
    root->left->left = createNode(4);
    root->left->right = createNode(5);

    printf("中序遍历: ");
    inorderTraversal(root);
    printf("\n");

    return 0;
}

```

练习3: 共用体实现类型转换

```

#include <stdio.h>

union FloatBits {
    float f;
    unsigned int i;
};

void printFloatBits(float num) {
    union FloatBits fb;
    fb.f = num;

    printf("浮点数: %.6f\n", num);
    printf("二进制表示: ");

    for (int j = 31; j >= 0; j--) {
        printf("%d", (fb.i >> j) & 1);
        if (j == 31 || j == 23) printf(" ");
    }
    printf("\n");
}

int main() {
    printFloatBits(3.14f);
    printFloatBits(-2.5f);
    printFloatBits(0.0f);

    return 0;
}

```

九、总结与延伸

本节重点回顾

1. **结构体**：将不同类型数据组合在一起的自定义类型
2. **结构体定义**：`struct 结构体名 { 成员列表 };`
3. **结构体访问**：使用 `.` 运算符，指针使用 `->` 运算符
4. **结构体数组**：存储多个结构体变量
5. **嵌套结构体**：结构体中包含另一个结构体
6. **共用体**：成员共享同一块内存空间
7. **typedef**：为类型创建别名
8. **位域**：按位分配内存空间
9. **枚举**：表示一组命名常量

下节预告

下一篇文章我们将学习《文件操作：数据的持久化存储》，深入理解如何读写文件。

📌 知识点卡片

- 结构体：`struct Name { 成员列表 };`
- 结构体访问：`.` 和 `->`
- 共用体：成员共享内存，同一时刻只能访问一个成员
- typedef：为类型创建别名
- 位域：按位分配内存
- 枚举：一组命名常量

💡 小技巧

- 使用typedef简化结构体定义
- 结构体作为函数参数时，通常使用指针传递
- 共用体可用于类型转换
- 位域适合存储标志位和状态信息
- 枚举提高代码可读性

⚠️ 注意事项

- 共用体成员共享内存，修改一个会影响其他成员
- 结构体大小可能因对齐而大于成员大小之和
- 枚举本质上是整数类型
- 位域的位顺序依赖于系统架构

如果你对结构体和共用体有任何疑问，欢迎在评论区留言讨论！下一篇见！

13 文件操作：数据的持久化存储

将数据保存到磁盘

在上一篇文章中，我们学习了结构体和共用体的使用。现在，让我们来学习C语言中的文件操作。文件操作允许我们将数据持久化存储到磁盘上，实现数据的长期保存和读取。

一、文件操作的基本概念

1.1 什么是文件

文件是存储在外部存储设备上的数据集合。在C语言中，文件分为两种类型：

文件类型	描述	特点
文本文件	以字符形式存储	可读，占用空间较大
二进制文件	以二进制形式存储	高效，占用空间较小

1.2 文件指针

```
#include <stdio.h>

int main() {
    // 声明文件指针
    FILE *fp;

    // 打开文件
    fp = fopen("example.txt", "w");

    if (fp == NULL) {
        printf("文件打开失败\n");
        return 1;
    }

    // 写入内容
    fprintf(fp, "Hello, File!\n");

    // 关闭文件
    fclose(fp);

    return 0;
}
```

1.3 文件打开模式

模式	描述
"r"	只读方式打开文本文件
"w"	只写方式打开文本文件，覆盖原有内容
"a"	追加方式打开文本文件
"r+"	读写方式打开文本文件
"w+"	读写方式创建新文件
"a+"	读写方式打开文件，追加模式
"rb"	只读方式打开二进制文件
"wb"	只写方式打开二进制文件
"ab"	追加方式打开二进制文件

二、文件的打开与关闭

2.1 fopen函数

```
#include <stdio.h>

int main() {
    FILE *fp;

    // 打开文件
    fp = fopen("test.txt", "w");

    // 检查是否成功打开
    if (fp == NULL) {
        printf("文件打开失败! \n");
        perror("原因");
        return 1;
    }

    printf("文件打开成功! \n");

    // 关闭文件
    fclose(fp);

    return 0;
}
```

2.2 fclose函数

```
#include <stdio.h>
```



```

int main() {
    FILE *fp = fopen("data.txt", "w");

    if (fp == NULL) {
        perror("文件打开失败");
        return 1;
    }

    // 使用文件...

    // 关闭文件
    if (fclose(fp) != 0) {
        printf("文件关闭失败! \n");
        return 1;
    }

    printf("文件关闭成功! \n");

    return 0;
}

```

三、文件的读写操作

3.1 字符读写

```

#include <stdio.h>

int main() {
    FILE *fp;
    char ch;

    // 写入文件
    fp = fopen("chars.txt", "w");
    if (fp == NULL) {
        perror("打开文件失败");
        return 1;
    }

    for (ch = 'A'; ch <= 'Z'; ch++) {
        fputc(ch, fp);
    }
    fclose(fp);

    // 读取文件
    fp = fopen("chars.txt", "r");
    if (fp == NULL) {
        perror("打开文件失败");
        return 1;
    }

    while ((ch = fgetc(fp)) != EOF) {

```

```

        printf("%c ", ch);
    }
    printf("\n");
    fclose(fp);

    return 0;
}

```

3.2 字符串读写

```

#include <stdio.h>

int main() {
    FILE *fp;
    char buffer[100];

    // 写入字符串
    fp = fopen("strings.txt", "w");
    if (fp == NULL) {
        perror("打开文件失败");
        return 1;
    }

    fputs("Hello, World!\n", fp);
    fputs("这是第二行.\n", fp);
    fclose(fp);

    // 读取字符串
    fp = fopen("strings.txt", "r");
    if (fp == NULL) {
        perror("打开文件失败");
        return 1;
    }

    while (fgets(buffer, sizeof(buffer), fp) != NULL) {
        printf("%s", buffer);
    }
    fclose(fp);

    return 0;
}

```

3.3 格式化读写

```

#include <stdio.h>

struct Student {
    char name[50];
    int age;
    float score;
};

```

```

int main() {
    struct Student s = {"张三", 25, 95.5f};
    FILE *fp;

    // 写入格式化数据
    fp = fopen("student.txt", "w");
    if (fp == NULL) {
        perror("打开文件失败");
        return 1;
    }

    fprintf(fp, "%s %d %.1f\n", s.name, s.age, s.score);
    fclose(fp);

    // 读取格式化数据
    struct Student s2;
    fp = fopen("student.txt", "r");
    if (fp == NULL) {
        perror("打开文件失败");
        return 1;
    }

    fscanf(fp, "%s %d %f", s2.name, &s2.age, &s2.score);
    printf("姓名: %s, 年龄: %d, 成绩: %.1f\n", s2.name, s2.age, s2.score);
    fclose(fp);

    return 0;
}

```

四、二进制文件操作

4.1 二进制写入

```

#include <stdio.h>

struct Student {
    char name[50];
    int age;
    float score;
};

int main() {
    struct Student students[3] = {
        {"张三", 25, 95.5f},
        {"李四", 23, 88.0f},
        {"王五", 24, 92.3f}
    };

    FILE *fp = fopen("students.dat", "wb");
    if (fp == NULL) {
        perror("打开文件失败");
        return 1;
    }

```

```

}

// 写入二进制数据
fwrite(students, sizeof(struct Student), 3, fp);
fclose(fp);

printf("二进制文件写入成功! \n");

return 0;
}

```

4.2 二进制读取

```

#include <stdio.h>

struct Student {
    char name[50];
    int age;
    float score;
};

int main() {
    struct Student students[3];

    FILE *fp = fopen("students.dat", "rb");
    if (fp == NULL) {
        perror("打开文件失败");
        return 1;
    }

    // 读取二进制数据
    fread(students, sizeof(struct Student), 3, fp);
    fclose(fp);

    // 输出数据
    for (int i = 0; i < 3; i++) {
        printf("学生%d: \n", i + 1);
        printf("  姓名: %s\n", students[i].name);
        printf("  年龄: %d\n", students[i].age);
        printf("  成绩: %.1f\n", students[i].score);
    }

    return 0;
}

```

五、文件定位

5.1 fseek函数

```

#include <stdio.h>

```

```

int main() {
    FILE *fp = fopen("example.txt", "w+");
    if (fp == NULL) {
        perror("打开文件失败");
        return 1;
    }

    // 写入内容
    fprintf(fp, "Hello, World!");

    // 将文件指针移动到文件开头
    fseek(fp, 0, SEEK_SET);

    // 读取内容
    char buffer[20];
    fgets(buffer, sizeof(buffer), fp);
    printf("读取内容: %s\n", buffer);

    // 将文件指针移动到第7个字符
    fseek(fp, 6, SEEK_SET);
    fgets(buffer, sizeof(buffer), fp);
    printf("从第7个字符开始: %s\n", buffer);

    // 将文件指针向前移动5个字符
    fseek(fp, -5, SEEK_CUR);
    fgets(buffer, sizeof(buffer), fp);
    printf("向前移动5个字符: %s\n", buffer);

    fclose(fp);

    return 0;
}

```

5.2 ftell函数

```

#include <stdio.h>

int main() {
    FILE *fp = fopen("example.txt", "r");
    if (fp == NULL) {
        perror("打开文件失败");
        return 1;
    }

    // 获取当前位置
    long pos = ftell(fp);
    printf("初始位置: %ld\n", pos);

    // 读取一些内容
    char buffer[10];
    fgets(buffer, sizeof(buffer), fp);

    // 获取当前位置

```

```
pos = ftell(fp);
printf("读取后的位置: %ld\n", pos);

fclose(fp);

return 0;
}
```

5.3 rewind函数

```
#include <stdio.h>

int main() {
    FILE *fp = fopen("example.txt", "r");
    if (fp == NULL) {
        perror("打开文件失败");
        return 1;
    }

    // 读取第一行
    char buffer[50];
    fgets(buffer, sizeof(buffer), fp);
    printf("第一行: %s", buffer);

    // 回到文件开头
    rewind(fp);

    // 再次读取第一行
    fgets(buffer, sizeof(buffer), fp);
    printf("再次读取第一行: %s", buffer);

    fclose(fp);

    return 0;
}
```

六、文件操作的常见函数

6.1 常用函数汇总

函数	功能	示例
<code>fopen</code>	打开文件	<code>fp = fopen("file.txt", "r");</code>
<code>fclose</code>	关闭文件	<code>fclose(fp);</code>
<code>fgetc</code>	读取一个字符	<code>ch = fgetc(fp);</code>
<code>fputc</code>	写入一个字符	<code>fputc('A', fp);</code>
<code>fgets</code>	读取一行字符串	<code>fgets(buf, size, fp);</code>
<code>fputs</code>	写入一行字符串	<code>fputs("hello", fp);</code>
<code>fscanf</code>	格式化读取	<code>fscanf(fp, "%d", &num);</code>
<code>fprintf</code>	格式化写入	<code>fprintf(fp, "%d", num);</code>
<code>fread</code>	二进制读取	<code>fread(buf, size, count, fp);</code>
<code>fwrite</code>	二进制写入	<code>fwrite(buf, size, count, fp);</code>
<code>fseek</code>	文件定位	<code>fseek(fp, offset, origin);</code>
<code>ftell</code>	获取位置	<code>pos = ftell(fp);</code>
<code>rewind</code>	回到开头	<code>rewind(fp);</code>
<code>feof</code>	判断文件结束	<code>while (!feof(fp))</code>

七、文件操作的错误处理

7.1 检查文件是否存在

```
#include <stdio.h>

int fileExists(const char *filename) {
    FILE *fp = fopen(filename, "r");
    if (fp != NULL) {
        fclose(fp);
        return 1;
    }
    return 0;
}

int main() {
    if (fileExists("example.txt")) {
        printf("文件存在\n");
    } else {
        printf("文件不存在\n");
    }
}
```

```
    return 0;
}
```

7.2 错误处理示例

```
#include <stdio.h>

int main() {
    FILE *fp = fopen("nonexistent.txt", "r");

    if (fp == NULL) {
        perror("无法打开文件");
        printf("错误代码: %d\n", ferror(fp));
        return 1;
    }

    // 检查是否到达文件末尾
    if (feof(fp)) {
        printf("已到达文件末尾\n");
    }

    fclose(fp);

    return 0;
}
```

八、实战练习

练习1: 文件复制

```
#include <stdio.h>

int main() {
    FILE *source, *dest;
    char ch;

    source = fopen("source.txt", "r");
    if (source == NULL) {
        perror("无法打开源文件");
        return 1;
    }

    dest = fopen("dest.txt", "w");
    if (dest == NULL) {
        perror("无法打开目标文件");
        fclose(source);
        return 1;
    }

    // 逐字符复制
```



```

while ((ch = fgetc(source)) != EOF) {
    fputc(ch, dest);
}

printf("文件复制成功! \n");

fclose(source);
fclose(dest);

return 0;
}

```

练习2: 统计文件行数

```

#include <stdio.h>

int countLines(const char *filename) {
    FILE *fp = fopen(filename, "r");
    if (fp == NULL) {
        perror("无法打开文件");
        return -1;
    }

    int count = 0;
    char ch;

    while ((ch = fgetc(fp)) != EOF) {
        if (ch == '\n') {
            count++;
        }
    }

    // 如果文件不以换行结尾, 最后一行也算
    if (count > 0 || !feof(fp)) {
        count++;
    }

    fclose(fp);
    return count;
}

int main() {
    int lines = countLines("example.txt");
    if (lines >= 0) {
        printf("文件共有%d行\n", lines);
    }

    return 0;
}

```

练习3: 学生信息管理系统 (带文件存储)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_STUDENTS 100

typedef struct {
    int id;
    char name[50];
    float score;
} Student;

void saveStudents(Student students[], int count) {
    FILE *fp = fopen("students.dat", "wb");
    if (fp == NULL) {
        perror("保存失败");
        return;
    }

    fwrite(students, sizeof(Student), count, fp);
    fclose(fp);
    printf("保存成功! \n");
}

int loadStudents(Student students[]) {
    FILE *fp = fopen("students.dat", "rb");
    if (fp == NULL) {
        return 0;
    }

    int count = 0;
    while (fread(&students[count], sizeof(Student), 1, fp) == 1) {
        count++;
    }

    fclose(fp);
    return count;
}

int main() {
    Student students[MAX_STUDENTS];
    int count = loadStudents(students);

    // 添加新学生
    strcpy(students[count].name, "新学生");
    students[count].id = count + 1;
    students[count].score = 85.5f;
    count++;

    saveStudents(students, count);

    return 0;
}
```

九、总结与延伸

本节重点回顾

1. **文件类型**：文本文件和二进制文件
2. **文件指针**：`FILE *` 类型
3. **文件打开**：`fopen(filename, mode)`
4. **文件关闭**：`fclose(fp)`
5. **字符读写**：`fgetc` 和 `fputc`
6. **字符串读写**：`fgets` 和 `fputs`
7. **格式化读写**：`fscanf` 和 `fprintf`
8. **二进制读写**：`fread` 和 `fwrite`
9. **文件定位**：`fseek`、`ftell`、`rewind`
10. **错误处理**：检查返回值、使用 `perror`

下节预告

下一篇文章我们将学习《综合实战：实现一个简易计算器》，将前面所学知识综合应用。

📌 知识点卡片

- 文件类型：文本文件（可读）、二进制文件（高效）
- 文件指针：`FILE *fp`
- 打开模式：`r`、`w`、`a`、`rb`、`wb`、`ab` 等
- 字符读写：`fgetc`、`fputc`
- 字符串读写：`fgets`、`fputs`
- 格式化读写：`fscanf`、`fprintf`
- 二进制读写：`fread`、`fwrite`
- 文件定位：`fseek(fp, offset, SEEK_SET/SEEK_CUR/SEEK_END)`

💡 小技巧

- 始终检查 `fopen` 的返回值
- 成对使用 `fopen` 和 `fclose`
- 使用 `perror` 打印错误信息
- 二进制文件适合存储结构体数据
- 使用 `feof` 判断文件是否结束

⚠️ 注意事项

- 文件操作完成后必须关闭文件
- 注意文件路径的正确性
- 二进制文件和文本文件不能混用
- `fgets` 会读取换行符，需要注意处理

如果你对文件操作有任何疑问，欢迎在评论区留言讨论！下一篇见！

14 综合实战：实现一个简易计算器

将知识融会贯通

在前面的文章中，我们学习了C语言的各种基础知识。现在，让我们将这些知识综合运用起来，实现一个简易计算器程序。这个项目将涵盖变量、函数、指针、数组、字符串处理、流程控制等多个知识点。

一、需求分析

1.1 功能需求

我们的计算器需要支持以下功能：

功能	说明	示例
基本运算	加、减、乘、除	2 + 3, 10 * 5
取模运算	求余数	10 % 3
表达式计算	支持括号和运算符优先级	(2 + 3) * 4
连续计算	支持链式运算	5 + 3 * 2 - 1
结果复用	用上一次结果继续计算	ANS + 5

1.2 技术要求

- 使用栈数据结构处理表达式
- 实现运算符优先级
- 支持错误处理
- 提供友好的用户界面

二、设计思路

2.1 整体架构

计算器主程序
1. 输入模块：读取用户输入的表达式
2. 词法分析：将表达式拆分为token
3. 中缀转后缀：转换为逆波兰表达式
4. 后缀求值：计算最终结果
5. 输出模块：显示计算结果

2.2 数据结构设计

```
// 运算符优先级
int priority(char op) {
```

```

        switch(op) {
            case '(': return 0;
            case '+': case '-': return 1;
            case '*': case '/': case '%': return 2;
            default: return -1;
        }
    }
}

// 栈结构
typedef struct {
    double data[100];
    int top;
} Stack;

```

三、核心代码实现

3.1 栈操作函数

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <ctype.h>

#define MAX_LEN 100

typedef struct {
    double data[MAX_LEN];
    int top;
} NumStack;

typedef struct {
    char data[MAX_LEN];
    int top;
} OpStack;

// 数字栈操作
void numPush(NumStack *s, double val) {
    if (s->top < MAX_LEN - 1) {
        s->data[++(s->top)] = val;
    }
}

double numPop(NumStack *s) {
    if (s->top >= 0) {
        return s->data[(s->top)--];
    }
    return 0;
}

// 运算符栈操作
void opPush(OpStack *s, char op) {

```

```

    if (s->top < MAX_LEN - 1) {
        s->data[++(s->top)] = op;
    }
}

char opPop(OpStack *s) {
    if (s->top >= 0) {
        return s->data[(s->top)--];
    }
    return '\0';
}

char opPeek(OpStack *s) {
    if (s->top >= 0) {
        return s->data[s->top];
    }
    return '\0';
}

```

3.2 运算符优先级判断

```

int getPriority(char op) {
    switch(op) {
        case '+':
        case '-':
            return 1;
        case '*':
        case '/':
        case '%':
            return 2;
        case '(':
            return 0;
        default:
            return -1;
    }
}

int isOperator(char ch) {
    return ch == '+' || ch == '-' || ch == '*' || ch == '/' || ch == '%';
}

```

3.3 中缀表达式转后缀表达式

```

void infixToPostfix(char *infix, char *postfix) {
    OpStack opStack = {-1};
    int i = 0, j = 0;

    while (infix[i] != '\0') {
        // 跳过空格
        if (infix[i] == ' ') {
            i++;
            continue;
        }
    }
}

```

```

    }

    // 数字
    if (isdigit(infix[i]) || infix[i] == '.') {
        while (isdigit(infix[i]) || infix[i] == '.') {
            postfix[j++] = infix[i++];
        }
        postfix[j++] = ' ';
    }

    // 左括号
    else if (infix[i] == '(') {
        opPush(&opStack, infix[i]);
        i++;
    }

    // 右括号
    else if (infix[i] == ')') {
        while (opStack.top >= 0 && opPeek(&opStack) != '(') {
            postfix[j++] = opPop(&opStack);
            postfix[j++] = ' ';
        }
        opPop(&opStack); // 弹出左括号
        i++;
    }

    // 运算符
    else if (isOperator(infix[i])) {
        while (opStack.top >= 0 && getPriority(opPeek(&opStack)) >= getPriority(infix[i])) {
            postfix[j++] = opPop(&opStack);
            postfix[j++] = ' ';
        }
        opPush(&opStack, infix[i]);
        i++;
    }
    else {
        i++;
    }
}

// 弹出剩余运算符
while (opStack.top >= 0) {
    postfix[j++] = opPop(&opStack);
    postfix[j++] = ' ';
}

postfix[j] = '\0';
}

```

3.4 后缀表达式求值

```

double evaluatePostfix(char *postfix) {
    NumStack numStack = {-1};
    int i = 0;

    while (postfix[i] != '\0') {

```

```

// 跳过空格
if (postfix[i] == ' ') {
    i++;
    continue;
}

// 数字
if (isdigit(postfix[i]) || postfix[i] == '.') {
    double num = 0.0;
    int decimal = 0;
    double factor = 0.1;

    while (isdigit(postfix[i]) || postfix[i] == '.') {
        if (postfix[i] == '.') {
            decimal = 1;
            i++;
            continue;
        }
        if (decimal) {
            num += (postfix[i] - '0') * factor;
            factor *= 0.1;
        } else {
            num = num * 10 + (postfix[i] - '0');
        }
        i++;
    }
    numPush(&numStack, num);
}

// 运算符
else if (isOperator(postfix[i])) {
    double b = numPop(&numStack);
    double a = numPop(&numStack);
    double result;

    switch(postfix[i]) {
        case '+': result = a + b; break;
        case '-': result = a - b; break;
        case '*': result = a * b; break;
        case '/':
            if (b == 0) {
                printf("错误: 除数不能为零! \n");
                return NAN;
            }
            result = a / b;
            break;
        case '%':
            if (b == 0) {
                printf("错误: 模数不能为零! \n");
                return NAN;
            }
            result = (int)a % (int)b;
            break;

        default: result = 0;
    }
}

```



```

    }
    numPush(&numStack, result);
    i++;
}
else {
    i++;
}
}

return numPop(&numStack);
}

```

四、主程序实现

4.1 主函数

```

int main() {
    char expression[MAX_LEN];
    char postfix[MAX_LEN * 2];
    double result = 0;
    int choice;

    printf("=====\n");
    printf("          简易计算器 v1.0\n");
    printf("=====\n");
    printf("支持的运算符: + - * / %\n");
    printf("支持括号: ()\n");
    printf("输入 ANS 使用上一次结果\n");
    printf("输入 quit 退出程序\n");
    printf("=====\n\n");

    while (1) {
        printf("> ");
        fgets(expression, MAX_LEN, stdin);

        // 移除换行符
        expression[strcspn(expression, "\n")] = '\0';

        // 退出命令
        if (strcmp(expression, "quit") == 0) {
            printf("感谢使用计算器, 再见! \n");
            break;
        }

        // 空输入
        if (strlen(expression) == 0) {
            continue;
        }

        // 处理 ANS
        char processed[MAX_LEN];
        char *p = expression;
    }
}

```

```

char *q = processed;
while (*p != '\0') {
    if (strncmp(p, "ANS", 3) == 0) {
        sprintf(q, "%.6f", result);
        q += strlen(q);
        p += 3;
    } else {
        *q++ = *p++;
    }
}
*q = '\0';

// 转换为后缀表达式
infixToPostfix(processed, postfix);

// 计算结果
result = evaluatePostfix(postfix);

if (isnan(result)) {
    continue;
}

printf("= %.6f\n\n", result);
}

return 0;
}

```

五、完整代码

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <math.h>
#include <ctype.h>

#define MAX_LEN 100

typedef struct {
    double data[MAX_LEN];
    int top;
} NumStack;

typedef struct {
    char data[MAX_LEN];
    int top;
} OpStack;

void numPush(NumStack *s, double val) {
    if (s->top < MAX_LEN - 1) {
        s->data[++(s->top)] = val;
    }
}

```

```

    }
}

double numPop(NumStack *s) {
    if (s->top >= 0) {
        return s->data[(s->top)--];
    }
    return 0;
}

void opPush(OpStack *s, char op) {
    if (s->top < MAX_LEN - 1) {
        s->data[++(s->top)] = op;
    }
}

char opPop(OpStack *s) {
    if (s->top >= 0) {
        return s->data[(s->top)--];
    }
    return '\0';
}

char opPeek(OpStack *s) {
    if (s->top >= 0) {
        return s->data[s->top];
    }
    return '\0';
}

int getPriority(char op) {
    switch(op) {
        case '+':
        case '-':
            return 1;
        case '*':
        case '/':
        case '%':
            return 2;
        case '(':
            return 0;
        default:
            return -1;
    }
}

int isOperator(char ch) {
    return ch == '+' || ch == '-' || ch == '*' || ch == '/' || ch == '%';
}

void infixToPostfix(char *infix, char *postfix) {
    OpStack opStack = {-1};

    int i = 0, j = 0;

```

```

while (infix[i] != '\0') {
    if (infix[i] == ' ') {
        i++;
        continue;
    }

    if (isdigit(infix[i]) || infix[i] == '.') {
        while (isdigit(infix[i]) || infix[i] == '.') {
            postfix[j++] = infix[i++];
        }
        postfix[j++] = ' ';
    }
    else if (infix[i] == '(') {
        opPush(&opStack, infix[i]);
        i++;
    }
    else if (infix[i] == ')') {
        while (opStack.top >= 0 && opPeek(&opStack) != '(') {
            postfix[j++] = opPop(&opStack);
            postfix[j++] = ' ';
        }
        opPop(&opStack);
        i++;
    }
    else if (isOperator(infix[i])) {
        while (opStack.top >= 0 && getPriority(opPeek(&opStack)) >= getPriority(infix[i])) {
            postfix[j++] = opPop(&opStack);
            postfix[j++] = ' ';
        }
        opPush(&opStack, infix[i]);
        i++;
    }
    else {
        i++;
    }
}

while (opStack.top >= 0) {
    postfix[j++] = opPop(&opStack);
    postfix[j++] = ' ';
}

postfix[j] = '\0';
}

double evaluatePostfix(char *postfix) {
    NumStack numStack = {-1};
    int i = 0;

    while (postfix[i] != '\0') {
        if (postfix[i] == ' ') {
            i++;

```

```

        continue;
    }

    if (isdigit(postfix[i]) || postfix[i] == '.') {
        double num = 0.0;
        int decimal = 0;
        double factor = 0.1;

        while (isdigit(postfix[i]) || postfix[i] == '.') {
            if (postfix[i] == '.') {
                decimal = 1;
                i++;
                continue;
            }
            if (decimal) {
                num += (postfix[i] - '0') * factor;
                factor *= 0.1;
            } else {
                num = num * 10 + (postfix[i] - '0');
            }
            i++;
        }
        numPush(&numStack, num);
    }

    else if (isOperator(postfix[i])) {
        double b = numPop(&numStack);
        double a = numPop(&numStack);
        double result;

        switch(postfix[i]) {
            case '+': result = a + b; break;
            case '-': result = a - b; break;
            case '*': result = a * b; break;
            case '/':
                if (b == 0) {
                    printf("错误: 除数不能为零! \n");
                    return NAN;
                }
                result = a / b;
                break;
            case '%':
                if (b == 0) {
                    printf("错误: 模数不能为零! \n");
                    return NAN;
                }
                result = (int)a % (int)b;
                break;
            default: result = 0;
        }
        numPush(&numStack, result);
        i++;
    }

    else {

```

```

        i++;
    }
}

return numPop(&numStack);
}

int main() {
    char expression[MAX_LEN];
    char postfix[MAX_LEN * 2];
    double result = 0;

    printf("=====\n");
    printf("          简易计算器 v1.0\n");
    printf("=====\n");
    printf("支持的运算符: + - * / %\n");
    printf("支持括号: ()\n");
    printf("输入 ANS 使用上一次结果\n");
    printf("输入 quit 退出程序\n");
    printf("=====\n\n");

    while (1) {
        printf("> ");
        fgets(expression, MAX_LEN, stdin);

        expression[strcspn(expression, "\n")] = '\0';

        if (strcmp(expression, "quit") == 0) {
            printf("感谢使用计算器, 再见! \n");
            break;
        }

        if (strlen(expression) == 0) {
            continue;
        }

        char processed[MAX_LEN];
        char *p = expression;
        char *q = processed;
        while (*p != '\0') {
            if (strncmp(p, "ANS", 3) == 0) {
                sprintf(q, "%.6f", result);
                q += strlen(q);
                p += 3;
            } else {
                *q++ = *p++;
            }
        }
        *q = '\0';

        infixToPostfix(processed, postfix);

        result = evaluatePostfix(postfix);
    }
}

```

```
        if (isnan(result)) {
            continue;
        }

        printf("= %.6f\n\n", result);
    }

    return 0;
}
```

六、编译与运行

6.1 编译命令

```
gcc calculator.c -o calculator -lm
```

6.2 运行示例

```
=====
          简易计算器 v1.0
=====
支持的运算符: + - * / %
支持括号: ( )
输入 ANS 使用上一次结果
输入 quit 退出程序
=====

> 2 + 3 * 4
= 14.000000

> ANS + 5
= 19.000000

> (10 + 5) * 2
= 30.000000

> 10 % 3
= 1.000000

> 10 / 0
错误: 除数不能为零!

> quit
感谢使用计算器, 再见!
```

七、功能扩展

7.1 添加更多数学函数

```
// 在后缀表达式求值中添加
else if (postfix[i] == 's' && strncmp(postfix+i, "sin", 3) == 0) {
    double x = numPop(&numStack);
    numPush(&numStack, sin(x));
    i += 3;
}
else if (postfix[i] == 'c' && strncmp(postfix+i, "cos", 3) == 0) {
    double x = numPop(&numStack);
    numPush(&numStack, cos(x));
    i += 3;
}
```

7.2 添加变量支持

```
// 添加变量存储
double variables[26] = {0}; // a-z

// 在主程序中处理变量赋值
if (strchr(expression, '=') != NULL) {
    char var = expression[0];
    if (var >= 'a' && var <= 'z') {
        // 提取等号右边的表达式
        char *expr = strchr(expression, '=') + 1;
        // 计算并赋值
        infixToPostfix(expr, postfix);
        variables[var - 'a'] = evaluatePostfix(postfix);
        printf("%c = %.6f\n\n", var, variables[var - 'a']);
        continue;
    }
}
```

八、总结与延伸

本节重点回顾

1. **数据结构**：使用栈处理表达式求值
2. **算法**：中缀表达式转后缀表达式（逆波兰表达式）
3. **运算符优先级**：正确处理加减乘除的优先级
4. **错误处理**：处理除零错误
5. **用户体验**：支持ANSI复用、友好的界面

下节预告

下一篇文章我们将学习《C语言学习路线与资源推荐》，总结学习方法和优质资源。

📌 知识点卡片

- 栈：先进后出的数据结构
- 中缀表达式：日常使用的表达式形式
- 后缀表达式：适合计算机计算的形式

- 运算符优先级：乘除高于加减
- 逆波兰表达式：后缀表达式的另一种称呼

💡 小技巧

- 使用栈可以很好地处理表达式求值
- 中缀转后缀是经典的编译原理算法
- 错误处理很重要，避免程序崩溃

⚠️ 注意事项

- 注意除零错误的处理
- 括号必须成对出现
- 输入验证很重要

如果你对计算器的实现有任何疑问，欢迎在评论区留言讨论！下一篇见！

15 C语言学习路线与资源推荐

踏上编程之路

在前面的文章中，我们系统地学习了C语言的基础知识。现在，让我们来总结一下学习方法和推荐的学习资源，帮助你更好地掌握C语言并继续进阶。

一、学习路线规划

1.1 入门阶段（第1-2周）

目标：建立编程思维，掌握基本概念

学习内容	时间安排	关键技能
开发环境搭建	1天	安装编译器、配置IDE
基本数据类型	2天	int、float、char等
变量与常量	1天	变量声明、初始化
运算符与表达式	2天	算术、关系、逻辑运算
第一个程序	1天	Hello World

关键产出：

- 能够编写简单的计算程序
- 理解编译运行流程

1.2 基础阶段（第3-4周）

目标：掌握流程控制和函数

学习内容	时间安排	关键技能
条件语句	2天	if-else、switch
循环结构	3天	for、while、do-while
函数定义与调用	3天	参数传递、返回值
数组与字符串	3天	数组遍历、字符串处理

关键产出：

- 能够编写结构化程序
- 实现简单的算法（排序、查找）

1.3 进阶阶段（第5-6周）

目标：掌握指针和内存管理

学习内容	时间安排	关键技能
指针基础	3天	地址、指针变量、解引用
指针与数组	2天	数组名即指针、指针运算
动态内存分配	3天	malloc、calloc、realloc、free
结构体与共用体	3天	自定义数据类型

关键产出：

- 理解内存布局
- 实现复杂数据结构（链表、栈、队列）

1.4 高级阶段（第7-8周）

目标：掌握文件操作和综合应用

学习内容	时间安排	关键技能
文件操作	3天	fopen、fread、fwrite、fclose
标准库函数	2天	stdio、string、stdlib
综合项目实战	5天	完整程序开发

关键产出：

- 开发完整的应用程序
 - 理解程序架构设计
-

二、推荐学习资源

2.1 书籍推荐

书名	作者	推荐指数	适合阶段
《C程序设计语言》	Brian Kernighan & Dennis Ritchie	★★★★★	入门到进阶
《C Primer Plus》	Stephen Prata	★★★★★	入门
《C和指针》	Kenneth Reek	★★★★★	进阶
《深入理解计算机系统》	Randal Bryant	★★★★☆	高级
《C陷阱与缺陷》	Andrew Koenig	★★★★☆	进阶

2.2 在线资源

学习网站：

- C语言中文网：<https://c.biancheng.net/>
- 菜鸟教程：<https://www.runoob.com/cprogramming/c-tutorial.html>
- LeetCode：<https://leetcode-cn.com/>（算法练习）
- HackerRank：<https://www.hackerrank.com/domains/c>

视频教程：

- B站：C语言入门到精通：<https://www.bilibili.com/>
- Coursera：C Programming：<https://www.coursera.org/>

2.3 开发工具

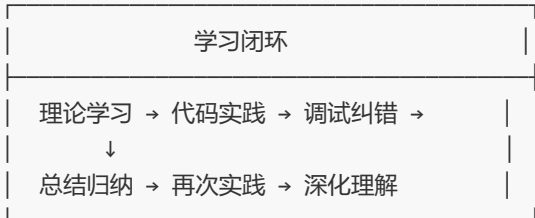
工具	类型	推荐度
VS Code	编辑器	★★★★★
CLion	IDE	★★★★☆
Code::Blocks	IDE	★★★★☆
GCC	编译器	★★★★★
GDB	调试器	★★★★☆

2.4 练习平台

平台	特点	适合人群
POJ	算法竞赛	进阶学习者
HDU	ACM练习	竞赛选手
Codeforces	国际竞赛	高级学习者
牛客网	面试题库	求职准备

三、学习方法与技巧

3.1 高效学习方法



3.2 每日学习清单

任务	时间	说明
理论学习	30分钟	阅读教程或书籍
代码练习	60分钟	编写或调试代码
算法练习	30分钟	解决一个小问题
总结笔记	15分钟	记录学到的知识

3.3 调试技巧

- 使用printf调试**：在关键位置输出变量值
- 使用调试器**：设置断点、单步执行
- 检查返回值**：验证函数返回是否正确
- 边界测试**：测试特殊情况和边界值

3.4 常见错误排查

错误类型	常见原因	排查方法
编译错误	语法错误	查看错误提示行
运行崩溃	空指针、越界	添加调试输出
逻辑错误	算法错误	逐步验证逻辑
内存泄漏	未释放内存	使用工具检测

四、常见问题解答

4.1 Q&A

Q1: 如何记住那么多语法?

A: 不需要死记硬背，通过大量练习自然会记住。重点是理解原理，而不是记忆语法。

Q2: 为什么要学指针?

A: 指针是C语言的灵魂，掌握指针才能真正理解内存管理和高效编程。

Q3: 学习C语言需要数学基础吗?

A: 基本的数学知识就足够了，但如果要深入算法，需要一定的数学基础。

Q4: 学完C语言后学什么?

A: 可以学习数据结构、算法、操作系统，或者转向 [C++](#)、Python 等其他语言。

Q5: 如何提高编程能力?

A: 多练习、多阅读优秀代码、参与开源项目、解决实际问题。

五、进阶学习方向

5.1 深入底层

C语言 → 汇编语言 → 计算机组成原理 → 操作系统

5.2 应用开发

C语言 → [C++](../cpp/) → Qt/GTK → 图形界面开发

5.3 系统编程

C语言 → Linux系统编程 → 网络编程 → 嵌入式开发

5.4 进阶学习路径

方向	推荐学习内容
系统编程	Linux系统调用、进程管理、线程同步
网络编程	Socket编程、HTTP协议、TCP/IP
嵌入式	MCU开发、驱动程序、RTOS
高性能计算	并行编程、SIMD优化、GPU编程
安全领域	逆向工程、漏洞分析、安全编程

六、学习项目推荐

6.1 入门项目

- 简易计算器**：实现基本运算和表达式求值
- 学生管理系统**：结构体、文件操作
- 猜数字游戏**：随机数生成、流程控制
- 通讯录管理**：链表操作、文件存储

6.2 进阶项目

- 贪吃蛇游戏**：图形界面、键盘控制
- HTTP服务器**：网络编程、协议解析
- 内存分配器**：实现简易malloc/free
- 数据库引擎**：B树实现、索引管理

6.3 开源项目

项目	难度	学习价值
Redis	中高	数据结构、内存管理
Nginx	高	网络编程、架构设计
SQLite	中高	数据库原理
Linux内核	极高	操作系统原理

七、总结与寄语

7.1 学习总结

- 打好基础**：掌握变量、函数、指针等核心概念
- 多写代码**：实践是最好的学习方式
- 阅读源码**：学习优秀代码的风格和思路
- 参与社区**：加入技术社区，交流学习

7.2 学习心态

- **耐心**：编程需要时间积累，不要急于求成
- **坚持**：每天进步一点点，积少成多
- **好奇心**：保持对技术的热情和好奇心
- **不怕错误**：错误是学习的最好机会

7.3 最后的建议

"Talk is cheap, show me the code." — Linus Torvalds

学习编程最重要的是**动手实践**。看完教程后，一定要自己动手写代码，解决实际问题。

附录：常用代码模板

A.1 主函数模板

```
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

A.2 文件操作模板

```
#include <stdio.h>

int main() {
    FILE *fp = fopen("file.txt", "r");
    if (fp == NULL) {
        perror("Failed to open file");
        return 1;
    }

    // 读取或写入操作

    fclose(fp);
    return 0;
}
```

A.3 动态数组模板

```
#include <stdlib.h>

int* createArray(int size) {
    int *arr = (int *)malloc(size * sizeof(int));
    return arr;
}
```

专栏完结

恭喜你完成了本专栏的学习！C语言是编程的基石，掌握它将为你打开计算机科学的大门。

专栏文章列表：

序号	文章标题
1	《C语言入门：为什么它是编程的基石》
2	《C语言开发环境搭建：从安装到Hello World》
3	《Hello World背后：C语言程序的执行原理》
4	《数据类型与变量：编程的基本积木》
5	《运算符与表达式：程序的计算逻辑》
6	《流程控制：让程序学会做选择》
7	《循环结构：重复执行的艺术》
8	《数组与字符串：批量数据处理》
9	《函数：代码复用的利器》
10	《指针：C语言的灵魂》
11	《内存管理：堆与栈的奥秘》
12	《结构体与共用体：自定义数据类型》
13	《文件操作：数据的持久化存储》
14	《综合实战：实现一个简易计算器》
15	《C语言学习路线与资源推荐》

愿你在编程的道路上越走越远！🚀

如果你对C语言学习有任何疑问，欢迎在评论区留言讨论！感谢你的阅读！

💡 学习建议

每日学习时间建议

时间段	活动	时长
上午	学习新知识、观看教学视频	2-3小时
下午	编写代码、完成作业练习	2-3小时
晚上	复习总结、阅读参考资料	1小时

学习方法

1. **动手实践**：学习编程最好的方式是动手写代码，不要只看不动手
2. **调试能力**：学会在 Dev-C++ 中设断点、单步执行，观察变量如何变化
3. **阅读代码**：阅读优秀的代码，学习他人的编程风格
4. **记录笔记**：记录重要知识点和遇到的问题
5. **交流讨论**：遇到问题不要独自埋头苦思，积极与同学交流

常见问题解决

- **编译错误**：仔细阅读 Dev-C++ 下方编译日志，根据行号检查语法和拼写
- **运行错误**：使用调试功能逐步执行，观察变量值的变化
- **逻辑错误**：画出流程图，分析程序的执行逻辑
- **环境问题**：回到 Day 1，检查编译器是否被正确检测到，项目/文件是否保存为 `.c`

学习目标检验

第一周检验

1. 能否用 Dev-C++ 独立编写、编译并运行一个C程序？
2. 能否使用变量、运算符和表达式进行计算，并理解常见易错点（如 `=` 与 `==`）？
3. 能否使用if-else和switch实现条件判断？
4. 能否使用while、do-while和for实现循环？
5. 能否完成每天的基础练习，并尝试至少一道挑战题？

第二周检验

1. 能否使用数组存储和处理批量数据？
2. 能否使用字符串处理函数？
3. 能否定义和调用函数？
4. 能否使用指针进行内存操作？
5. 能否使用结构体创建复杂数据结构？
6. 能否完成综合实战项目？

推荐学习资源

书籍

1. 《C程序设计语言》（K&R）- C语言经典教材
2. 《C Primer Plus》- 适合初学者的入门教材
3. 《C和指针》- 深入理解指针的权威著作

在线资源

1. [C语言中文网](#) - 中文教程
2. [LeetCode](#) - 编程练习平台
3. [CSDN](#) - 技术社区

视频教程

1. B站搜索"C语言教程"
2. MOOC平台的C语言课程

加入学习交流群

扫码加入 **C 语言学习交流群**，一起答疑、打卡、交流作业思路：

群聊: C 语言学习交流群



该二维码 7 天内 (8月2日前) 有效, 重新进入将更新

说明: 微信群二维码通常有时效 (图中标注 7 天内有效)。若过期无法加入, 请重新获取最新二维码。

祝你学习愉快! 编程之路从这里开始, 加油! 📌